

Plateforme de gestion de rendez-vous médicaux

Projet réalisé dans le cadre de la présentation au titre professionnel

Développeur Web et Web Mobile

présenté par:

Yassine ECHCHOUROUQ
AFPA 2025/2026

SOMMAIRE

SOMMAIRE **2**
INTRODUCTION **4**
COMPÉTENCES PRATIQUÉES SUR LE PROJET **5**
1. DÉVELOPPER LE BACK-END DE L'APPLICATION 5
1.1. Création de la base de données 5
1.1.1. Les relations 5
1.1.2. Contraintes d'intégrité 5
1.1.3. Convention 5
1.2. Développer les composants d'accès aux données 6
1.2.1. Choisir et mettre en place le stack technique 6
1.2.2. La couche d'accès aux données 6
1.2.3. Gestion des fichiers: UPLOAD 7
1.3. Développer la couche de sécurité et applicatif 8
1.4. La logique métier et développement des fonctionnalités 9
2. DÉVELOPPER LE FRONT-END DE L'APPLICATION 10
2.1. Maquetter une application 10
2.1.1. Partir du Use Case et du besoin du client 10
2.1.2. Concevoir une expérience utilisateur optimisée 10
2.1.3. Concevoir pour tous les écrans 11
2.2. Développer les composants UI/UX 14
2.2.1. Le stack technique 14
2.2.2. Une approche par composants 14
2.3. Réaliser une interface Web dynamique 15
2.3.1. Une application monopage pilotée par l'état 15
2.3.2. Échanges avec l'API et l'affichage des données 15
PRÉSENTATION DU PROJET **16**
1. CAHIER DES CHARGES ET UML 16
1.1. Cahier des charges 16
1.2. UML 18
1.2.1. Acteurs et Use Case 18
1.2.2. MCD et DB Diagram 20
2. GIT ET AGILITÉ 21
2.1. Git 21
2.2. La méthode Agile 23
3. LE WORKFLOW 24
4. LOUPE TECHNIQUE 26
4.1. Architecture générale 26
4.2. Environnement de développement 26
4.3. Base de données 27
4.4. Conception du Back-end API REST 29
4.4.1. Convention 29
4.4.2. Routes 32
4.4.3. DTO 35

4.4.4. Authentification....38
4.5. Conception du Front-end React....42
4.5.1. Convention....42
4.5.2. Toubib....43
4.5.3. ToubibBO: BackOffice....46
5. PRÉSENTATION DU JEU D'ESSAI....50
5.1. Réserver un Rendez-vous....50
5.2. Responsivité....58
5.3. Accessibilité....62
6. VULNÉRABILITÉS DE SÉCURITÉ ET VEILLE....62
6.1. Veille technologique....62
6.1.1. PHP....62
6.1.2. Symfony....63
6.1.3. React....63
6.2. Veille de sécurité....63
6.2.1. Symfony/LexikJWT....63
6.2.2. Symfony/Doctrine....64
6.2.3. FTP....64
6.3. Processus de recherche....65
CONCLUSION....66
SOURCES ET BIBLIOGRAPHIE....67
ANNEXE....68

INTRODUCTION

Depuis l'obtention de mon diplôme de développeur informatique en 2015, j'ai toujours conservé un intérêt particulier pour le développement et les nouvelles technologies. J'ai eu l'occasion d'exercer le métier de développeur pendant trois ans, avant de m'orienter vers un autre domaine professionnel. Malgré ce changement, le développement informatique est toujours resté présent dans mon parcours, notamment à travers une pratique personnelle et régulière qui m'a permis de maintenir mes connaissances.

En 2022, j'ai rejoint Genilink filiale du groupe SGS en tant que technicien, l'entreprise est spécialisée dans la conception de solution digitale. Cette expérience m'a permis de retrouver un environnement technique et de participer à différents projets au sein de l'entreprise. Au fil du temps, mes différentes participations, les feedbacks que j'ai pu apporter ainsi que les solutions que j'ai proposées et mises en place ont progressivement fait évoluer mes responsabilités. Cette dynamique m'a naturellement conduit à revenir vers le développement informatique au sein de l'entreprise.

Afin de concrétiser cette démarche, il était nécessaire pour moi de remettre à niveau mes connaissances et de me familiariser avec les technologies et les pratiques actuelles du développement. Le métier ayant considérablement évolué depuis, j'ai entrepris une année sabbatique de formation en alternance dans mon entreprise avec l'AFPA, ce qui m'a permis de consolider mes bases, d'acquérir de nouvelles compétences et de renouer pleinement avec le développement dans un contexte professionnel, cette démarche m'a permis de mettre à jour mes connaissances tout en développant ma capacité à apprendre avec mon formateur et aussi de manière autonome, une compétence qui me paraît essentielle dans un domaine en constante évolution.

Au cours de cette année, j'ai eu l'opportunité de participer pleinement à la vie d'une équipe de développement. Au-delà des tâches techniques qui m'ont été confiées, j'ai pris part aux différents backlogs et aux rituels de l'équipe, ce qui m'a permis de mieux comprendre l'organisation d'un projet informatique et les méthodes de travail collaboratives. Cette expérience m'a particulièrement intéressé, car elle m'a permis de compléter mon expérience technique par une véritable expérience du travail en équipe et de la gestion collective d'un projet.

Dans le cadre de la formation, j'ai réalisé le projet **`Toubib`**, une plateforme de gestion de rendez-vous médicaux, il m'a permis de mettre en pratique les compétences acquises ou actualisées durant cette année, tout en travaillant sur les autres projets de l'entreprise. Il a également constitué une occasion de mettre en application les méthodes de développement, l'organisation du travail et les bonnes pratiques découvertes au cours de ma formation et de mon expérience en entreprise.

Cette année représente ainsi une étape importante dans mon parcours professionnel. Elle m'a permis de faire le lien entre mon expérience initiale de développeur, mon parcours dans l'IT et mon souhait de retrouver pleinement le métier du développement informatique.

COMPÉTENCES PRATIQUÉES SUR LE PROJET

1. DÉVELOPPER LE BACK-END DE L'APPLICATION

1.1. Création de la base de données

La création de la base de données de l'application m'a permis de mettre en pratique la conception d'un modèle relationnel complet, du choix technique jusqu'aux règles d'intégrité. La persistance du projet repose sur une pile open source, entièrement conteneurisée. Le serveur de base de données est MySQL 8.0 (en local il est exécuté dans le conteneur `toubib_db`), donc le choix du SGBD MySQL 8 s'explique par son support natif de l'Unicode complet indispensable pour les noms et les textes en français et en arabe, par ses performances en lecture pour un usage transactionnel, et par sa large adoption dans l'écosystème PHP.

1.1.1. Les relations

En concevant le modèle, j'ai appris à identifier la nature de chaque lien entre les données et à le traduire en relation. J'ai mis en œuvre quatre familles de relations, toutes matérialisées par des clés étrangères et contrôlées par le moteur InnoDB.

La première famille regroupe les relations OneToMany (un-à-plusieurs) les plus fréquentes :

- un utilisateur possède plusieurs rendez-vous, possède plusieurs messages envoyés et reçus, plusieurs notifications, et plusieurs jetons de rafraîchissement
- une région regroupe plusieurs cabinets
- une spécialité regroupe plusieurs médecins
- un médecin rédige plusieurs ordonnances et reçoit plusieurs avis

La deuxième famille est la OneToOne (relation un-à-un):

- un utilisateur peut être médecin, ce qui fait d'un utilisateur un médecin sans dupliquer l'identité.

La troisième famille est l'association ManyToMany (plusieurs-à-plusieurs):

- un médecin peut exercer sur plusieurs cabinets et un cabinet peut contenir plusieurs médecins

La quatrième famille concerne les relations optionnelles, ce sont les relations avec une cardinalité minimale à zéro :

- le médecin traitant d'un patient
- le rattachement facultatif d'une ordonnance, d'un avis ou d'un message à un rendez-vous

1.1.2. Contraintes d'intégrité

J'ai appris à choisir la règle de suppression de chaque relation selon la nature du lien, plutôt que d'appliquer une règle unique. Les règles de suppression sont choisies selon la nature du lien. Les données qui n'ont de sens que rattachées à un parent (jetons, réinitialisations, cartes bancaires, liens médecin–cabinet, historique) sont supprimées en cascade, le médecin traitant d'un patient est remis à *NULL* si le médecin est supprimé. Les rendez-vous, en revanche, sont protégés par défaut : leurs clés étrangères sont obligatoires, afin qu'un rendez-vous ne puisse pas perdre son patient, son médecin ou son cabinet.

1.1.3. Convention

Pour que la base reste lisible et cohérente, j'ai appliqué des conventions de nommage strictes. La base compte donc 18 tables métier et une table technique. Elles se répartissent en cinq domaines

fonctionnels, les clés primaires sont de type entier auto-incrémenté (`id`), et les clés étrangères portent le nom de la relation suffixé par `_id` par exemple `doctor_id`, à l'exception des colonnes de jointure explicitement nommées. Les contraintes d'unicité métier sont portées par la base elle-même : l'adresse e-mail, le numéro de téléphone et le numéro de sécurité sociale des utilisateurs, ainsi que le numéro d'ordre (`license_number`) des médecins

1.2. Développer les composants d'accès aux données

1.2.1. Choisir et mettre en place le stack technique

J'ai développé l'API de l'application, ToubibAPI, avec Symfony 8 et PHP 8.4. Ce projet m'a permis d'approfondir mes connaissances sur le framework structuré autour de composants réutilisables et de conventions éprouvées. J'ai choisi Symfony pour trois raisons principales :

- La demande sur le marché du travail, et plus précisément dans mon entreprise, où PHP et Symfony sont utilisés.
- Son architecture cadrée, qui impose une séparation claire des responsabilités entre les contrôleurs, les services, les entités et les dépôts de données. Cette organisation rend le code lisible, testable et facile à faire évoluer, ce qui est essentiel pour une API destinée à alimenter plusieurs applications.
- Ses composants intégrés couvrent la plupart des besoins d'une API sans avoir à tout réécrire : la sécurité et les pare-feux, la validation des données, la sérialisation en JSON ou encore l'envoi de courriels. La documentation officielle est complète, et la communauté, très active, garantit des ressources à tous les niveaux.

Autour de Symfony, j'ai appris à utiliser Doctrine, l'ORM (*Object-Relational Mapping*) qui fait le lien entre les objets PHP et la base de données. Chaque table est représentée par une entité, et les requêtes sont écrites dans des dépôts (*repositories*), ce qui évite d'écrire du SQL dispersé dans le code et protège contre les injections SQL grâce aux requêtes préparées. Les données sont stockées dans une base MySQL utilisant le moteur InnoDB, qui assure l'intégrité des relations grâce aux clés étrangères. En développement, la base tourne dans un conteneur Docker, ce qui me garantit un environnement identique d'une machine à l'autre. L'authentification repose sur des jetons JWT, transmis dans des cookies sécurisés et complétés par un système de refresh token, ce qui permet à l'API de rester sans état côté serveur. Enfin, j'ai organisé les échanges avec les clients au moyen de DTO (*Data Transfer Objects*), qui décrivent précisément les données attendues en entrée et sont validés avant tout traitement. L'API constitue ainsi le point central du projet : c'est la seule brique qui accède à la base de données, et elle sert les deux applications React, Toubib et ToubibB0, ainsi que de futurs clients éventuels, comme une application mobile.

1.2.2. La couche d'accès aux données

La couche d'accès aux données repose sur trois dépendances : DBAL 4.4 gère la connexion à la base et l'exécution des requêtes SQL, ORM 3.6 assure le mapping objet-relationnel, et Doctrine Migrations Bundle 4.0 versionne les évolutions du schéma de la base, ce qui m'a permis d'apprendre à construire la couche d'accès aux données avec l'écosystème Doctrine intégré à Symfony.

Cette couche est organisée en dépôts (*repositories*), un par entité persistée, soit dix-huit classes regroupées dans le dossier src/Repository/. Toutes héritent de ServiceEntityRepository, la classe de base fournie par Doctrine pour Symfony. Elle reçoit le ManagerRegistry par injection dans son constructeur, associe le dépôt à son entité et fournit les méthodes génériques find, findBy, findOneBy et

findAll. En regroupant toutes les requêtes dans cette couche, je maintiens les contrôleurs à l'écart du SQL : ils demandent des objets métier sans savoir comment ceux-ci sont récupérés. Je peux ainsi modifier une requête sans toucher aux endroits où elle est appelée.

Chaque dépôt suit le même gabarit. Les méthodes add() et remove() encapsulent respectivement persist() et remove() de l'EntityManager, avec un paramètre $flush à false par défaut. L'écriture en base n'est donc pas déclenchée immédiatement, ce qui permet d'intégrer ces opérations dans un traitement plus large et de tout enregistrer en une seule fois.

Pour les requêtes métier, j'utilise le QueryBuilder, qui permet de composer une requête par étapes. Les champs y sont désignés dans le vocabulaire des entités (a.patient, a.startTime) plutôt que par les noms de colonnes, et Doctrine se charge de la traduction en SQL grâce aux métadonnées du mapping. Surtout, toutes les valeurs variables sont transmises par setParameter() et ne sont jamais concaténées dans la requête : elles sont envoyées séparément sous forme de requête préparée, ce qui écarte tout risque d'injection SQL. Par exemple, la méthode findFutureAppointmentsByPatient() renvoie les rendez-vous à venir d'un patient, triés par ordre chronologique, à l'aide d'une simple condition comparant a.startTime à la date courante.

L'exemple ci-dessous compare une requête SQL écrite à la main avec son équivalent construit par le QueryBuilder. La méthode findByUserId() récupère le profil d'un médecin à partir de l'identifiant de son compte utilisateur, avec ses cabinets et sa spécialité en une seule requête.

Requête SQL brute (risque d'injection si userId est concaténé dans la chaîne) :

```sql
SELECT
    d.*,
    dbs.*,
    bs.*,
    s.*
FROM doctors d
INNER JOIN user u
    ON u.id = d.user_id
LEFT JOIN doctor_business_sites dbs
    ON dbs.doctor_id = d.id
LEFT JOIN business_site bs
    ON bs.id = dbs.business_site_id
LEFT JOIN speciality s
    ON s.id = d.speciality_id
WHERE u.id = [userId]; /* ←PARAM*/
```

SQL Query (HIGH SQL INJECT RISK)

```php
// @Param int $userId
// @return Doctors|null
public function findByUserId(int $userId): ?Doctors {
    return $this->createQueryBuilder('d')
        ->join('d.user', 'u')
        ->andWhere('u.id = :userId')
        ->setParameter('userId', $userId) /* ←PARAM*/
        ->leftJoin('d.doctorBusinessSites', 'dbs')
        ->addSelect('dbs')
        ->leftJoin('dbs.businessSite', 'bs')
        ->addSelect('bs')
        ->leftJoin('d.speciality', 's')
        ->addSelect('s')
        ->getQuery()
        ->getOneOrNullResult();
}
```

QueryBuilder (SAFE SQL Prepared Statements)

1.2.3. Gestion des fichiers: UPLOAD

J'ai appris à gérer l'envoi de fichiers en appliquant le même principe que pour la base de données: les fichiers sont stockés en dehors de l'API, sur un serveur de fichiers dédié accessible en FTP. L'API ne conserve aucun fichier sur son propre disque, elle joue le rôle d'intermédiaire entre l'application cliente et ce serveur.

1.3. Développer la couche de sécurité et applicatif

La première couche de sécurité est bien les règles CORS, elle permettra d'identifier l'émetteur de la demande, et lui autoriser l'accès ou refuser selon les règles CORS imposées, ensuite viennent les couches suivantes:

J'ai utilisé une authentification par jetons avec LexikJWTAuthenticationBundle Symfony dependency. À la connexion, le serveur génère un jeton d'accès signé avec une clé privée RSA, dont la *passphrase* est fournie par une variable d'environnement, et d'une durée de vie d'une heure.

Ce jeton est transmis dans un cookie `app_auth` marqué `HttpOnly`, `Secure`, de sorte que le code JavaScript du navigateur ne puisse pas le lire et que le cookie ne circule que sur HTTPS. Le bundle accepte également le jeton dans l'en-tête `Authorization: Bearer`, ce qui permet d'utiliser l'API depuis des clients non navigateurs. Un second jeton, le jeton de rafraîchissement est généré aléatoirement, enregistré en base avec une date d'expiration de sept jours, et placé dans un cookie `app_refresh` distinct. Lorsque le jeton d'accès expire, le client appelle `POST /api/auth/refresh` pour en obtenir un nouveau. À chaque connexion, tous les jetons de rafraîchissement précédents de l'utilisateur sont révoqués (`revokeAllByUser`) : une seule session active est ainsi garantie par compte. La déconnexion révoque le jeton courant et supprime les cookies en les expirant.

Le firewall `api` est configuré en mode sans état (`stateless: true`) : aucune session serveur n'est créée, chaque requête est authentifiée par son jeton. Les deux points d'entrée de connexion distinguent les profils. `/api/auth/login` sert aux patients, tandis que `/api/auth/doctor/login` vérifie en plus que le rôle est `ROLE_DOCTOR` ou `ROLE_ADMIN`.

L'autorisation est appliquée à deux niveaux:
Le premier niveau est déclaratif, dans `config/packages/security.yaml`. Une liste ordonnée de règles `access_control` associe des expressions de chemin à des rôles : l'espace `/api/admin` exige `ROLE_ADMIN`, les routes de recherche, la fiche publique d'un médecin et les créneaux disponibles sont accessibles sans authentification, et le reste de `/api/` exige un utilisateur authentifié (`IS_AUTHENTICATED_FULLY`). Les requêtes `OPTIONS` sont laissées publiques afin que les pré-requêtes CORS aboutissent.

Le second niveau est fondé sur la propriété des données et c'est lui qui protège les espaces médecin. Un Controller médecin commence par retrouver le profil `Doctors` lié à l'utilisateur connecté ; si ce profil n'existe pas, la requête s'arrête avec une erreur 404. Un patient ne peut donc pas agir sur l'espace médecin, même s'il est authentifié. Les opérations sur un patient vérifient en outre que ce patient est bien rattaché au médecin, via le médecin traitant (`mainDoctor`) : la lecture de l'historique ou la modification d'un patient d'un autre médecin renvoie « Patient introuvable ». Les rendez-vous suivent le même principe, en comparant le médecin du rendez-vous à celui de l'utilisateur connecté avant toute modification de statut.

Les mots de passe ne sont jamais stockés en clair. Le hachage est délégué au hasheur de Symfony, configuré en mode `auto`, qui choisit l'algorithme le plus robuste disponible et permet de faire évoluer ce choix sans recalculer les mots de passe existants. Le service `AuthValidatorService` impose une politique minimale à l'inscription : au moins huit caractères, une majuscule, une minuscule, un chiffre et un caractère spécial. Il vérifie également l'unicité de l'adresse e-mail et la longueur du prénom et du nom.

Les contraintes Symfony Validator complètent ces règles : l'e-mail, le téléphone et le numéro de sécurité sociale sont déclarés uniques au niveau de l'entité, et les Controllers appellent les DTO (exemple: `LoginRequest`) et valident la forme des champs avant tout traitement.

Les entités déclarent les champs visibles par la sérialisation au moyen de groupes (`user:read`, `doctor:read`, `doctor:search`, `businesssite:read`, `patient_history:read`). Un champ sans groupe n'est jamais renvoyé par l'API. Cette approche « liste blanche » protège les données sensibles, comme le mot de passe haché ou le jeton de réinitialisation, et évite les cycles de sérialisation entre entités liées.

La connexion est protégée par `LoggingSecurityService`, qui s'appuie sur la table `logging_attempt`. Chaque échec de connexion, ou requête mal formée, y est enregistré avec l'adresse e-mail, l'adresse IP et l'agent utilisateur. Avant toute vérification du mot de passe, le service compte les tentatives récentes pour l'adresse IP et pour l'adresse e-mail, sur des fenêtres de temps configurables (par défaut 30 minutes pour l'IP et 5 minutes pour l'e-mail), et refuse la connexion avec une réponse 429 lorsque le seuil est atteint. Après une connexion réussie, les tentatives associées à l'utilisateur et à l'IP sont purgées. Les seuils sont injectés depuis le fichier `.env`, ce qui permet d'ajuster la politique sans modifier le code.

1.4. La logique métier et développement des fonctionnalités

Inspiré des projets sur lesquels j’ai travaillé en entreprise, j’ai appliqué les mêmes à mon projet.

Il met ainsi en relation trois catégories d’acteurs :

- le patient, qui peut créer un compte et le vérifier par courriel, rechercher un médecin par nom, spécialité ou localisation, consulter ses disponibilités, réserver un rendez-vous et suivre la liste de ses rendez-vous.
- le médecin, qui exerce dans un ou plusieurs cabinets, gère son cabinet et son agenda, confirme ou annule les rendez-vous et suit ses patients au moyen de dossiers et d'un historique médical.
- l'administrateur, qui supervise l'ensemble de l’application.

Selon la convention que j’ai mise en place sur ce projet Symfony, les règles métier ne sont pas dans les Controllers, les règles sont globales et utilisées dans plusieurs endroits, elles sont donc stockées dans les services de l'API, par exemple `AppointmentsService` pour le calcul des créneaux et les services du dossier `Auth` pour la gestion des comptes. Il y a aussi les Helpers qui contiennent des fonctions de traitement et/ou formatage de données.

Les Controllers orchestrent les Repositories, les Services et les Helpers. Pour prendre une décision le Controller interroge le Service dédié pour apprendre la logique métier, ensuite il reçoit les données depuis les Repositories, à ce stade la décision peut être prise, en cas de besoin un Helper est appelé, et enfin retourne la réponse en format JSON.

Avec cette approche j’obtiens plusieurs couches distinctes, ce qui permettra de tester les règles par les Tests Unitaires depuis les Controllers de test.

2. DÉVELOPPER LE FRONT-END DE L'APPLICATION

2.1. Maquetter une application

2.1.1. Partir du Use Case et du besoin du client

Je n'ai pas commencé le maquettage de **`Toubib`** par le dessin des écrans, mais par une lecture des recherches de spécialiste en UX, l'analyse des cas d'utilisation et du besoin. Avant de concevoir la moindre interface, j'ai cherché à comprendre ce que l'utilisateur (acteur) attendait de la plateforme afin d'exercer ses rôles et actions: permettre à des patients de trouver un médecin et de réserver un rendez-vous en ligne, et offrir aux médecins un outil pour gérer leurs cabinets, leur agenda et leurs patients. J'ai formalisé ce besoin sous forme de fonctionnalités, puis de cas d'utilisation, qui m'ont servi de référence tout au long de la conception.

Cette étape m'a permis d'identifier les différents profils d'utilisateurs et leurs attentes, qui ne sont pas les mêmes. Le patient recherche la simplicité et la rapidité : il veut trouver un praticien disponible et réserver en quelques clics, souvent depuis son téléphone. Le médecin utilise l'outil au quotidien dans un cadre professionnel ; il a besoin d'une vue claire de son agenda, d'un accès rapide aux dossiers de ses patients et de formulaires efficaces pour configurer ses horaires et ses tarifs. C'est cette distinction qui m'a conduit à découper le projet en deux applications Front-end : `Toubib`, l'interface destinée aux patients, et `ToubibBO` (BackOffice), l'espace professionnel des médecins.

J'ai fait le choix de séparer le front-end en deux applications distinctes, `Toubib` pour les patients et `ToubibBO` pour les médecins, pour trois raisons:

La première tient aux responsabilités : les deux publics n'ont ni les mêmes besoins ni les mêmes usages. Le patient effectue des actions ponctuelles et simples, comme rechercher un médecin ou réserver un rendez-vous, alors que le médecin utilise un outil de travail quotidien, avec un agenda, des dossiers patients et la configuration de ses cabinets. Réunir ces deux logiques dans une seule application aurait multiplié les conditions d'affichage selon le rôle et rendu le code plus difficile à lire et à maintenir.

La deuxième raison relève de la sécurité. En isolant l'espace professionnel, je m'assure que le code des fonctionnalités médecin n'est jamais livré dans le navigateur d'un patient, ce qui réduit la surface exposée et ne révèle pas la structure des écrans réservés aux praticiens. Cette séparation se retrouve dans l'authentification, puisque l'espace médecin passe par une route de connexion dédiée qui n'accepte que les rôles `ROLE_DOCTOR` et `ROLE_ADMIN`. Je reste toutefois conscient que cette séparation ne remplace pas les contrôles côté serveur : c'est l'API qui vérifie les droits à chaque requête, le découpage du front ajoutant une couche de cloisonnement supplémentaire.

La troisième raison concerne le volume du projet. Avec une quinzaine de cas d'utilisation répartis entre trois acteurs, une application unique aurait rapidement atteint une taille importante, au détriment des performances et de la lisibilité. Deux applications plus légères se chargent plus vite, notamment sur mobile pour les patients, et peuvent évoluer et être déployées indépendamment, chacune dans son propre dépôt Git, sans qu'une modification de l'espace médecin risque de perturber le parcours des patients.

2.1.2. Concevoir une expérience utilisateur optimisée

L'expérience utilisateur a guidé l'ensemble de mes choix de conception. Le premier principe que j'ai retenu est la clarté : chaque écran met en avant une action principale, signalée par un bouton visible, tandis que les actions secondaires restent accessibles sans encombrer l'interface.

J'ai par exemple placé la barre de recherche au centre de la page d'accueil, car c'est le point de départ de la majorité des visites.

Le second principe est la cohérence. J'ai appliqué les mêmes règles de couleurs, de typographies, de boutons et de formulaires sur l'ensemble des écrans, afin que l'utilisateur retrouve ses repères d'une page à l'autre. J'ai associé les statuts des rendez-vous (planifié, confirmé, annulé) à des codes visuels constants, pour qu'ils soient compris d'un coup d'œil.

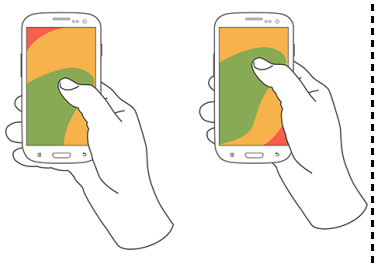
Je tenais à ce que l'utilisateur sache toujours ce qui se passe : un indicateur de chargement s'affiche pendant les requêtes, les champs obligatoires sont signalés, et les messages d'erreur expliquent concrètement le problème, par exemple les critères de mot de passe qui ne sont pas encore remplis. Enfin, j'ai pris en compte l'accessibilité, avec des contrastes suffisants, des tailles de texte lisibles et des zones cliquables assez grandes pour une utilisation au doigt.

2.1.3. Concevoir pour tous les écrans

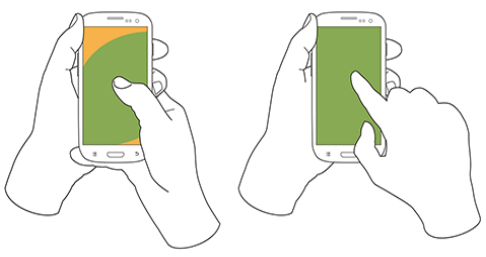
Pour une application WEB et WEB Mobile, j'ai pensé mes maquettes selon une approche mobile first : j'ai conçu l'écran mobile en premier, avec l'essentiel du contenu, puis j'ai enrichi la mise en page pour les tablettes et les ordinateurs. Cette démarche m'a obligé à hiérarchiser l'information et m'a permis d'obtenir des interfaces plus épurées pour Toubib, mais ToubibBO l'espace médecin, est utilisé sur ordinateur au cabinet, donc je n'ai pas rendu ToubibBO responsive.

Le pouce remplace le curseur, et la manière dont l'utilisateur tient son téléphone détermine les zones qu'il atteint facilement. J'en ai tiré plusieurs règles concrètes. J'ai placé les actions principales, comme la validation d'un créneau ou la confirmation d'une réservation, dans les zones naturellement accessibles au pouce. J'ai prévu des zones cliquables suffisamment grandes et espacées pour éviter les erreurs de saisie, en particulier sur la grille des créneaux horaires, où les boutons sont nombreux et proches. Enfin, je n'ai fait reposer aucune information importante sur le survol, qui n'existe pas sur un écran tactile. Ces principes ont rendu la prise de rendez-vous plus confortable sur smartphone, où j'imagine qu'une grande partie des usages des patients se concentre.

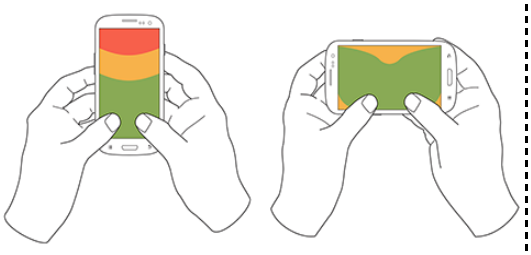
Afin de concevoir l'interface mobile, je me suis appuyé sur le livre *Design tactile* de Josh CLARK, qui s'appuie lui-même sur une étude de terrain menée en 2013 par le chercheur Steven HOOBER. Celui-ci a observé plus de 1 300 personnes en train d'utiliser leur téléphone dans la rue, et a relevé trois façons principales de le tenir : 49 % des personnes le tiennent et l'utilisent d'une seule main, 36 % le tiennent d'une main et touchent l'écran avec l'autre, et 15 % le tiennent à deux mains en tapant avec les deux pouces.



One-Handed Use 49%



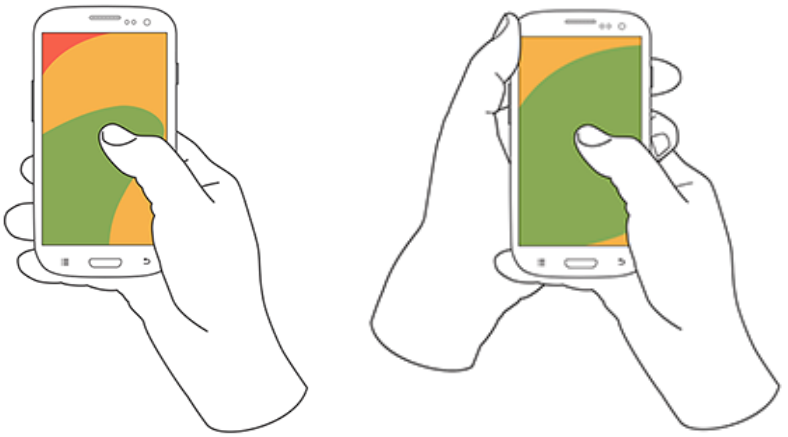
Cradling in Two Hands 36%



Two-Handed Use 15%

Cette étude montre aussi que nous ne gardons pas toujours la même prise : nous passons d'une main à l'autre selon le contexte, par exemple lorsque nous marchons, que nous portons un sac ou que nous tenons un café. Josh CLARK souligne d'ailleurs que, selon HOOBER, deux tiers des prises à une main se font de la main droite, une proportion inférieure à celle des droitiers. Une partie des utilisateurs se sert donc de sa main la moins habile pour manipuler son téléphone.

Le point essentiel de cette étude concerne le pouce. Lorsqu'on tient son téléphone d'une seule main, c'est le seul doigt disponible, et même parmi les personnes qui utilisent leurs deux mains, une partie touche l'écran avec le pouce. En additionnant ces usages, Josh CLARK arrive à une conclusion très nette : « 75 % des interactions sont effectuées avec le pouce ».

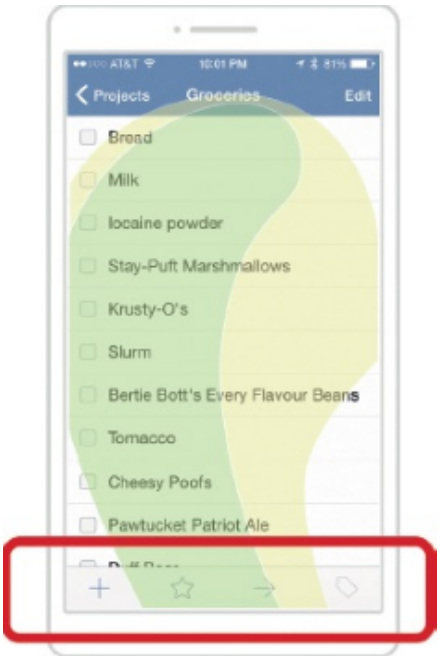


49% **+** ***26%*** **=** ***75%***

La zone du pouce

Le pouce peut atteindre presque tout l'écran d'un téléphone, mais seule une partie de cet écran est accessible sans effort. Josh CLARK précise que ce territoire confortable « ne représente qu'un tiers de l'écran ». Pour un droitier, il prend la forme d'un arc qui part du coin inférieur gauche. À l'intérieur de cette zone, toutes les positions n'offrent pas la même précision : une étude menée par Qian FEI chez Alibaba, également citée dans le livre, montre que le bas de l'écran et le coin inférieur droit sont les zones les moins précises pour un droitier.

Pour les gauchers, la zone s'inverse simplement de gauche à droite. Comme la plupart des utilisateurs changent régulièrement de main, Josh CLARK recommande de ne pas optimiser pour un seul côté, mais de placer les fonctionnalités principales au centre, là où les zones des deux pouces se rejoignent. Il retient surtout une règle simple : « la distinction entre le haut et le bas est plus importante que la distinction entre la gauche et la droite ». Le bas de l'écran est confortable, le haut demande un effort, et cet écart augmente avec la taille des téléphones.





Il ne s'agit pas pour autant de repousser toutes les actions en haut de l'écran. Lorsqu'une action est au cœur de l'application et revient souvent, elle a toute sa place en bas, au centre. Les applications Swarm et Instagram en sont de bons exemples : leur action principale, l'enregistrement d'un lieu ou la publication d'une photo, se trouve en bas de l'écran, accessible aussi bien de la main droite que de la main gauche.

Source :

- *Josh CLARK, Design tactile.*
- *Steven HOOBER, How Do Users Really Hold Mobile Devices? (2013).*

Application à Toubib

J'ai appliqué ces principes aux écrans de l'application patient. L'action principale de chaque écran, comme « Prendre rendez-vous » sur la fiche du médecin ou la confirmation d'un créneau, est placée en bas et au centre de l'écran sur mobile, dans la zone la plus confortable pour le pouce.

Les créneaux horaires, nombreux et proches les uns des autres, sont affichés avec une taille et un espacement suffisants pour être touchés sans erreur. Les éléments moins fréquents, comme le bouton de connexion ou le mode daltonien, restent dans l'en-tête, en haut de l'écran.

2.2. Développer les composants UI/UX

2.2.1. Le stack technique

J'ai développé les deux applications front-end, `Toubib` et `ToubibB0`, avec React, une bibliothèque JavaScript conçue pour construire des interfaces à partir de composants. J'ai choisi React pour trois raisons principales:

- La demande sur le marché du travail et précisément mon entreprise.
- Son DOM virtuel ne met à jour que les parties de la page qui changent, ce qui rend l'interface réactive même lorsque les données évoluent souvent, comme dans un agenda.
- L'écosystème est très riche et largement utilisé en entreprise, ce qui me garantit une documentation abondante et des tutoriels à tous les niveaux.

Autour de React, j'ai utilisé Vite comme outil de développement et de compilation : il offre un serveur de développement très rapide et produit, pour la mise en production, des fichiers optimisés et légers. La navigation entre les pages est gérée par React Router, qui permet de changer d'écran sans recharger la page.

Le code est écrit en JSX, une extension de syntaxe de JavaScript qui permet de réunir la logique et le balisage d'un composant dans un même fichier, sans pour autant les mélanger, cette logique (état, appels à l'API, traitement des données) est placée en haut du composant, tandis que le balisage, proche du HTML, se trouve uniquement dans l'instruction `return`, qui décrit ce que le composant affiche. Cette séparation rend la structure de chaque composant très lisible.

Les deux applications partagent le même stack, ce qui facilite la maintenance et me permet de réutiliser les mêmes pratiques d'un projet à l'autre. Elles communiquent toutes les deux avec la même API Symfony, qui centralise les données et la logique métier.

2.2.2. Une approche par composants

J'ai construit l'interface en découpant chaque écran en composants indépendants et réutilisables. J'ai distingué deux niveaux. Les composants génériques constituent la base de l'interface : boutons, champs de formulaire, fenêtres modales, cartes, indicateurs de chargement. Ils ne connaissent rien du domaine médical et peuvent être réutilisés partout. Les composants métier s'appuient sur ces briques pour représenter les éléments propres à `Toubib`: la carte d'un médecin dans les résultats de recherche, la grille des créneaux disponibles, le badge qui affiche le statut d'un rendez-vous, ou encore la fiche d'un patient.

Chaque composant reçoit les données dont il a besoin par ses propriétés (*props*) et ne s'occupe que de son propre affichage. Cette séparation présente plusieurs avantages. Un même composant, comme le bouton principal, garde un aspect et un comportement identiques sur tous les écrans, ce qui assure la cohérence visuelle définie lors du maquettage. Une correction ou une amélioration apportée à un composant profite immédiatement à toutes les pages qui l'utilisent. Enfin, des composants courts et spécialisés sont plus faciles à lire, à tester et à faire évoluer.

2.3. Réaliser une interface Web dynamique

2.3.1. Une application monopage pilotée par l'état

Les deux applications, `Toubib` et `ToubibB0`, fonctionnent selon le principe de l'application monopage (*Single Page Application*). Le navigateur ne charge l'application qu'une seule fois. React met ensuite à jour l'affichage au fil de la navigation, sans jamais recharger la page entière, et React Router associe chaque adresse à l'écran correspondant. Seules les données transitent entre le navigateur et le serveur, au format JSON. La navigation est ainsi fluide, proche de celle d'une application mobile, et le volume de données échangé est bien plus faible qu'avec des pages entièrement générées par le serveur. Ce caractère dynamique repose sur la gestion d'état de React. J'utilise le hook `useState` pour conserver les données propres à un composant, comme la date sélectionnée ou la liste des créneaux affichés. Chaque fois que cet état change, React met automatiquement à jour la partie concernée de l'écran, sans que j'aie à manipuler le DOM moi-même. Le hook `useEffect`, lui, me permet de déclencher une action au bon moment, par exemple charger les créneaux disponibles dès que l'utilisateur choisit une nouvelle date.

Certaines informations doivent en outre être accessibles depuis toute l'application, en particulier l'utilisateur connecté. Plutôt que de les transmettre de composant en composant, je les ai centralisées dans un contexte d'authentification. N'importe quel composant peut ainsi savoir si l'utilisateur est connecté, afficher son nom ou adapter le menu de navigation à son rôle.

2.3.2. Échanges avec l'API et l'affichage des données

J'ai regroupé tous les échanges avec l'API Symfony dans des fonctions de service, séparées des composants d'affichage. Chaque requête est envoyée avec l'option credentials: include, qui demande au navigateur de joindre automatiquement les cookies d'authentification. Ces cookies étant marqués HttpOnly, ils ne sont pas lisibles par le JavaScript : un script malveillant injecté dans la page ne pourrait donc pas dérober le jeton. Le front n'a jamais à manipuler le jeton lui-même.

Grâce à ces échanges, les écrans des deux Frontend évoluent en fonction des actions de l'utilisateur et des réponses de l'API. Dans la recherche, les résultats sont chargés page par page et s'actualisent à mesure que l'utilisateur affine sa saisie. Sur la fiche d'un médecin, la grille des créneaux est rechargée à chaque changement de date. Dans l'espace médecin, l'agenda affiche les rendez-vous de la période choisie, et la modification du statut d'un rendez-vous est immédiatement visible à l'écran, sans rechargement de la page. Pendant chaque échange, l'interface tient l'utilisateur informé : un indicateur de chargement s'affiche pendant la requête, puis laisse place soit aux données, soit à un message d'erreur compréhensible, soit à un message indiquant qu'aucun résultat n'a été trouvé.

Tout au long du développement de l'interface, j'ai appliqué un principe simple : le front affiche et met en forme, et peut effectuer des vérifications de confort, mais c'est l'API qui décide. Les créneaux présentés sur la fiche d'un médecin, par exemple, sont exclusivement ceux calculés par le serveur. Le front ne calcule jamais une disponibilité, ne vérifie jamais seul un droit d'accès et n'applique aucune règle qui engage les données. Ces règles restent centralisées côté serveur, où elles ne peuvent pas être contournées. React m'apporte de son côté une interface rapide, réactive et agréable, qui réagit instantanément aux actions de l'utilisateur.

PRÉSENTATION DU PROJET

1. CAHIER DES CHARGES ET UML

1.1. Cahier des charges

Contexte

La prise de rendez-vous médicaux reste souvent contraignante : le patient doit appeler le cabinet pendant les heures d'ouverture, attendre qu'une ligne se libère et noter lui-même son rendez-vous. Du côté des médecins, la gestion de l'agenda, des patients et des cabinets repose encore fréquemment sur des outils dispersés, l'application répond à ce double constat en proposant une plateforme en ligne de gestion de rendez-vous médicaux, qui met en relation des patients et des médecins.

Objectifs

Le projet poursuit trois objectifs. Il doit d'abord offrir au patient un point d'entrée unique pour trouver un médecin, consulter ses disponibilités et réserver un rendez-vous à tout moment. Il doit ensuite fournir au médecin un outil de gestion de son activité : profil professionnel, cabinets, agenda et patientèle. Enfin, l'ensemble doit reposer sur une API centralisée qui porte toutes les règles métier, afin de garantir leur application quelle que soit l'interface utilisée et de pouvoir accueillir de nouveaux clients à l'avenir.

Besoin: Comptes et authentification

Un visiteur crée un compte patient, qu'il active en confirmant son adresse électronique par un lien envoyé par courriel. La connexion se fait par adresse et mot de passe, avec un parcours distinct pour les médecins, et la session se renouvelle automatiquement. Un mot de passe oublié peut être réinitialisé par courriel. Les mots de passe respectent une politique de robustesse, et les tentatives de connexion répétées sont limitées.

Besoin: Recherche et découverte des médecins

La recherche est accessible sans compte. Elle porte sur le nom, la spécialité, la ville ou la région, et renvoie des résultats paginés. Chaque médecin et chaque cabinet disposent d'une fiche publique.

Besoin: Prise de rendez-vous

Le patient consulte les créneaux disponibles d'un médecin et en réserve un. Les créneaux tiennent compte des horaires, de la durée de consultation, de la pause déjeuner et des rendez-vous existants, avec un délai minimal de trois heures. Le médecin confirme ou annule les rendez-vous, et peut déclarer des indisponibilités.

Besoin: Espace médecin

Le médecin gère son profil professionnel et un ou plusieurs cabinets, chacun avec ses horaires, sa durée et son tarif de consultation. Le propriétaire d'un cabinet peut y inviter des confrères, qui peuvent partager leur agenda. Le médecin consulte son agenda sous plusieurs vues et gère ses patients : création de dossier, recherche, modification et historique médical.

Besoin: Administration

L'administrateur doit pouvoir consulter la liste des comptes, suspendre ou réactiver un compte, promouvoir un compte au statut de médecin ou désactiver un profil médecin, et imposer un changement de mot de passe.

Besoin: Sécurité

La sécurité est une exigence centrale, car la plateforme manipule des données personnelles et médicales. Les mots de passe sont hachés et ne sont jamais stockés ni journalisés en clair. L'authentification repose sur un jeton d'accès de courte durée et un jeton de rafraîchissement, transmis dans des cookies sécurisés et inaccessibles au JavaScript. L'API n'expose que des champs explicitement autorisés, grâce aux groupes de sérialisation, et n'accepte que les requêtes provenant des origines déclarées (CORS).

Tout point d'entrée d'écriture vérifie que l'utilisateur connecté est bien propriétaire de la ressource qu'il modifie. Les erreurs inattendues ne doivent révéler aucun détail technique au client, et aucune donnée de paiement sensible, comme le cryptogramme d'une carte, ne doit être conservée. Les applications front-end ne contiennent aucune logique métier. Les contrôles qu'elles effectuent avant l'envoi d'une requête améliorent l'expérience, mais ne constituent jamais une garantie : seule l'API fait foi. Ce principe s'impose à toute évolution future du projet.

Besoin: Performance

Les listes volumineuses (patients, rendez-vous, résultats de recherche) sont paginées côté serveur. Les opérations lentes, comme l'envoi de courriels, sont traitées de manière asynchrone afin de ne pas retarder la réponse à l'utilisateur.

Besoin: Ergonomie et accessibilité

Les interfaces sont simples, cohérentes et conçues *mobile first*, avec des zones tactiles adaptées et une bonne lisibilité.

Besoin: Maintenabilité et qualité

La logique métier est isolée dans des services et des dépôts dédiés, et le schéma de la base est versionné par des migrations. Les fonctionnalités critiques doivent être couvertes par des tests automatisés.

Besoin: Déploiement

Chaque application doit pouvoir être déployée et mise à jour indépendamment des deux autres. L'environnement de développement doit se reconstruire entièrement par une seule commande, sans dépendre d'une installation locale de PHP, de Node.js ou de MySQL.

Besoin: Contraintes techniques

Le projet est composé de trois applications indépendantes, chacune conteneurisée avec Docker et disposant de ses propres fichiers de configuration d'environnement.

Application	Rôle	Technologies
`ToubibAPI`	API REST, logique métier et accès aux données	PHP 8.4, Symfony 8, Doctrine ORM 3, MySQL 8, Redis 7, LexikJWT, Nelmio CORS, Nginx
`Toubib`	Application web patient	Nginx, React 19, Vite 7, React Router 7, Tailwind CSS 4
`ToubibBO`	Application web médecin et Administrateur	Nginx, React 19, Vite 7, React Router 7, Tailwind CSS 4, FullCalendar, TanStack Table

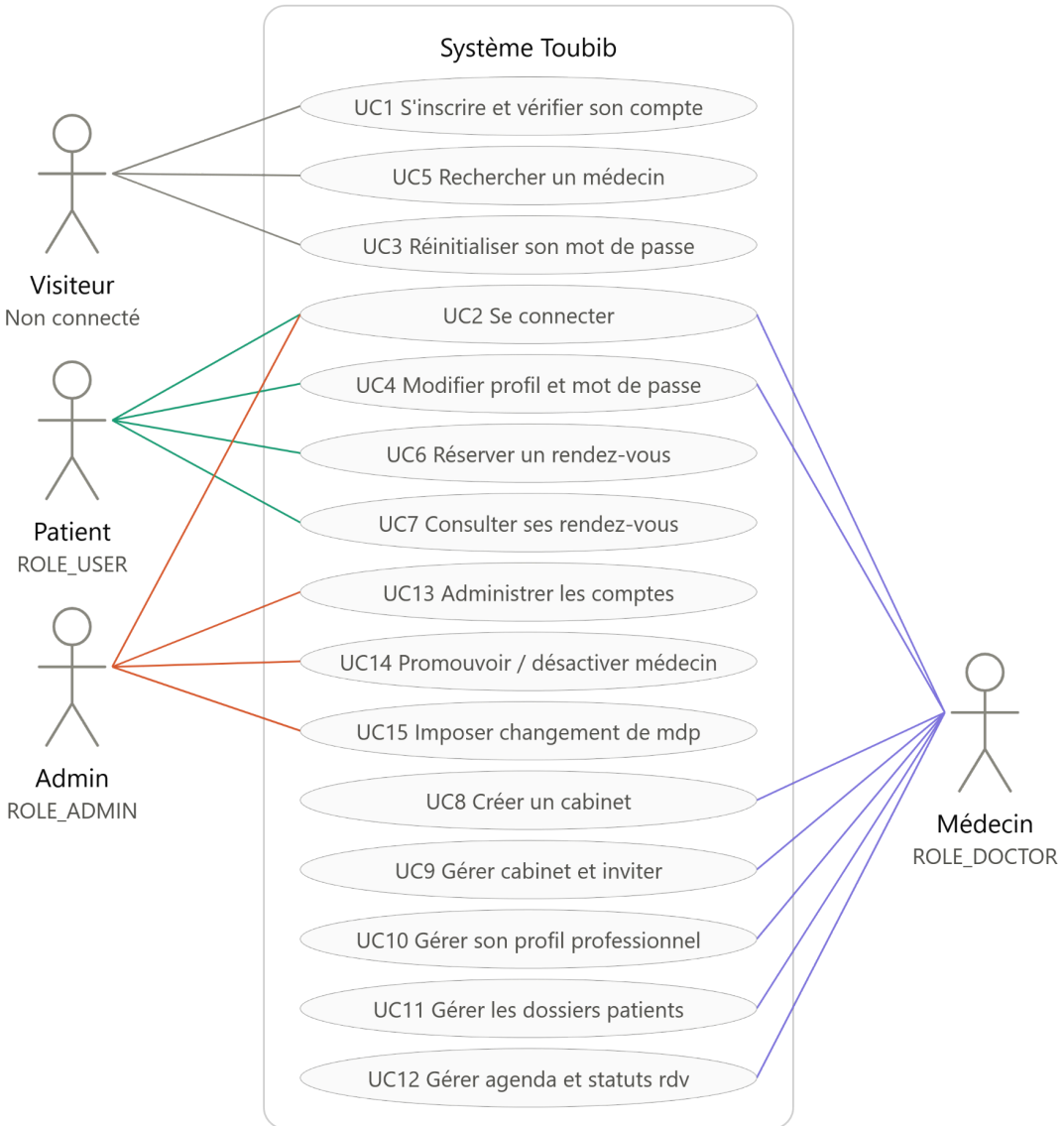
1.2. UML

1.2.1. Acteurs et Use Case

Acteur	Rôle	Description
Visiteur	Non connecté	Recherche des médecins, consulte les fiches publiques et peut créer un compte.
Patient	ROLE_USER	Réserve des rendez-vous, consulte son historique et gère son profil.
Médecin	ROLE_DOCTOR	Rattaché à un profil professionnel, il gère ses cabinets, son agenda et ses patients.
Administrateur	ROLE_ADMIN	Supervise les comptes de la plateforme.

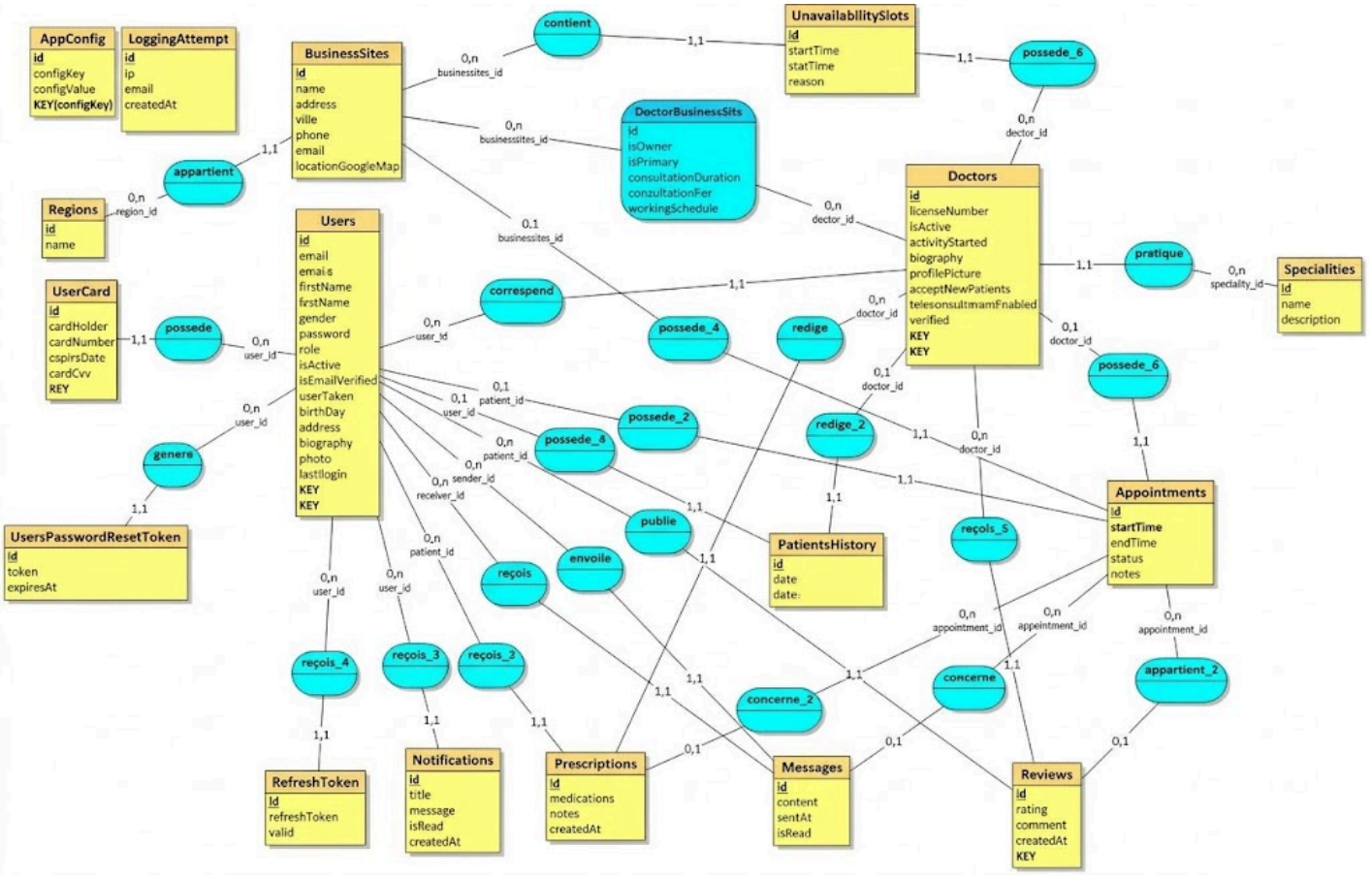
Ref	Cas d'utilisation	Acteur(s)	Point d'entrée API (EndPoint)
UC1	S'inscrire et vérifier son compte	**Visiteur,** Patient	**POST** /api/auth/register **POST** /api/auth/verify-account
UC2	Se connecter	Patient Médecin	**POST** /api/auth/login **POST** /api/auth/doctor/login
UC3	Réinitialiser son mot de passe	Patient Médecin	**POST** /api/auth/forgot-password **POST** /api/auth/reset-password
UC4	Modifier son profil et son mot de passe	Patient Médecin	**GET**, **PATCH** /api/users/me **PATCH** /api/users/me/change-password
UC5	Rechercher et consulter un médecin	**Visiteur**	**GET** /api/search **GET** /api/doctor/{id}/{slug}
UC6	Consulter les créneaux Réserver un RDV	Patient	**GET** /api/doctor/{id}/get-appointment/start/{startDate}/end/{endDate} **POST** /api/doctor/{idDoctor}/set-appointment

UC7	Consulter ses rendez-vous	**Patient**	**GET** /api/users/me/appointments
UC8	Créer un cabinet	**Médecin**	**POST** /api/businesssites
UC9	Gérer un cabinet	**Médecin**	**PATCH** /api/businesssites/{id}
UC10	Gérer son profil professionnel	**Médecin**	**GET**, **PATCH** /api/doctor/me
UC11	Gérer les dossiers patients	**Médecin**	**GET**, **POST**, **PATCH** /api/doctor/patients/{id}
UC12	Gérer son agenda et les statuts des RDV	**Médecin**	**GET** /api/doctor/calendar **PATCH** /api/doctor/appointments/{id}/status
UC13	Administrer les comptes	**Admin**	**GET** /api/admin/users **PATCH** /api/admin/users/{id}/suspend
UC14	Promouvoir ou désactiver un médecin	**Admin**	**PATCH** /api/admin/users/{id}/toggle-doctor
UC15	Imposer un changement de mot de passe	**Admin**	**PATCH** /api/admin/users/{id}/force-password-change



1.2.2. MCD et DB Diagram

MCD:



Avant de créer la base, j'ai conçu le modèle conceptuel de données (MCD) selon la méthode Merise. Le MCD décrit les données du projet indépendamment de toute technologie : il représente les entités (les objets métier, comme un utilisateur, un cabinet ou un rendez-vous), leurs attributs et les associations qui les relient, chacune portée par des cardinalités. Les cardinalités indiquent combien de fois une entité peut participer à une association, au minimum et au maximum : par exemple, un rendez-vous concerne exactement un médecin (1,1), alors qu'un médecin peut avoir aucun ou plusieurs rendez-vous (0,n). Ce modèle a ensuite été traduit en modèle logique, puis en tables MySQL au moyen des entités Doctrine.

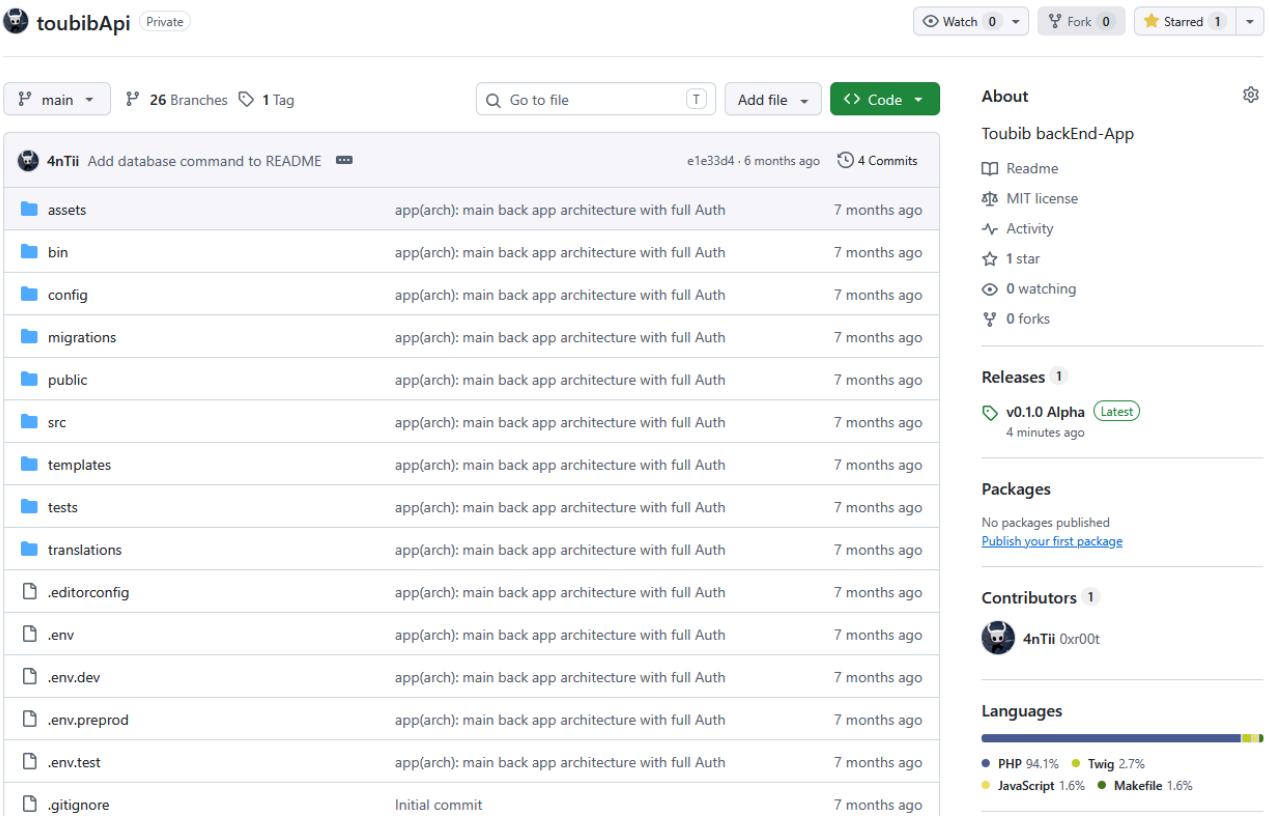
La table Users est le pivot du modèle, elle regroupe tous les comptes de la plateforme, quel que soit leur profil : patients, médecins et administrateurs. J'ai fait le choix de ne pas créer une table par type d'utilisateur, car ces profils partagent l'essentiel de leurs informations : identité, adresse électronique, téléphone, mot de passe haché, statut du compte. Le profil est déterminé par les rôles du compte (`ROLE_USER`, `ROLE_DOCTOR`, `ROLE_ADMIN`), stockés dans la table `Users` elle-même.

Cette centralisation simplifie fortement l'authentification : il n'existe qu'une seule table dans laquelle chercher un compte lors de la connexion, et une seule règle d'unicité pour l'adresse électronique. Elle explique aussi pourquoi la plupart des autres entités se rattachent à `Users` : les rendez-vous d'un patient, les messages, les notifications, les jetons de rafraîchissement, les demandes de réinitialisation de mot de passe ou encore les cartes de paiement.

2. GIT ET AGILITÉ

2.1. Git

J'ai versionné l'ensemble du code de **`Toubib`** avec Git, en appliquant le workflow de branches que j'ai appris en entreprise, le repository repose sur deux branches permanentes, toutes deux protégées : on ne peut pas y pousser de modifications directement, tout changement passe par une *merge request*.



J'ai appris à utiliser Git au quotidien en entreprise, puis je l'ai appliqué sur l'application. Je travaille avec des branches : chaque fonctionnalité ou correction est développée sur une branche dédiée, isolée de la branche principale, qui contient toujours une version stable du projet. Une fois la fonctionnalité terminée et testée, la branche est fusionnée dans la branche develop.

Cette organisation me permet de travailler sur une évolution sans risquer de casser ce qui fonctionne déjà, et d'abandonner simplement une piste qui n'aboutit pas. J'ai également pris l'habitude de rédiger des messages de commit clairs, qui décrivent ce qui a été fait et pourquoi, afin que l'historique reste lisible pour moi comme pour un futur collaborateur.

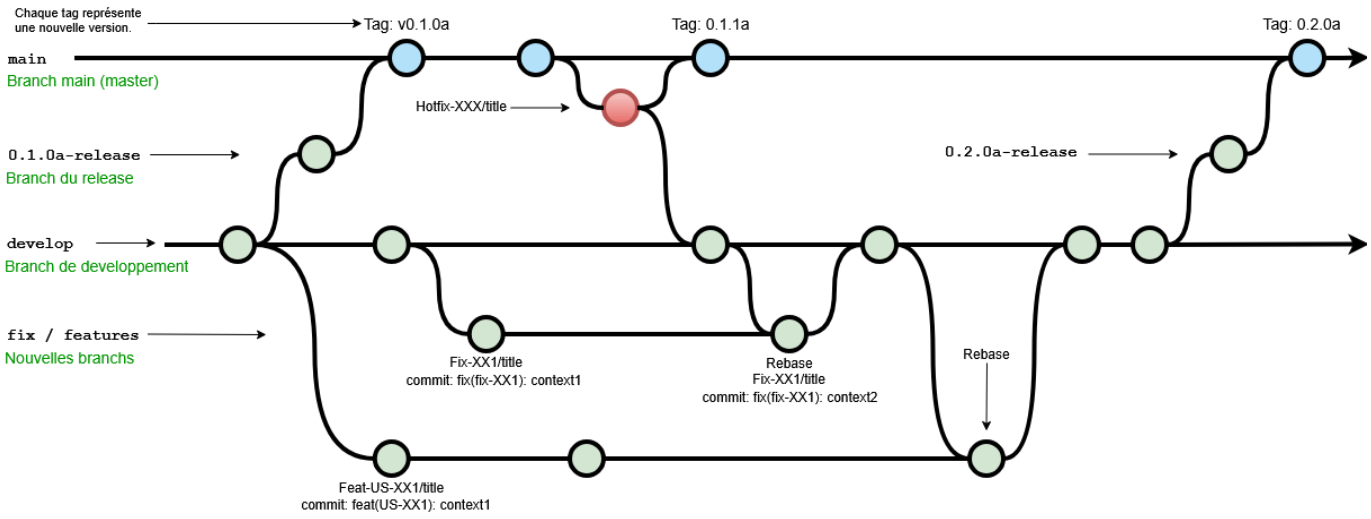
Git Workflow :

Toubib possède deux branches permanentes, elles sont toutes les deux protégées:

- **main** *Protected*, contient le code en production. Chaque état de cette branche correspond à une version stable et livrable.
- **develop** *Protected*, est la branche par défaut, elle regroupe les fonctionnalités terminées en attente de la prochaine livraison.

Les autres branches sont temporaires, et leur origine dépend de la nature du travail :

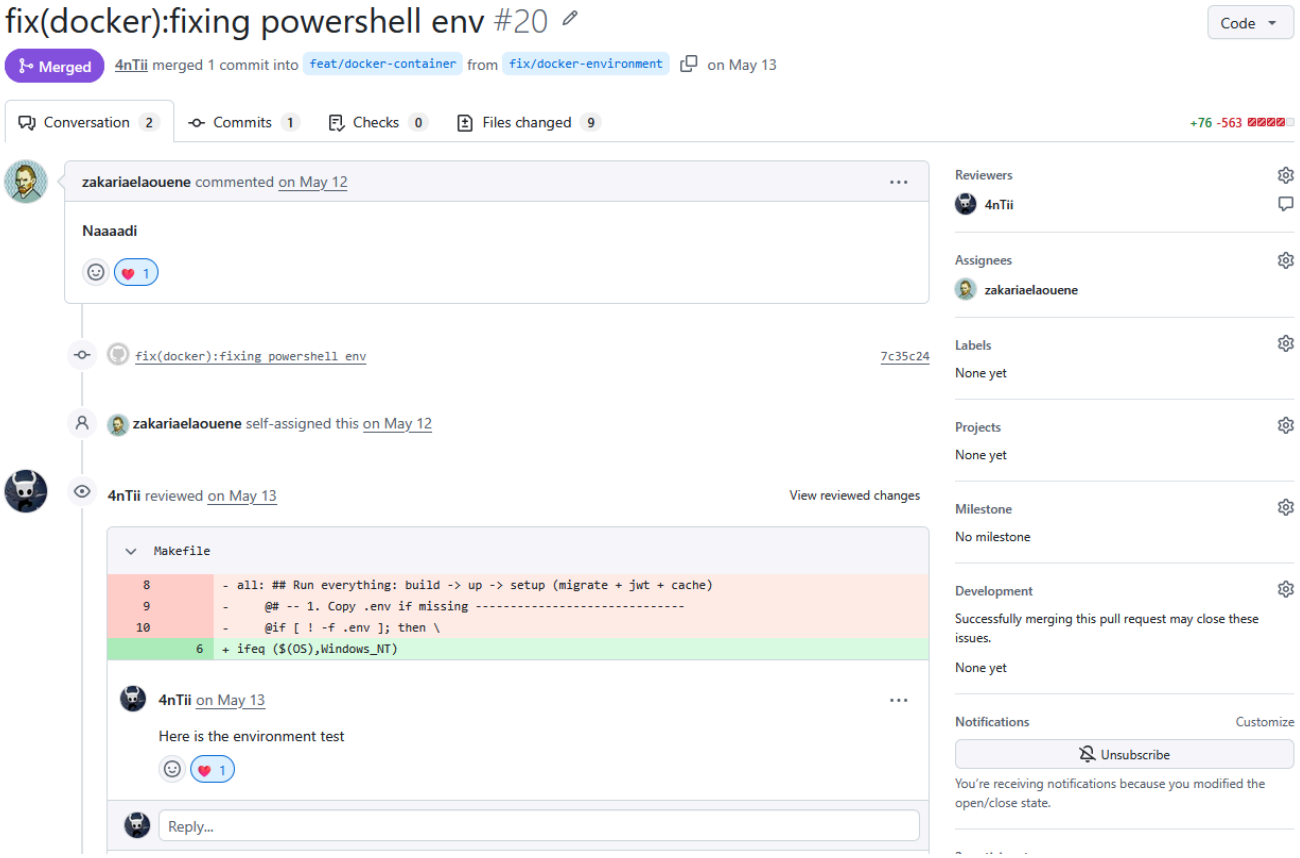
- Les fonctionnalités (Feat/…) et les corrections (Fix/…) sont créées depuis develop, puis fusionnées dans develop une fois terminées.
- Les correctifs urgents (Hotfix/…) sont créés depuis main, pour corriger un problème en production sans embarquer les développements en cours. Ils sont ensuite fusionnés dans main et dans develop, afin que la correction ne soit pas perdue lors de la livraison suivante.



Dans ce workflow, chaque *merge request* doit être relue par une autre personne avant d'être fusionnée. Cette revue de code permet de détecter les erreurs, de partager la connaissance du code et de garantir le respect des conventions de l'équipe. Sur **`Toubib`**, cette règle ne pouvait pas s'appliquer pleinement : je fusionnais mes propres *merge requests* sans relecture par un pair. En revanche, j'ai pratiqué la revue de code dans l'autre sens : lorsque des collègues me demandaient de l'aide, je relisais leurs *merge requests* et je les validais avant leur fusion. Cet exercice m'a appris à lire un code que je n'avais pas écrit, à formuler des remarques constructives et à vérifier qu'une modification répond bien au besoin sans introduire de régression.

Git m'a aussi servi à protéger les informations sensibles. Le fichier .gitignore exclut du dépôt les dépendances, les fichiers générés et surtout le fichier .env.local, qui contient les secrets : mots de passe de la base de données et clés de signature des jetons JWT. Seul le fichier .env.example est versionné, comme modèle de configuration.

Exemple de la MR *(merge request)*, d'un collègue qui m'a aidé à appliquer un correctif sur la conteneurisation du projet, j'ai fais moi-même la review pour s'assurer qu'il n'y a pas de problème au niveau des données, et à la fois apprendre comment le bogue était corrigé:



2.2. La méthode Agile

J'ai découvert la méthode agile et notamment le cadre Scrum avant cette formation, puis j'ai suivi une formation dispensée par ORSYS *Lyon,* et j'ai approfondi ces notions en participant aux rituels et backlog à Genilink. L'agilité privilégie un développement itératif et incrémental : le produit est construit par petites étapes, chacune aboutissant à une version utilisable, ce qui permet d'ajuster régulièrement la direction du projet. Scrum organise ce travail en cycles courts et de durée fixe, appelés sprints. Chaque sprint commence par une réunion de planification (*sprint planning*), au cours de laquelle l'équipe sélectionne, dans le product backlog (la liste priorisée des fonctionnalités attendues), les éléments qu'elle s'engage à réaliser. Pendant le sprint, une courte réunion quotidienne (*daily meeting*) permet à chacun de partager son avancement et ses blocages. Le sprint se termine par une revue, durant laquelle les fonctionnalités livrées sont présentées, puis par une rétrospective, consacrée à l'amélioration de la manière de travailler.

En entreprise, j'ai travaillé dans une équipe organisée autour des rôles définis par Scrum. Le Product Owner porte la vision du produit : il recueille les besoins, les traduit en *user stories* et décide de leur priorité. Le Scrum Master veille au bon déroulement de la méthode : il anime les cérémonies, aide

l'équipe à lever les obstacles et protège le sprint des interruptions. L'équipe de développement, enfin, estime la complexité des stories, s'engage sur un objectif de sprint et réalise les fonctionnalités. Cette expérience m'a appris à découper un besoin en éléments livrables, à estimer leur complexité et à travailler avec un objectif clair et limité dans le temps.

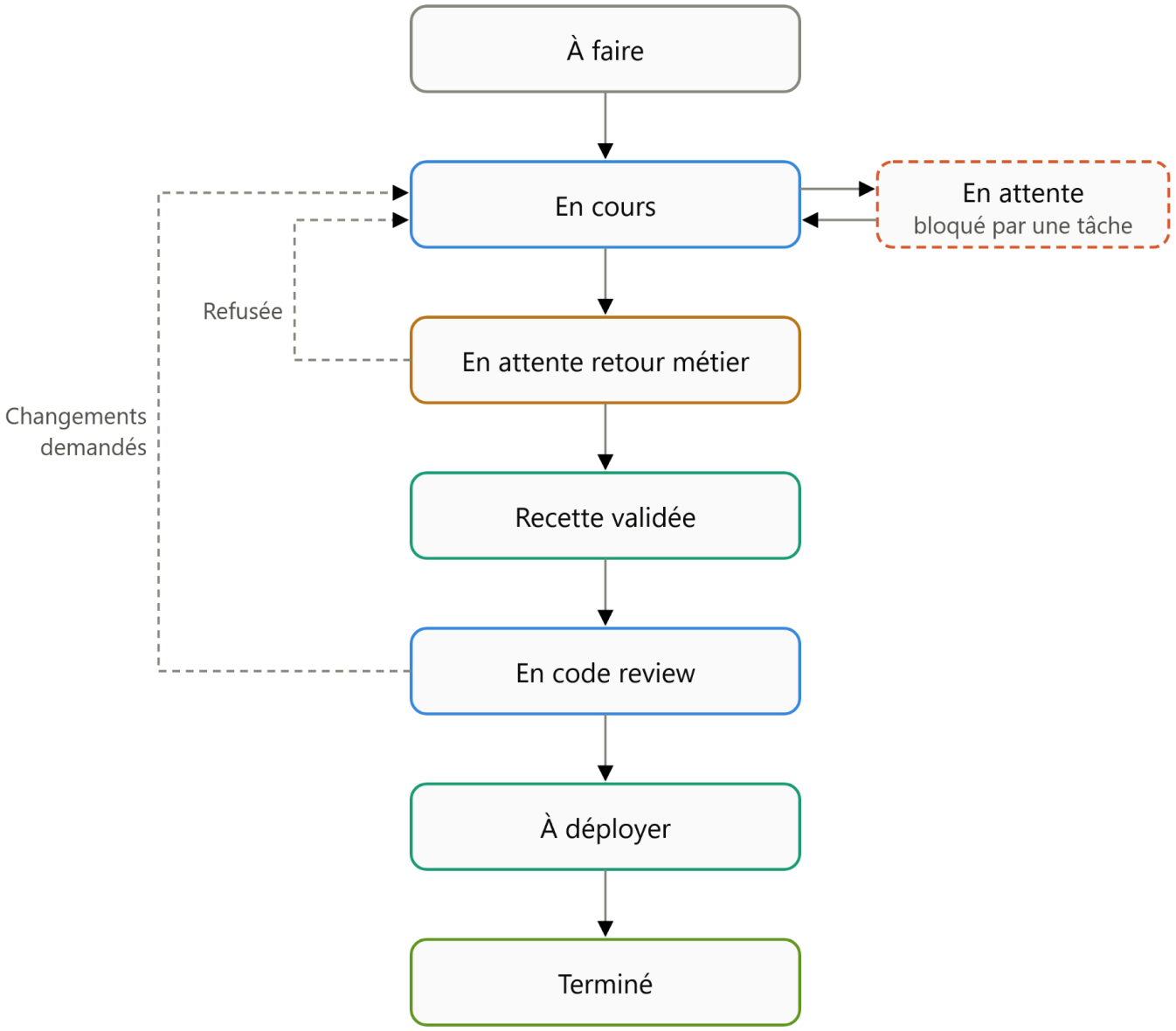
J'ai appliqué cette démarche à l'application, en l'adaptant à un projet mené seul. J'ai cumulé les rôles de Product Owner, en définissant et en priorisant les besoins, et de développeur. Pour ne pas valider mon propre travail, j'ai toutefois confié la recette à des collègues de formation et de travail. Je leur ai d'abord expliqué le métier de la plateforme : ses acteurs, leurs parcours et les règles de gestion des rendez-vous. Ils ont ensuite joué le rôle de Product Owner chargés de la validation : à la fin de chaque sprint, ils testaient les fonctionnalités livrées et confirmaient qu'elles répondaient au besoin exprimé, ou signalaient les écarts à corriger. Ce regard extérieur m'a permis de détecter des problèmes que je ne voyais plus, à force de connaître l'application de l'intérieur.

3. LE WORKFLOW

En entreprise, j'ai appris à utiliser Jira, l'outil de gestion de projet édité par Atlassian et largement utilisé par les équipes agiles. Jira permet de constituer le backlog, de rédiger les *user stories* avec leurs critères d'acceptation, de les estimer en points et de les regrouper en sprints. Le tableau de sprint présente les tickets en colonnes, ce qui rend l'avancement de l'équipe visible d'un coup d'œil.

Chaque ticket suit un workflow, c'est-à-dire une succession d'états définis à l'avance. Un ticket ne change d'état que lorsque les conditions requises sont remplies : une story ne peut être déclarée terminée qu'après vérification. Jira fournit également des indicateurs de suivi, comme le *burndown chart*, qui montre le travail restant au fil du sprint et permet de détecter rapidement un retard.

Faute de moyens, je n'ai pas pu mettre en place Jira et Confluence pour **`Toubib`**, ces outils étant payants pour un usage complet. J'ai toutefois structuré la documentation du projet pour qu'elle puisse y être migrée sans effort, ce qui pourrait être importé directement dans Jira, la documentation peut être reprise dans un espace Confluence relié au projet, et commencer un nouveau backlog.

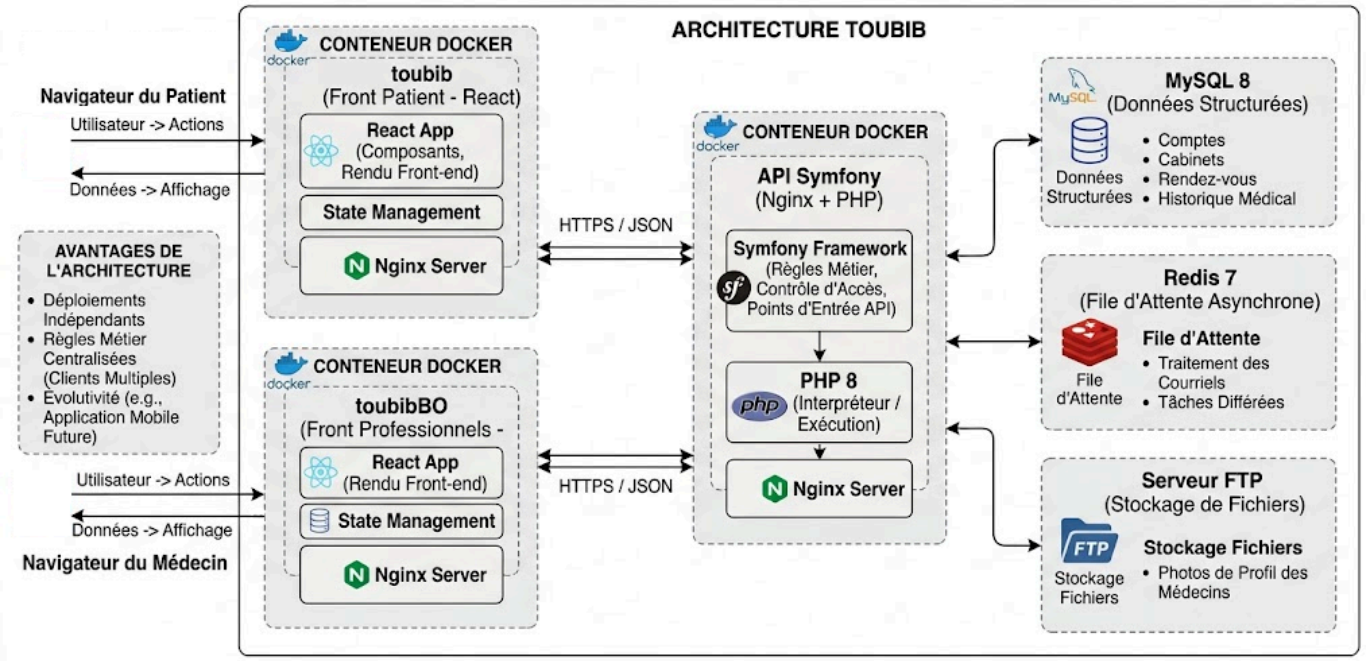

À faire
En cours
En attente
bloqué par une tâche
Refusée
En attente retour métier
Changements
demandés
Recette validée
En code review
À déployer
Terminé

4. LOUPE TECHNIQUE

4.1. Architecture générale

Toubib repose sur une architecture découplée, composée de trois applications indépendantes. L'API `ToubibAPI`, développée avec Symfony, est le cœur du système : c'est la seule application qui accède à la base de données et au serveur de fichiers, et elle porte l'ensemble des règles métier. Les deux applications React, `Toubib` pour les patients et `ToubibBO` pour les médecins et les administrateurs, ne font qu'afficher les données et transmettre les actions de l'utilisateur à l'API, au format JSON.

Chaque application est servie par Nginx et s'exécute dans son propre conteneur Docker. Ce découpage présente plusieurs avantages. Chaque application peut être déployée et mise à jour sans toucher aux autres. Les règles métier, centralisées dans l'API, s'appliquent quel que soit le client utilisé. Enfin, l'architecture est prête à accueillir de nouveaux clients, comme une application mobile, qui consommeraient la même API.



4.2. Environnement de développement

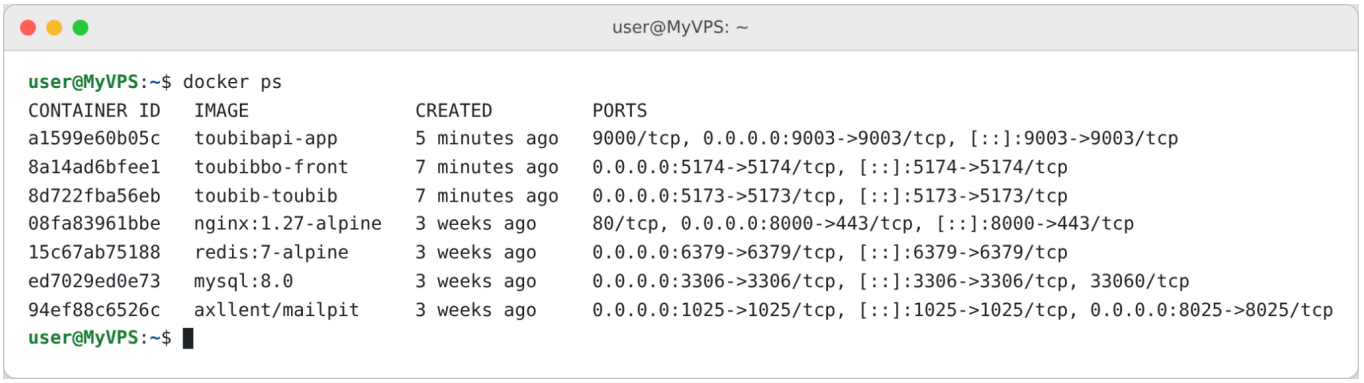
J'ai conteneurisé l'ensemble du projet avec Docker, afin de disposer d'un environnement identique sur toutes les machines, sans installer PHP, Node.js ou MySQL localement. Chaque application possède son propre `Dockerfile` et son propre fichier `docker-compose.yml`. Les `Dockerfile` sont multi-étapes : une même définition produit une image de développement, avec les outils de débogage et le rechargement à chaud, et une image de production, plus légère et optimisée.

En développement, les trois applications tournent sur des ports distincts : `8000` pour l'API, `5173` pour `Toubib` et `5174` pour `ToubibBO`. Une seule commande, `make`, reconstruit et démarre entièrement l'environnement de chaque application. Le HTTPS est activé dès le développement grâce

à un certificat auto-signé, car les cookies d'authentification ne fonctionnent que sur une connexion chiffrée.

Travailler en HTTPS dès le départ m'a ainsi évité de découvrir des problèmes de cookies au moment de la mise en production.

La configuration suit le même modèle dans les trois applications. Le fichier `.env` contient les valeurs par défaut et est versionné. Le fichier `.env.local` contient les secrets (mots de passe, clés de signature) et n'est jamais versionné. Le fichier `.env.example` sert de modèle complet pour installer le projet sur une nouvelle machine.

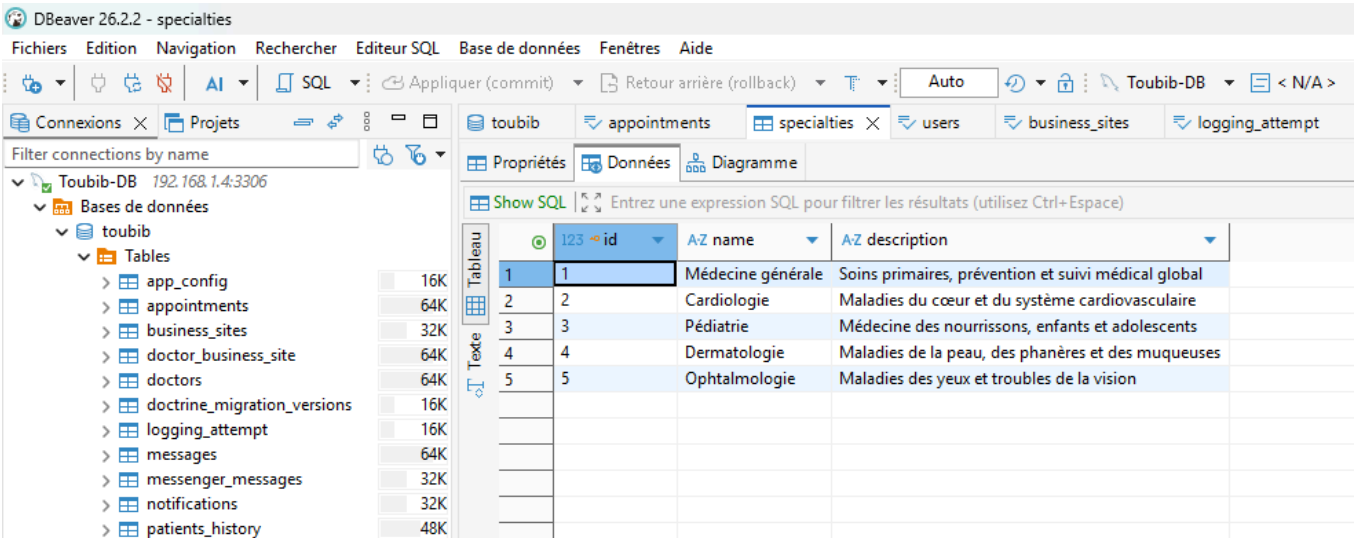


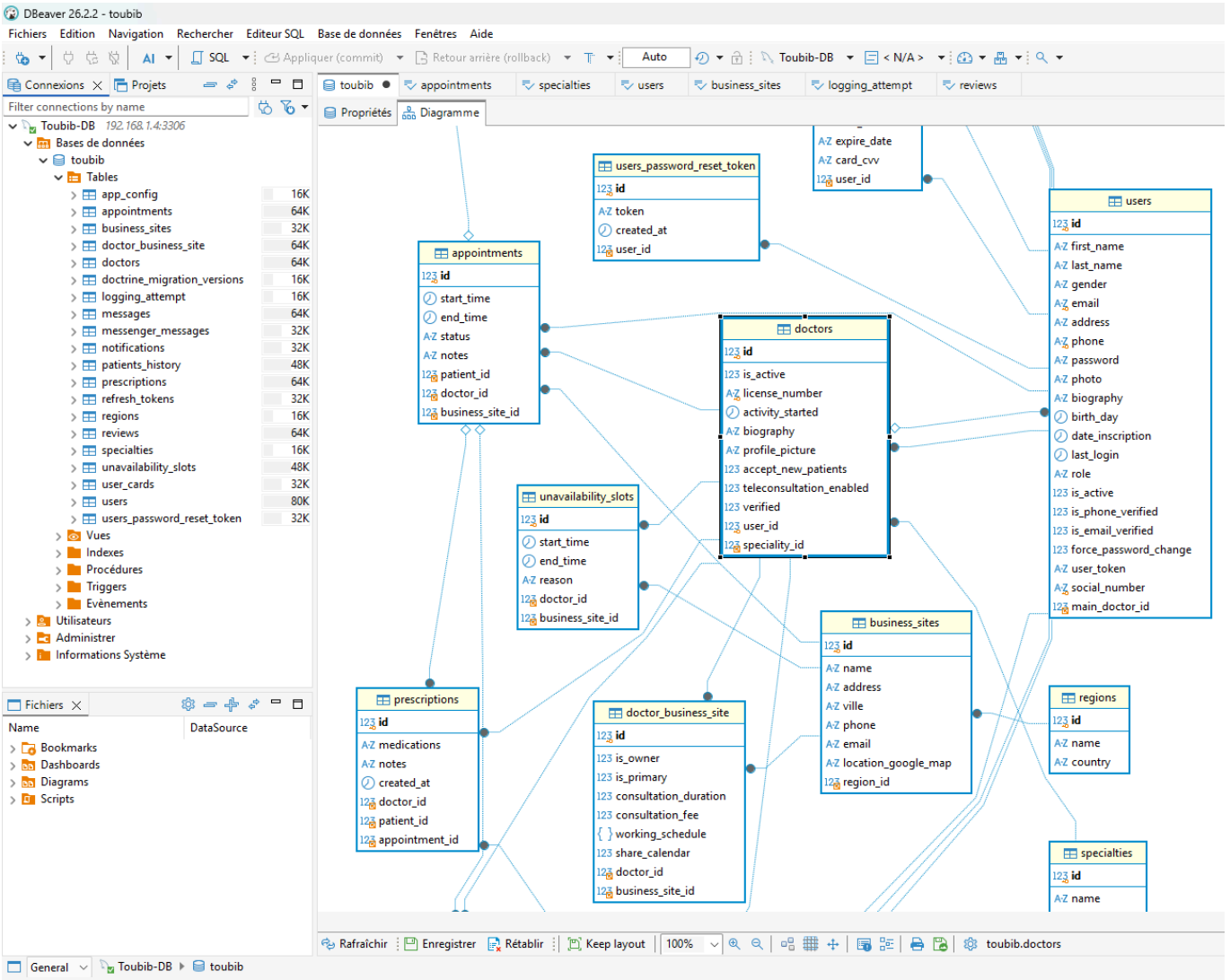
4.3. Base de données

Je fais évoluer la structure de la base exclusivement avec les migrations Doctrine, depuis la ligne de commande Symfony exécutée dans le conteneur PHP. Après une modification d'entité, je génère une migration, je relis le SQL qu'elle contient, puis je l'applique avec `doctrine:migrations:migrate`. La commande `doctrine:schema:validate` me permet aussi de vérifier que le mapping des entités et le schéma réel de la base sont bien synchronisés. Je ne modifie jamais la structure de la base à la main : chaque changement est versionné avec le code et peut être rejoué à l'identique sur un autre environnement.

Pour explorer la base pendant le développement, j'utilise DBeaver, un client de base de données graphique et gratuit. Le port de MySQL est exposé sur ma machine, ce qui permet à DBeaver de se connecter au conteneur comme à n'importe quel serveur local. Je m'en sers au quotidien pour :

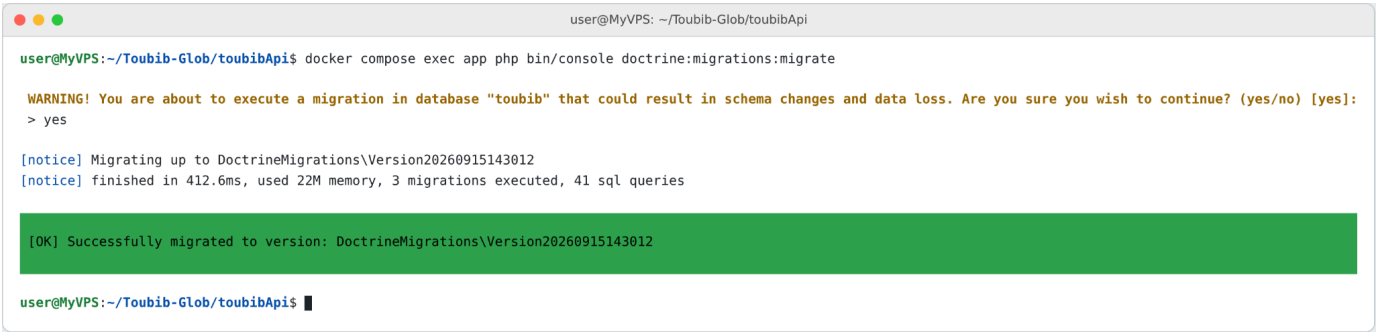
- parcourir les tables et vérifier les données enregistrées après un appel à l'API, par exemple un rendez-vous ou un jeton de rafraîchissement ;
- exécuter des requêtes SQL de contrôle, notamment pour comparer le résultat d'une requête écrite à la main avec celui du QueryBuilder ;
- contrôler la structure produite par les migrations : colonnes, index uniques, clés étrangères et règles de suppression ;
- visualiser les relations entre les tables grâce au diagramme généré automatiquement par DBeaver.


DBeaver 26.2.2 - specialties
Fichiers Edition Navigation Rechercher Editeur SQL Base de données Fenêtres Aide
Connexions Projets
Filter connections by name
Toubib-DB 192.168.1.4:3306
Bases de données
toubib
Tables
app_config 16K
appointments 64K
business_sites 32K
doctor_business_site 64K
doctors 64K
doctrine_migration_versions 16K
logging_attempt 16K
messages 64K
messenger_messages 32K
notifications 32K
patients_history 48K
toubib appointments specialties users business_sites logging_attempt
Propriétés Données Diagramme
Show SQL Entrez une expression SQL pour filtrer les résultats (utilisez Ctrl+Espace)
Tableau Texte
id name description
1 Médecine générale Soins primaires, prévention et suivi médical global
2 Cardiologie Maladies du cœur et du système cardiovasculaire
3 Pédiatrie Médecine des nourrissons, enfants et adolescents
4 Dermatologie Maladies de la peau, des phanères et des muqueuses
5 Ophtalmologie Maladies des yeux et troubles de la vision


DBeaver 26.2.2 - toubib
Fichiers Edition Navigation Rechercher Editeur SQL Base de données Fenêtres Aide
Connexions Projets
Toubib-DB 192.168.1.4:3306
Bases de données
toubib
Tables
app_config 16K
appointments 64K
business_sites 32K
doctor_business_site 64K
doctors 64K
doctrine_migration_versions 16K
logging_attempt 16K
messages 64K
messenger_messages 32K
notifications 32K
patients_history 48K
prescriptions 64K
refresh_tokens 32K
regions 16K
reviews 64K
specialties 16K
unavailability_slots 48K
user_cards 32K
users 80K
users_password_reset_token 32K
Vues
Indexes
Procédures
Triggers
Evènements
Utilisateurs
Administrer
Informations Système
Fichiers
Name DataSource
Bookmarks
Dashboards
Diagrams
Scripts
toubib appointments specialties users business_sites logging_attempt reviews
Propriétés Diagramme
users_password_reset_token
id
token
created_at
user_id
appointments
id
start_time
end_time
status
notes
patient_id
doctor_id
business_site_id
doctors
id
is_active
license_number
activity_started
biography
profile_picture
accept_new_patients
teleconsultation_enabled
verified
user_id
speciality_id
users
id
first_name
last_name
gender
email
address
phone
password
photo
biography
birth_day
date_inscription
last_login
role
is_active
is_phone_verified
is_email_verified
force_password_change
user_token
social_number
main_doctor_id
unavailability_slots
id
start_time
end_time
reason
doctor_id
business_site_id
business_sites
id
name
address
ville
phone
email
location_google_map
region_id
regions
id
name
country
prescriptions
id
medications
notes
created_at
doctor_id
patient_id
appointment_id
doctor_business_site
id
is_owner
is_primary
consultation_duration
consultation_fee
working_schedule
share_calendar
doctor_id
business_site_id
specialties
id
name
Rafraîchir Enregistrer Rétablir Keep layout 100% toubib.doctors
General Toubib-DB toubib

DBeaver me sert d'outil de vérification, toute écriture dans la base passe par l'API, et toute évolution de structure passe par une migration.

Les données sont conservées dans un **volume Docker**. Je peux donc arrêter, supprimer ou reconstruire le conteneur sans perdre le contenu de la base. À l'inverse, supprimer le volume permet de repartir d'une base vide en quelques secondes, ce qui est pratique pour tester une installation complète du projet, exemple de migration Doctrine:



4.4. Conception du Back-end API REST

4.4.1. Convention

Pour que le code de l'API reste lisible, homogène et compréhensible par n'importe quel développeur PHP, je me suis appuyé sur deux niveaux de conventions : les recommandations PSR, communes à tout l'écosystème PHP, et les standards propres à Symfony, qui les complètent.

Les recommandations PSR

Les PSR (*PHP Standards Recommendations*) sont publiées par le PHP-FIG (*PHP Framework Interop Group*), un groupe qui réunit les principaux projets de l'écosystème PHP, dont Symfony, Laravel et Composer. Leur but est que des bibliothèques écrites par des équipes différentes puissent fonctionner ensemble et se lire de la même façon. Trois d'entre elles s'appliquent directement à l'application.

PSR-1 (*Basic Coding Standard*) fixe les règles de base. Les classes sont nommées en *UpperCamelCase* (`LoginRequest`, `AppointmentsHelper`), les méthodes en *camelCase* (`findByUserId()`, `fromArray()`) et les constantes de classe en majuscules séparées par des tirets bas. Un fichier doit soit déclarer des éléments (classes, fonctions), soit produire des effets (afficher, modifier une configuration), mais jamais les deux.

PSR-4 (*Autoloader*) définit la correspondance entre l'espace de noms d'une classe et son emplacement sur le disque. Dans cette application, l'espace de noms racine `App\` correspond au dossier `src/`. La classe `App\Controller\Auth\DTO\LoginRequest` se trouve donc obligatoirement dans le fichier `src/Controller/Auth/DTO/LoginRequest.php`. Grâce à cette règle, Composer charge automatiquement chaque classe au moment où elle est utilisée, sans aucun `require` manuel, et l'emplacement d'une classe se déduit directement de son nom.

PSR-12 (*Extended Coding Style*) encadre la mise en forme du code : indentation de quatre espaces, accolade ouvrante d'une classe ou d'une méthode sur sa propre ligne, accolade d'une

structure de contrôle (`if`, `foreach`) sur la même ligne, une instruction `use` par classe importée, ou encore déclaration systématique de la visibilité (`public`, `private`) des propriétés et des méthodes.

Les standards de Symfony

Symfony reprend PSR-4 et PSR-12, et y ajoute ses propres règles. Elles sont documentées en deux endroits : les standards de code (*Coding Standards*), qui encadrent l'écriture du code, et les bonnes pratiques (*Best Practices*), qui concernent l'organisation d'une application.

Le nommage. Symfony distingue deux styles selon la nature de l'élément :

Élément	Convention	Exemple dans `ToubibAPI`
Classes, fichiers PHP	*UpperCamelCase*	`LoginRequest.php`, `DoctorsController`
Méthodes, variables, arguments	*camelCase*	`findDoctorById()`, `$doctorId`
Noms de routes, paramètres de configuration	*snake_case*	`app_auth_login`, `app_doctors_get_me`
Constantes	Majuscules et tirets bas	`ROLE_DOCTOR`
Interfaces, traits, exceptions	Suffixe `Interface`, `Trait`, `Exception`	`UserInterface`
Classes abstraites	Préfixe `Abstract`	`AbstractController`

J'ai également suivi les conventions de suffixe qui indiquent le rôle de chaque classe : `Controller` pour les contrôleurs, `Repository` pour les dépôts, `Service` pour les services métier (`AuthValidatorService`, `LoggingSecurityService`) et `Helper` pour les classes utilitaires (`AppointmentsHelper`, `FileUploadHelper`). Ce nommage permet de comprendre le rôle d'une classe sans l'ouvrir.

L'écriture du code. Les standards de Symfony imposent plusieurs règles que l'on retrouve dans le projet. Les comparaisons utilisent l'égalité stricte (`===`, `!==`), qui compare aussi le type, plutôt que l'égalité simple. Les structures de contrôle ont toujours des accolades, même pour une seule instruction. Avec la promotion des propriétés du constructeur, chaque paramètre est placé sur sa propre ligne et suivi d'une virgule, y compris le dernier : c'est exactement la forme du constructeur de `LoginRequest`. Symfony recommande aussi les conditions Yoda, qui placent la valeur constante à gauche d'une comparaison (`if (null === $user)`), pour éviter une affectation accidentelle à la place d'une comparaison. Pour vérifier et corriger automatiquement le respect de ces règles, Symfony propose l'outil PHP CS Fixer.

L'organisation de l'application. Les bonnes pratiques de Symfony ont guidé la structure de l'API :

- La structure de dossiers par défaut. Le code se trouve dans `src/`, la configuration dans `config/` et les migrations dans `migrations/`, sans réorganisation propre au projet. Un développeur Symfony s'y retrouve immédiatement.
- Des contrôleurs légers. Les contrôleurs héritent d'`AbstractController` et se limitent à orchestrer le traitement : ils valident l'entrée, délèguent la logique métier aux services et aux helpers, puis formulent la réponse.
- L'injection de dépendances et l'autowiring. Les services sont déclarés automatiquement à partir de leur type, et chaque classe reçoit ses dépendances dans son constructeur (par exemple le `ManagerRegistry` des dépôts), au lieu d'aller les chercher elle-même dans le conteneur.
- Les attributs PHP. Le *mapping* Doctrine des entités (`#[ORM\Entity]`, `#[ORM\Column]`) et les contraintes de validation (`#[Assert\NotBlank]`) sont déclarés par des attributs, directement à côté du code qu'ils décrivent.
- Les contraintes de validation sur l'objet. Elles sont placées sur les DTO et les entités, et non dans les contrôleurs, afin d'être réutilisables.
- La configuration par variables d'environnement. Les valeurs qui changent d'une machine à l'autre, comme la connexion à la base (`DATABASE_URL`), sont fournies par les fichiers `.env`.
- La sécurité. L'API utilise deux pare-feux : `login` pour les routes d'authentification, `api` pour le reste, et le *hasher* de mots de passe en mode `auto`, qui choisit l'algorithme le plus robuste disponible. Ce sont deux recommandations explicites de Symfony.

Les conventions de l'API REST

En complément, j'ai appliqué des conventions propres à la conception d'une API REST. Toutes les routes sont placées sous le préfixe `/api/` et organisées par ressource : `/api/users`, `/api/doctor`, `/api/businesssites`. Les verbes HTTP sont employés selon leur sens : `GET` pour lire, `POST` pour créer, `PATCH` pour modifier partiellement, `DELETE` pour supprimer. Le segment `/me` désigne l'utilisateur connecté, dont l'identité est déduite du jeton. Les réponses sont toujours en JSON et s'appuient sur les codes de statut HTTP standard. Enfin, les noms de routes suivent le motif `app_<ressource>_<action>`, par exemple `app_doctors_update_patient`, ce qui permet de retrouver une route sans ambiguïté dans la sortie de `debug:router`.

Sources

- PHP-FIG, *PSR-1: Basic Coding Standard* - https://www.php-fig.org/psr/psr-1/
- PHP-FIG, *PSR-4: Autoloader* - https://www.php-fig.org/psr/psr-4/
- PHP-FIG, *PSR-12: Extended Coding Style* - https://www.php-fig.org/psr/psr-12/
- PHP-FIG, *PER Coding Style* - https://www.php-fig.org/per/coding-style/
- Symfony, *Coding Standards* - https://symfony.com/doc/current/contributing/code/standards.html
- Symfony, *The Symfony Framework Best Practices* - https://symfony.com/doc/current/best_practices.html

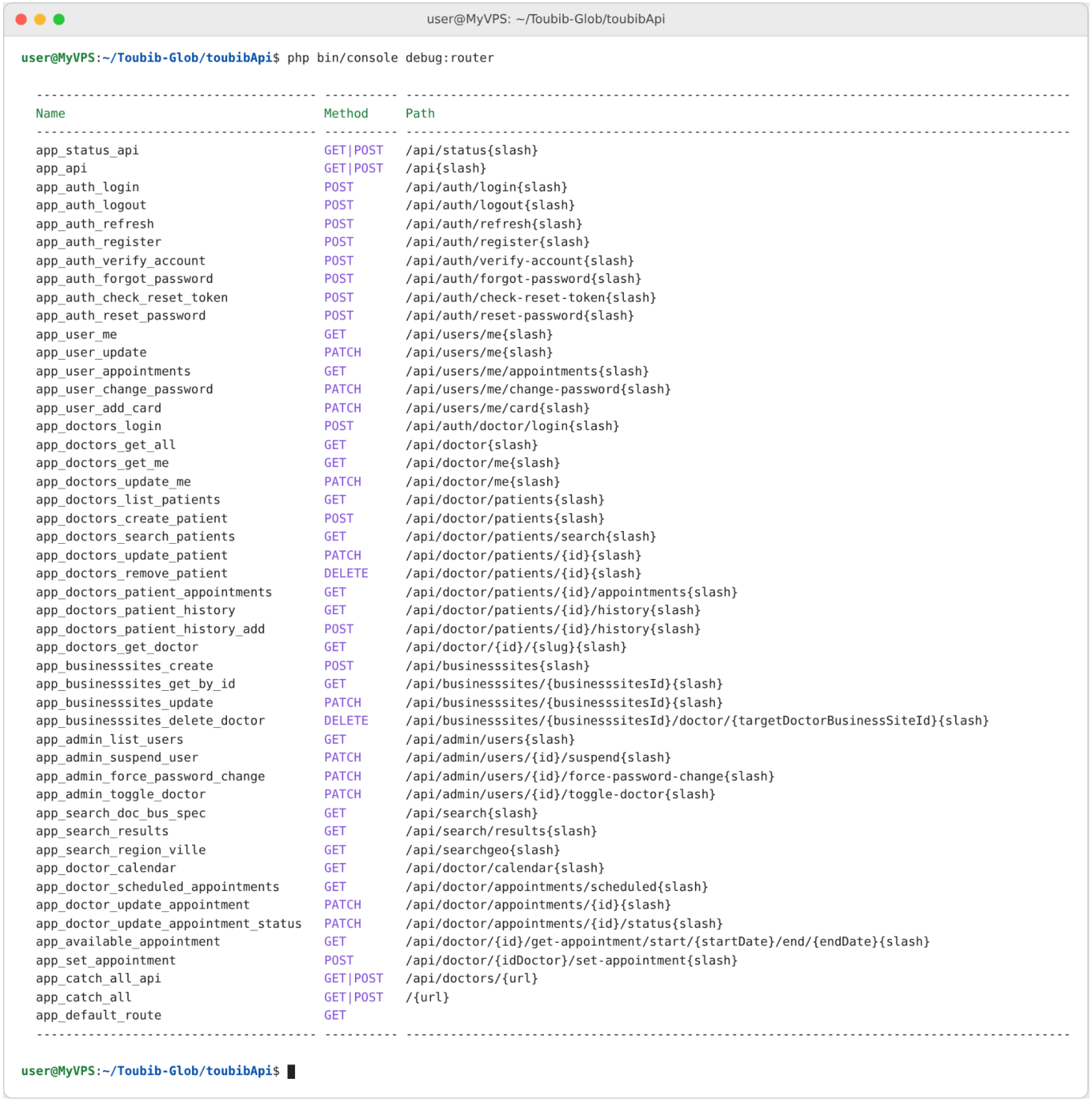
4.4.2. Routes

L'API suit les principes REST: chaque donnée est exposée comme une ressource identifiée par une URL, et l'action à réaliser est exprimée par le verbe HTTP. Toutes les routes sont placées sous le préfixe `/api/` et organisées par domaine : `/api/auth` pour l'authentification, `/api/users` pour les comptes, `/api/doctor` pour l'espace médecin, `/api/businesssites` pour les cabinets, `/api/search` pour la recherche et `/api/admin` pour l'administration. Le segment `/me` désigne l'utilisateur connecté.

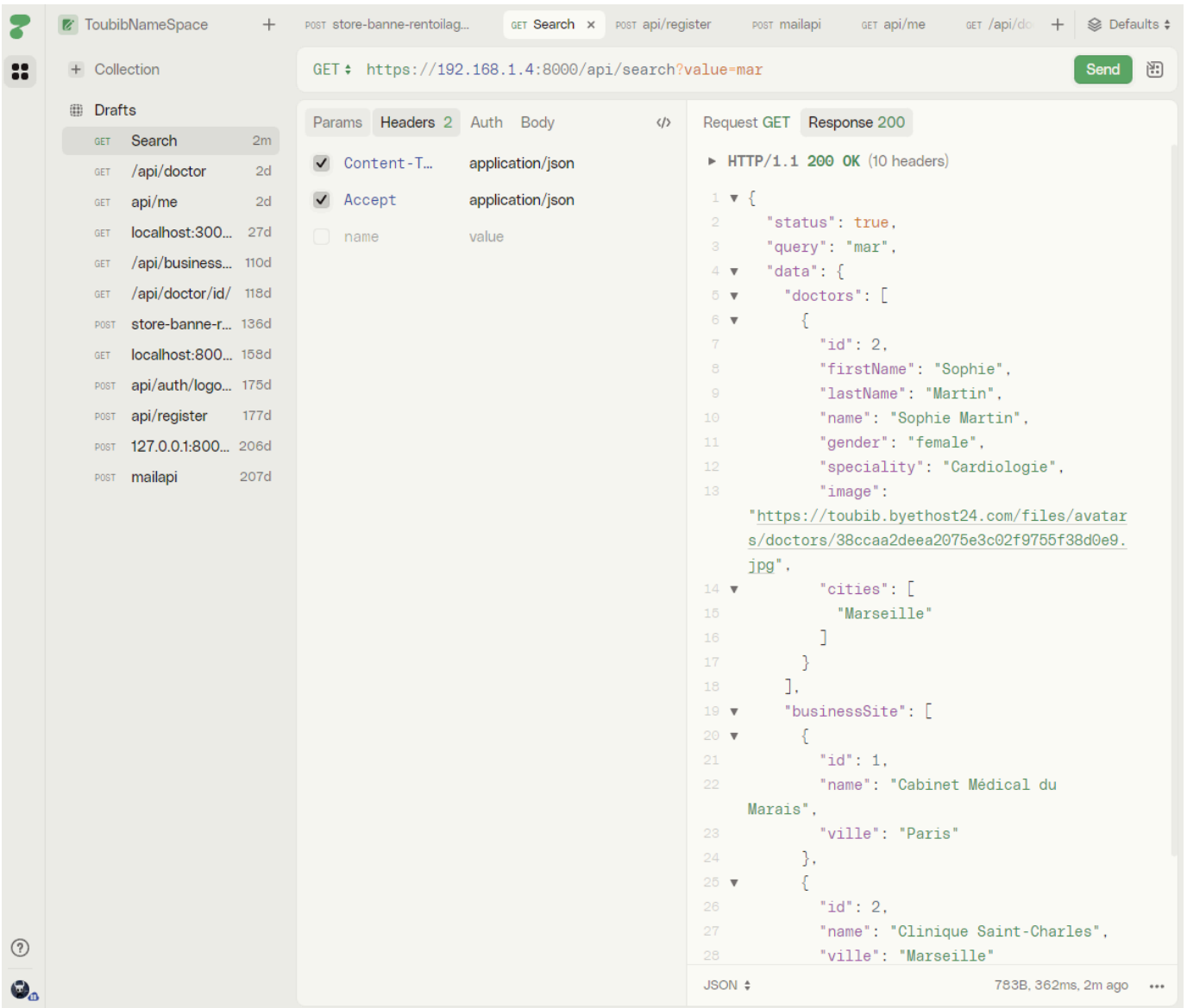
J'utilise les verbes HTTP selon leur sens. `GET` lit une ressource sans la modifier, `POST` en crée une, et `PATCH` en modifie une partie. Les réponses sont toujours au format JSON et s'appuient sur les codes de statut standard :

Code	Signification	Message
200 / 201	Succès / ressource créée	Profil renvoyé, rendez-vous créé
400	Requête invalide	Champ obligatoire manquant
401	Non authentifié	Jeton absent ou expiré
403	Accès interdit	Compte désactivé
404	Ressource introuvable	Patient d'un autre médecin
409	Conflit	Créneau déjà réservé
429	Trop de requêtes	Trop de tentatives de connexion
500	Erreur interne du serveur	Erreur API

Chaque requête traverse les mêmes couches. Le pare-feu authentifie l'utilisateur, le routeur désigne le contrôleur, puis le Controller orchestre le traitement : il valide les données entrantes au moyen d'un DTO, délègue la logique métier aux services et aux helpers, et récupère les données par les dépôts. Avant d'être renvoyées, les entités sont sérialisées en JSON à l'aide de groupes de sérialisation, qui n'exposent que les champs explicitement autorisés. Les listes volumineuses, comme les résultats de recherche ou les patients, sont paginées côté serveur.

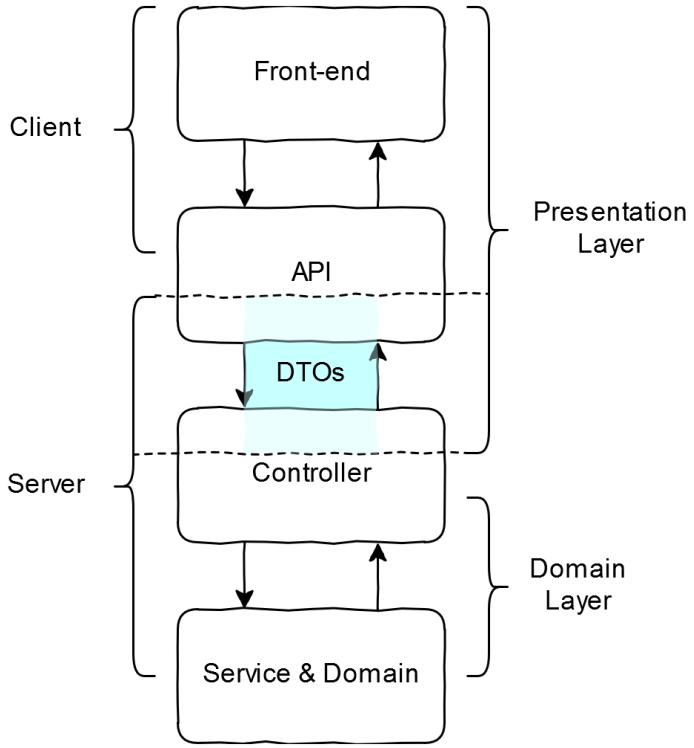

user@MyVPS: ~/Toubib-Glob/toubibApi
user@MyVPS:~/Toubib-Glob/toubibApi$ php bin/console debug:router
Name Method Path
app_status_api GET|POST /api/status{slash}
app_api GET|POST /api{slash}
app_auth_login POST /api/auth/login{slash}
app_auth_logout POST /api/auth/logout{slash}
app_auth_refresh POST /api/auth/refresh{slash}
app_auth_register POST /api/auth/register{slash}
app_auth_verify_account POST /api/auth/verify-account{slash}
app_auth_forgot_password POST /api/auth/forgot-password{slash}
app_auth_check_reset_token POST /api/auth/check-reset-token{slash}
app_auth_reset_password POST /api/auth/reset-password{slash}
app_user_me GET /api/users/me{slash}
app_user_update PATCH /api/users/me{slash}
app_user_appointments GET /api/users/me/appointments{slash}
app_user_change_password PATCH /api/users/me/change-password{slash}
app_user_add_card PATCH /api/users/me/card{slash}
app_doctors_login POST /api/auth/doctor/login{slash}
app_doctors_get_all GET /api/doctor{slash}
app_doctors_get_me GET /api/doctor/me{slash}
app_doctors_update_me PATCH /api/doctor/me{slash}
app_doctors_list_patients GET /api/doctor/patients{slash}
app_doctors_create_patient POST /api/doctor/patients{slash}
app_doctors_search_patients GET /api/doctor/patients/search{slash}
app_doctors_update_patient PATCH /api/doctor/patients/{id}{slash}
app_doctors_remove_patient DELETE /api/doctor/patients/{id}{slash}
app_doctors_patient_appointments GET /api/doctor/patients/{id}/appointments{slash}
app_doctors_patient_history GET /api/doctor/patients/{id}/history{slash}
app_doctors_patient_history_add POST /api/doctor/patients/{id}/history{slash}
app_doctors_get_doctor GET /api/doctor/{id}/{slug}{slash}
app_businesssites_create POST /api/businesssites{slash}
app_businesssites_get_by_id GET /api/businesssites/{businesssitesId}{slash}
app_businesssites_update PATCH /api/businesssites/{businesssitesId}{slash}
app_businesssites_delete_doctor DELETE /api/businesssites/{businesssitesId}/doctor/{targetDoctorBusinessSiteId}{slash}
app_admin_list_users GET /api/admin/users{slash}
app_admin_suspend_user PATCH /api/admin/users/{id}/suspend{slash}
app_admin_force_password_change PATCH /api/admin/users/{id}/force-password-change{slash}
app_admin_toggle_doctor PATCH /api/admin/users/{id}/toggle-doctor{slash}
app_search_doc_bus_spec GET /api/search{slash}
app_search_results GET /api/search/results{slash}
app_search_region_ville GET /api/searchgeo{slash}
app_doctor_calendar GET /api/doctor/calendar{slash}
app_doctor_scheduled_appointments GET /api/doctor/appointments/scheduled{slash}
app_doctor_update_appointment PATCH /api/doctor/appointments/{id}{slash}
app_doctor_update_appointment_status PATCH /api/doctor/appointments/{id}/status{slash}
app_available_appointment GET /api/doctor/{id}/get-appointment/start/{startDate}/end/{endDate}{slash}
app_set_appointment POST /api/doctor/{idDoctor}/set-appointment{slash}
app_catch_all_api GET|POST /api/doctors/{url}
app_catch_all GET|POST /{url}
app_default_route GET
user@MyVPS:~/Toubib-Glob/toubibApi$

Pour tester l'API pendant le développement, j'ai utilisé HTTPie, un client HTTP comparable à Postman. Il permet d'envoyer des requêtes aux différents points d'entrée de l'API, sans passer par les interfaces React, et d'inspecter précisément les réponses : code de statut, en-têtes et corps JSON. Je m'en suis servi pour vérifier chaque route au fur et à mesure de son développement : contrôler qu'une requête valide renvoie bien les données attendues, qu'un champ manquant déclenche une erreur 400, ou qu'une route protégée refuse l'accès avec un code 401 lorsque le jeton est absent. HTTPie m'a aussi permis de suivre le mécanisme d'authentification : après la connexion, il conserve les cookies `app_auth` et `app_refresh` reçus et les renvoie automatiquement dans les requêtes suivantes, ce qui m'a permis de tester les routes protégées et le renouvellement du jeton. Tester l'API de manière isolée m'a permis de distinguer rapidement un problème côté serveur d'un problème côté interface.



4.4.3. DTO

Un DTO (*Data Transfer Object*, ou objet de transfert de données) est une classe très simple, sans logique métier, qui sert uniquement à transporter des données d'un endroit à un autre. Dans une API, il décrit précisément la forme des données qu'un point d'entrée accepte en entrée : quels champs, de quel type, avec quelles contraintes.



Sans DTO, un Controller lit directement le JSON envoyé par le client, par exemple avec `json_decode($request->getContent(), true)`, puis va chercher chaque clé à la main dans le tableau obtenu. Cette approche pose trois problèmes :

- Rien ne garantit la forme des données. Un champ peut manquer, être du mauvais type ou contenir des clés en trop, et chaque Controller doit le vérifier lui-même.
- La validation est dispersée dans le code, avec des `if` répétés d'un Controller à l'autre.
- La tentation de remplir directement une entité avec les données reçues. C'est dangereux : un client malveillant pourrait envoyer un champ `roles` ou `isActive` et modifier des données. C'est ce qu'on appelle l'assignation de masse (*mass assignment*).

Le DTO s'interpose entre la requête et l'entité. Prenons `LoginRequest`, utilisé par la route de connexion :

src > Controller > Auth > DTO > LoginRequest.php

```
You, 2 months ago | author (You)
<?php       You, 2 months ago • fix(refacto): code refactorisation

namespace App\Controller\Auth\DTO;

use Symfony\Component\Validator\Constraints as Assert;

class LoginRequest
{
    public function __construct(
        #[Assert\NotBlank(message: 'Nom d\'utilisateur requis')]
        #[Assert\Email(message: 'Format d\'email invalide')]
        public readonly string $username,

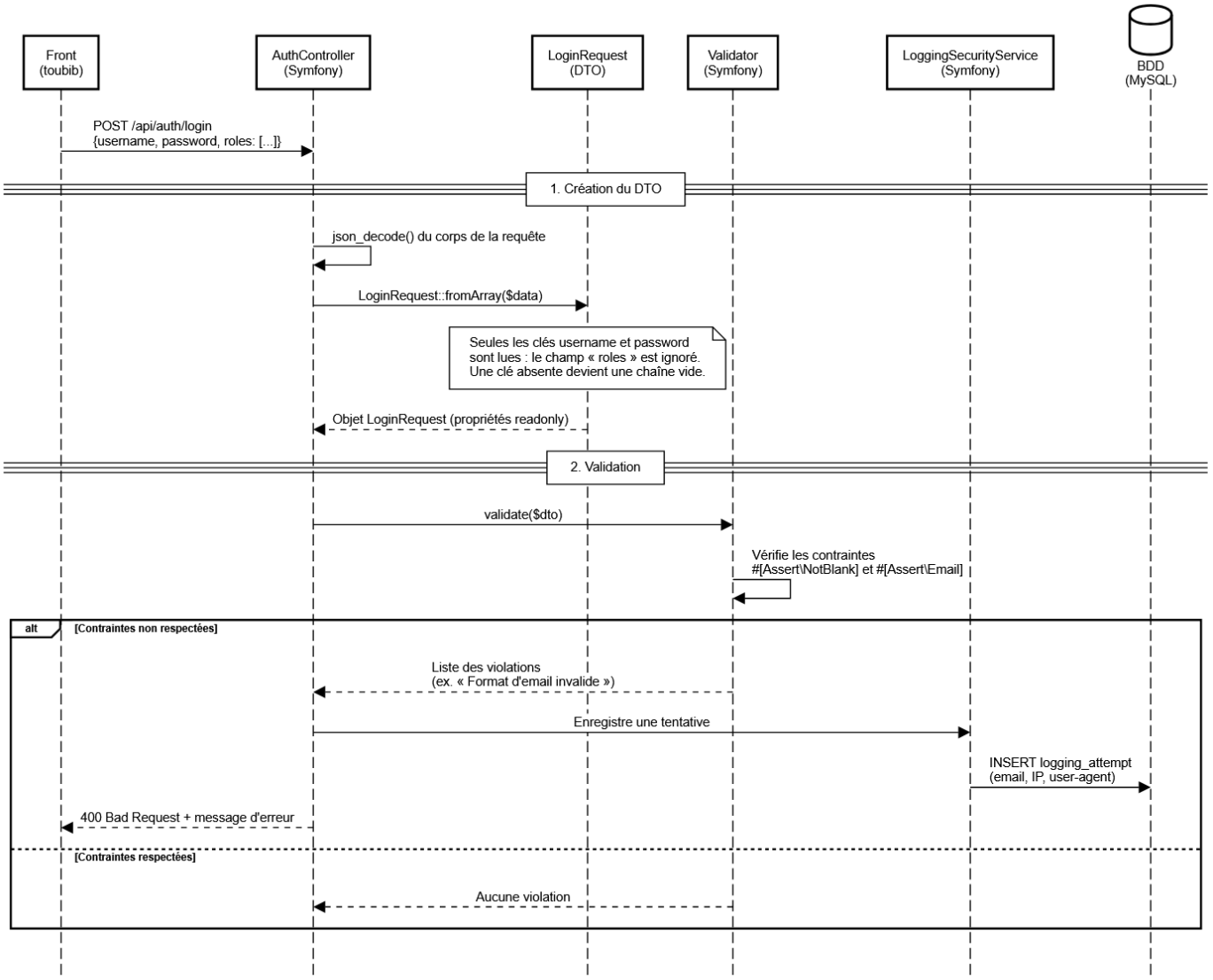
        #[Assert\NotBlank(message: 'Mot de passe requis')]
        public readonly string $password,
    ) {}

    public static function fromArray(array $data): self
    {
        return new self(
            username: $data['username'] ?? '',
            password: $data['password'] ?? '',
        );
    }
}

```

Le traitement suit trois étapes :

1. La création du DTO: le Controller décode le JSON reçu, puis le transmet à la méthode statique `fromArray()`, qui construit l'objet `LoginRequest`. Seules les clés `username` et `password` sont lues : tout autre champ envoyé par le client est ignoré. Si une clé est absente, elle est remplacée par une chaîne vide, que la contrainte `NotBlank` rejettera à l'étape suivante.
2. La validation: le Validator de Symfony vérifie les contraintes déclarées par les attributs (`NotBlank`, `Email`). Si une règle n'est pas respectée, l'API répond 400 avec le message correspondant, par exemple « Format d'email invalide », avant tout traitement, c'est aussi à ce moment qu'une tentative est enregistrée dans `logging_attempt`.
3. L'utilisation: le Controller et les services manipulent un objet typé et déjà validé. Ils écrivent `$dto->username` au lieu de `$data['username'] ?? null`, et l'éditeur de code peut les aider grâce aux types. Comme les propriétés sont déclarées `readonly`, les valeurs validées ne peuvent plus être modifiées par la suite.



Le diagramme de séquence ci-dessus détaille le traitement d'un DTO lors de la connexion, à travers six participants : le front, l'`AuthController`, le DTO `LoginRequest`, le Validator de Symfony, le `LoggingSecurityService` et la base de données.

Le traitement se déroule en deux phases:

- Lors de la création du DTO, le contrôleur décode le JSON reçu et le transmet à la méthode `fromArray()`, qui ne lit que les champs `username` et `password`. Tout autre champ envoyé par le client est ignoré, et un champ absent est remplacé par une chaîne vide.
- Lors de la validation, le contrôleur soumet le DTO au Validator, qui vérifie les contraintes déclarées par les attributs `NotBlank` et `Email`.

Le diagramme montre alors deux scénarios, représentés par un bloc `alt`. Si une contrainte n'est pas respectée, le contrôleur enregistre une tentative dans la table `logging_attempt` par l'intermédiaire du `LoggingSecurityService`, puis renvoie une erreur 400 accompagnée d'un message explicite, sans poursuivre le traitement. Si les données sont valides, on entre dans la phase

d’utilisation : le contrôleur manipule un objet typé et déjà contrôlé pour poursuivre la connexion (limite de tentatives, vérification du mot de passe, création des jetons), puis renvoie une réponse 200 avec les cookies d’authentification. Ce diagramme met en évidence le rôle du DTO comme point de contrôle unique à l’entrée de l’API : aucune donnée n’atteint la logique métier sans avoir été filtrée et validée.

4.4.4. Authentification

L'authentification repose sur des jetons, générés par le bundle de référence pour l'authentification JWT avec Symfony `LexikJWTAuthentication`. Le pare-feu de l'API est configuré en mode sans état (`stateless`) : aucune session n'est conservée côté serveur, et chaque requête est authentifiée par le jeton qu'elle transporte.

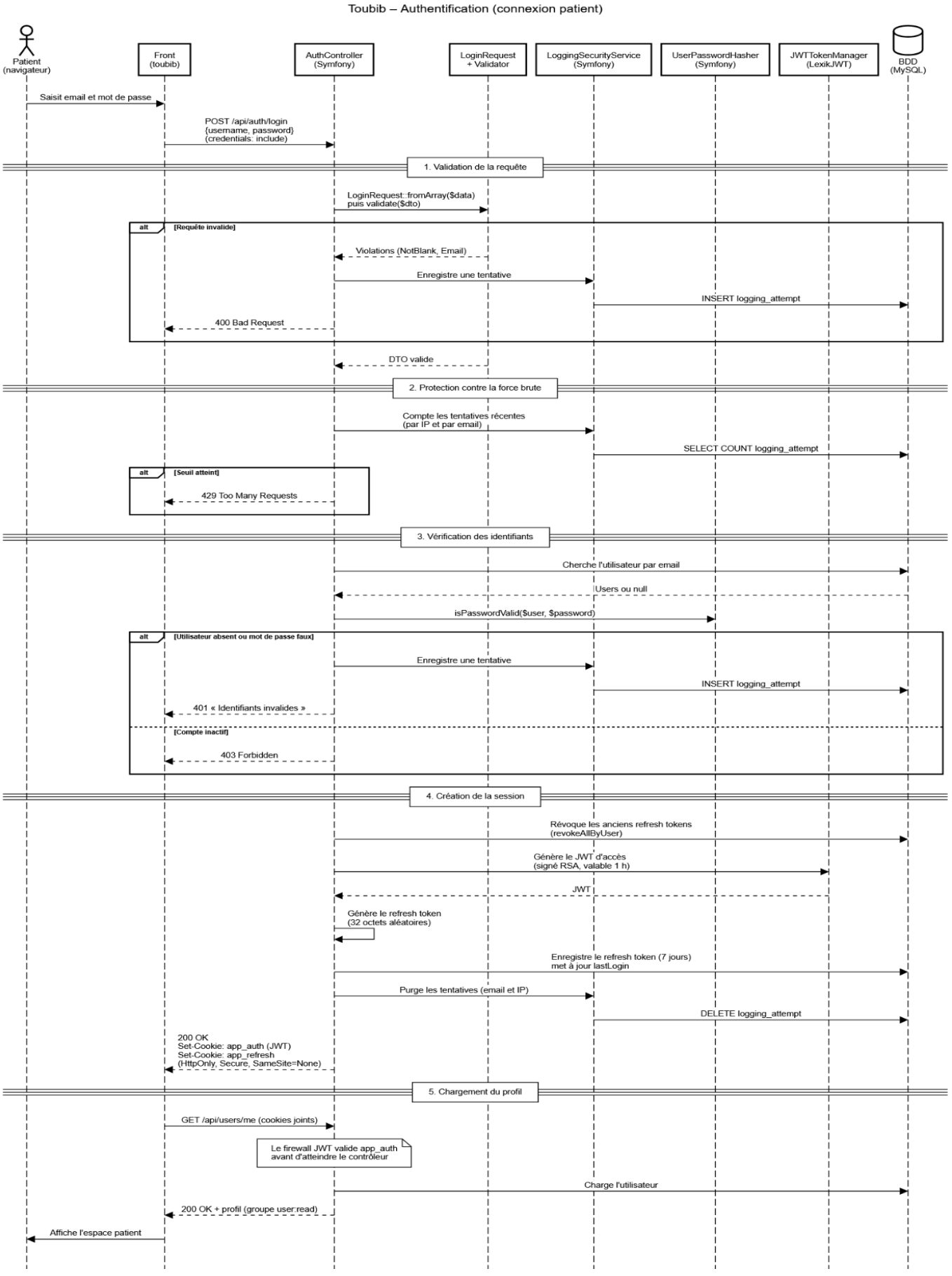
À la connexion, l'API vérifie d'abord que le nombre de tentatives récentes, par adresse IP et par adresse électronique, ne dépasse pas le seuil autorisé. Elle contrôle ensuite les identifiants et l'état du compte. Si tout est valide, elle révoque les anciennes sessions de l'utilisateur et génère deux jetons :

- un jeton d'accès, signé avec une clé privée et valable une heure, placé dans le cookie `app_auth`
- un jeton de rafraîchissement, aléatoire, enregistré en base et valable sept jours, placé dans le cookie `app_refresh`.

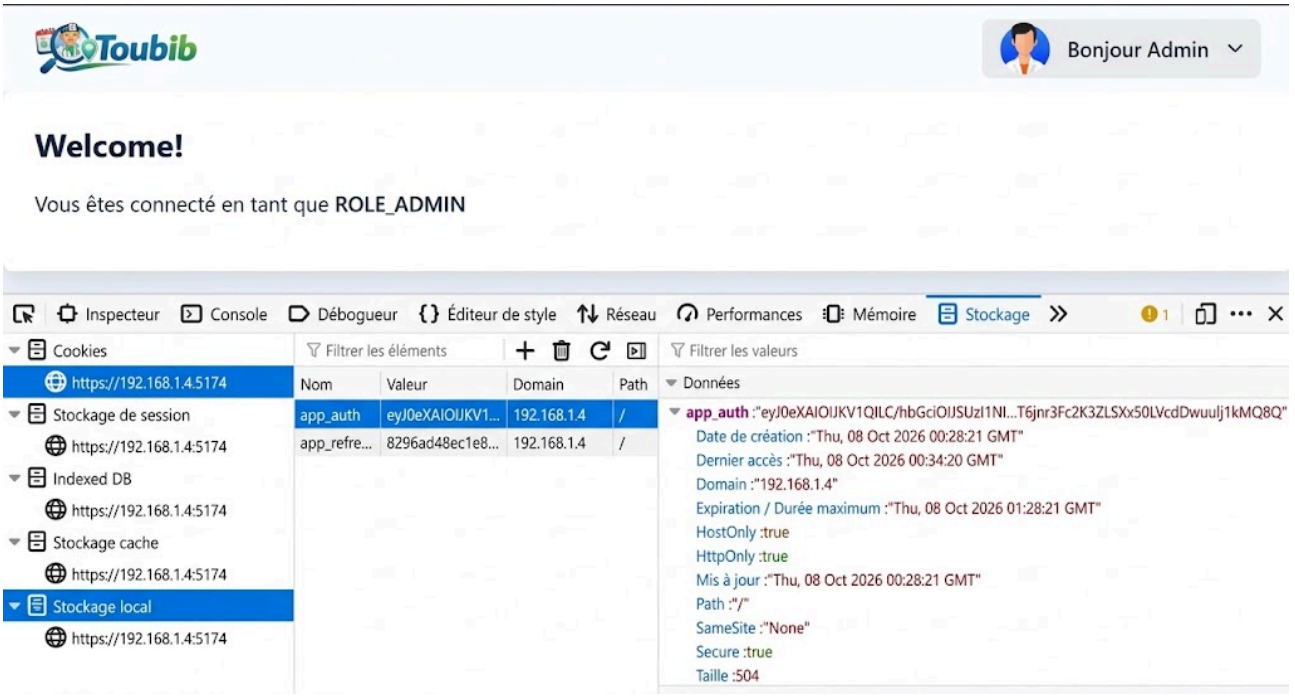
Les deux cookies sont marqués `HttpOnly`, `Secure` et `SameSite=None` : ils ne sont pas lisibles par le JavaScript, ce qui les protège contre le vol par injection de script (XSS), et ne circulent que sur HTTPS. Deux routes de connexion distinctes séparent les profils : `/api/auth/login` pour les patients, et `/api/auth/doctor/login`, qui n'accepte que les rôles `ROLE_DOCTOR` et `ROLE_ADMIN`.

Lorsque le jeton d'accès expire, l'API répond 401, donc le front appelle alors automatiquement `POST /api/auth/refresh`, qui vérifie le jeton de rafraîchissement en base, délivre un nouveau jeton d'accès, puis le front rejoue la requête initiale. L'utilisateur n'a donc pas à se reconnecter toutes les heures.

La déconnexion révoque le jeton de rafraîchissement en base et expire les deux cookies, voir le diagramme de séquence suivant: (voir la page suivante)


Toubib – Authentification (connexion patient)
Patient (navigateur)
Front (toubib)
AuthController (Symfony)
LoginRequest + Validator
LoggingSecurityService (Symfony)
UserPasswordHasher (Symfony)
JWTTokenManager (LexikJWT)
BDD (MySQL)
Saisit email et mot de passe
POST /api/auth/login {username, password} (credentials: include)
1. Validation de la requête
LoginRequest::fromArray($data) puis validate($dto)
alt [Requête invalide]
Violations (NotBlank, Email)
Enregistre une tentative
INSERT logging_attempt
400 Bad Request
DTO valide
2. Protection contre la force brute
Compte les tentatives récentes (par IP et par email)
SELECT COUNT logging_attempt
alt [Seuil atteint]
429 Too Many Requests
3. Vérification des identifiants
Cherche l'utilisateur par email
Users ou null
isPasswordValid($user, $password)
alt [Utilisateur absent ou mot de passe faux]
Enregistre une tentative
INSERT logging_attempt
401 « Identifiants invalides »
[Compte inactif]
403 Forbidden
4. Création de la session
Révoque les anciens refresh tokens (revokeAllByUser)
Génère le JWT d'accès (signé RSA, valable 1 h)
JWT
Génère le refresh token (32 octets aléatoires)
Enregistre le refresh token (7 jours) met à jour lastLogin
Purge les tentatives (email et IP)
DELETE logging_attempt
200 OK
Set-Cookie: app_auth (JWT)
Set-Cookie: app_refresh
(HttpOnly, Secure, SameSite=None)
5. Chargement du profil
GET /api/users/me (cookies joints)
Le firewall JWT valide app_auth avant d'atteindre le contrôleur
Charge l'utilisateur
200 OK + profil (groupe user:read)
Affiche l'espace patient

Le résultat après une authentification réussie:



config > packages > ! security.yaml

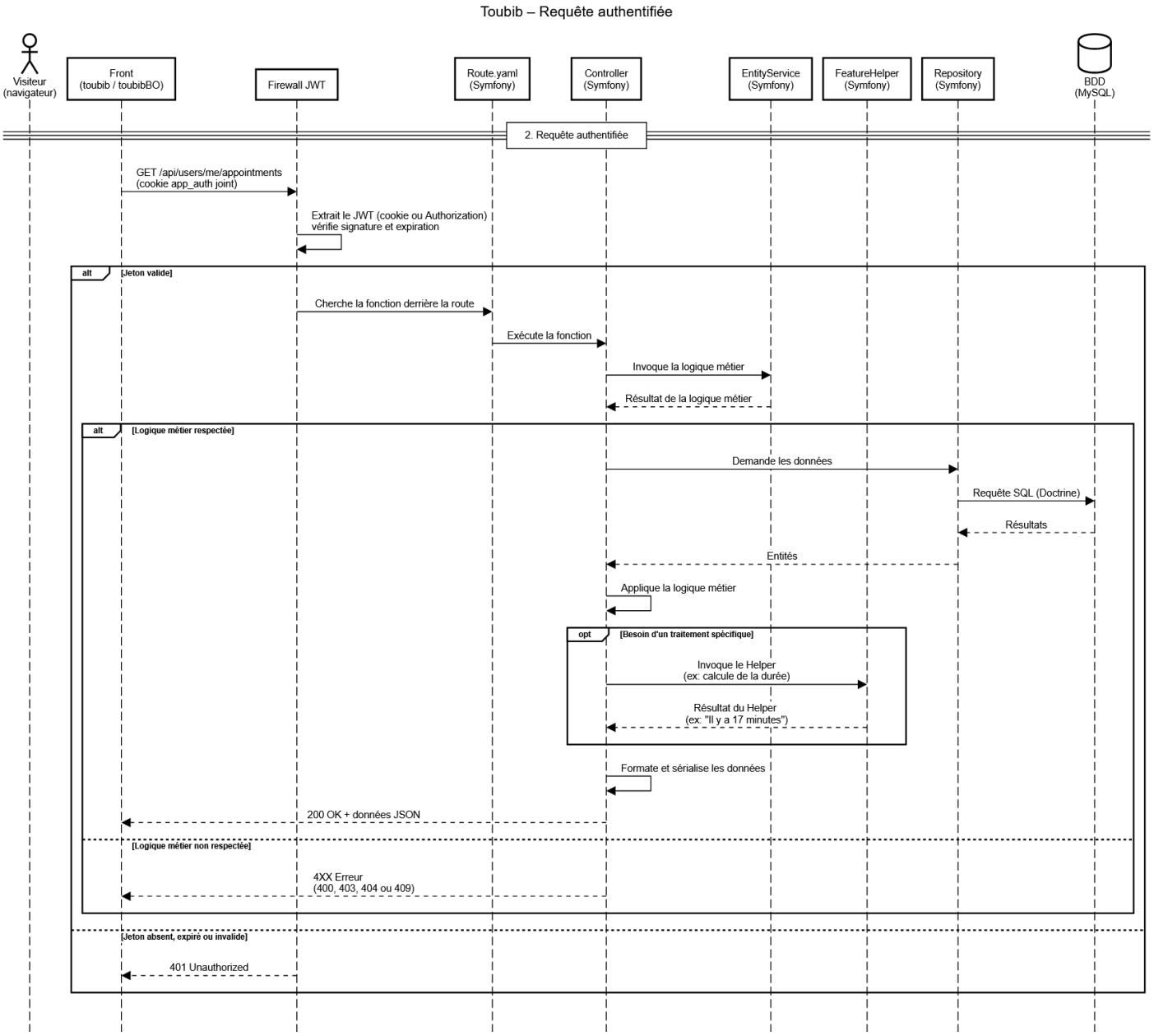
You, 3 months ago | authors (Yassine ECHCHOUROUQ and one other)

```yaml
security:       Yassine ECHCHOUROUQ, 6 months ago • app(arch): main back app architecture with full…
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: "auto"

    providers:
        app_user_provider:
            entity:
                class: App\Entity\Users
                property: email

    firewalls:
        login:
            pattern: ^/api/auth
            stateless: true
            security: false
        api:
            pattern: ^/api
            stateless: true
            provider: app_user_provider
            jwt: ~
```

Diagramme de séquence: **Requête authentifiée**



Le diagramme de séquence ci-dessus présente le cheminement d'une requête authentifiée, par exemple la consultation des rendez-vous d'un patient (`GET /api/users/me/appointments`).

Le pare-feu JWT intercepte d'abord la requête et vérifie le contenu dans le cookie `app_auth`. S'il est absent, expiré ou invalide, il répond directement par une erreur 401. S'il est valide, le routeur transmet la requête au contrôleur, qui interroge le service chargé de la logique métier, éventuellement un helper, puis demande les données au repository, qui interroge la base de données.

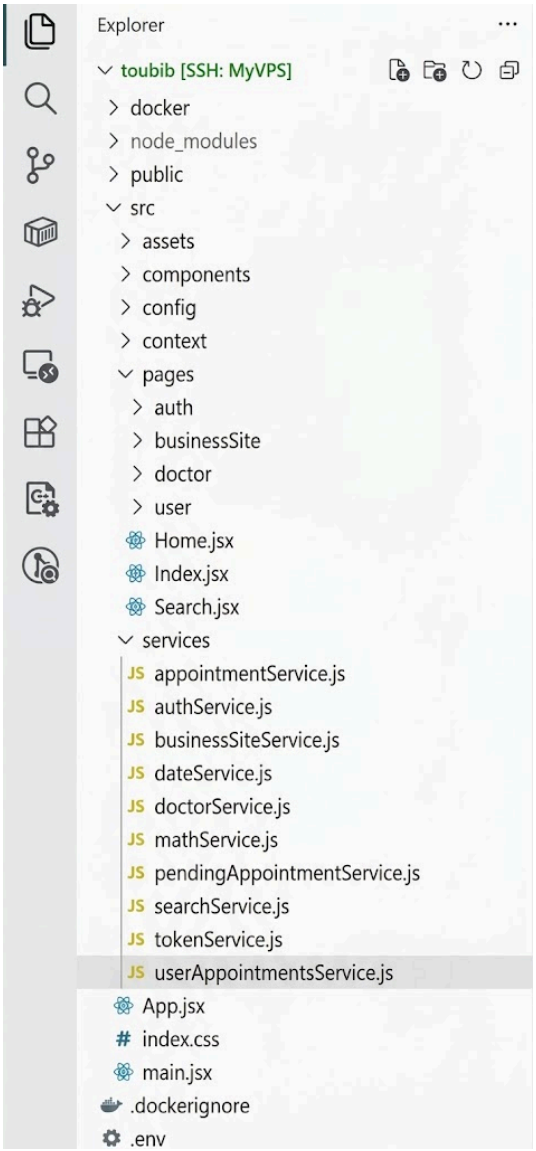
Le contrôleur renvoie enfin une réponse 200 au format JSON, ou une erreur 4XX si une règle métier n'est pas respectée. Chaque couche a ainsi un rôle bien délimité : le pare-feu assure la sécurité, le service décide, le repository accède aux données, et le contrôleur coordonne l'ensemble.

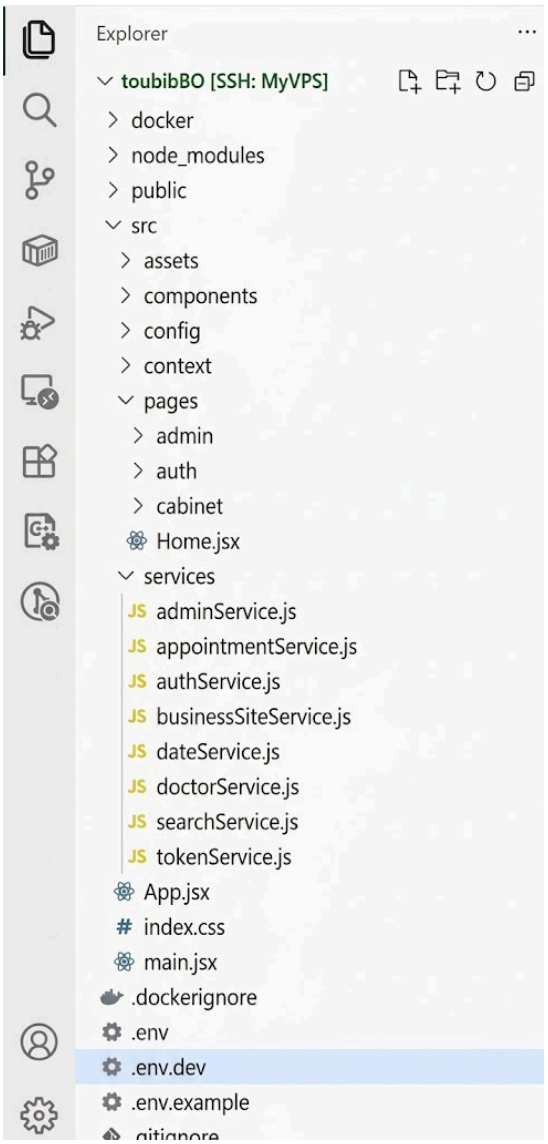
4.5. Conception du Front-end React

4.5.1. Convention

React est une bibliothèque, et non un framework complet : il se concentre sur la construction de l'interface et n'impose ni organisation des fichiers ni conventions de nommage. Un framework comme Next.js, construit sur React, ou comme Angular définit au contraire une structure à respecter : dans Next.js, par exemple, l'arborescence des dossiers détermine directement les routes de l'application. Avec React, ces choix reviennent au développeur. J'ai donc défini mes propres conventions, en m'inspirant des pratiques courantes de la communauté. Le code est rangé par rôle : les composants réutilisables, les pages associées à une route, les services d'appel à l'API et le contexte d'authentification occupent chacun leur dossier. Les routes sont déclarées explicitement avec React Router, et les noms de fichiers et de composants suivent les mêmes règles dans les deux applications. Cette liberté m'a obligé à réfléchir à l'architecture dès le départ, et à appliquer ces conventions avec rigueur pour que le code reste lisible et cohérent d'un projet à l'autre.

Ce qui distingue les deux applications, c'est leur public, et donc leurs écrans et leurs priorités d'ergonomie.





Les deux interfaces de l'application partagent le même socle technique. Elles suivent aussi les mêmes principes. Chacune est une application monopage qui ne contient aucune logique métier : elle affiche les données renvoyées par l'API et lui transmet les actions de l'utilisateur. Toutes les requêtes passent par des fonctions de service, envoyées avec l'option `credentials: include` pour que le navigateur joigne automatiquement les cookies d'authentification. L'utilisateur connecté est partagé dans toute l'application grâce à un contexte d'authentification.

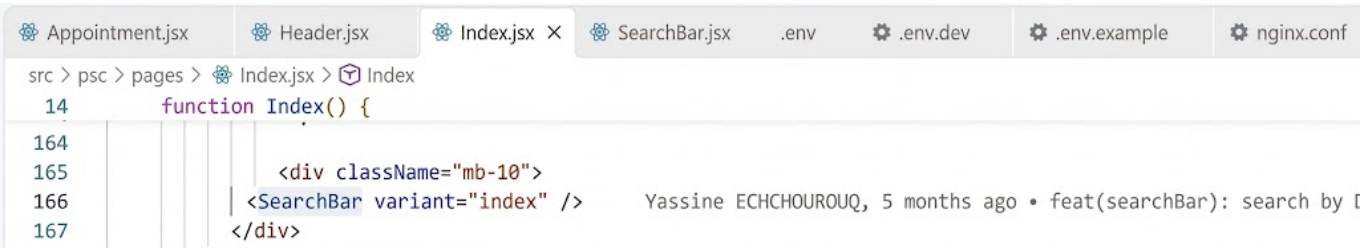
4.5.2. Toubib

`Toubib` est l'application destinée aux patients et aux visiteurs. Son objectif est de permettre de trouver un médecin et de réserver un rendez-vous en un minimum d'étapes.

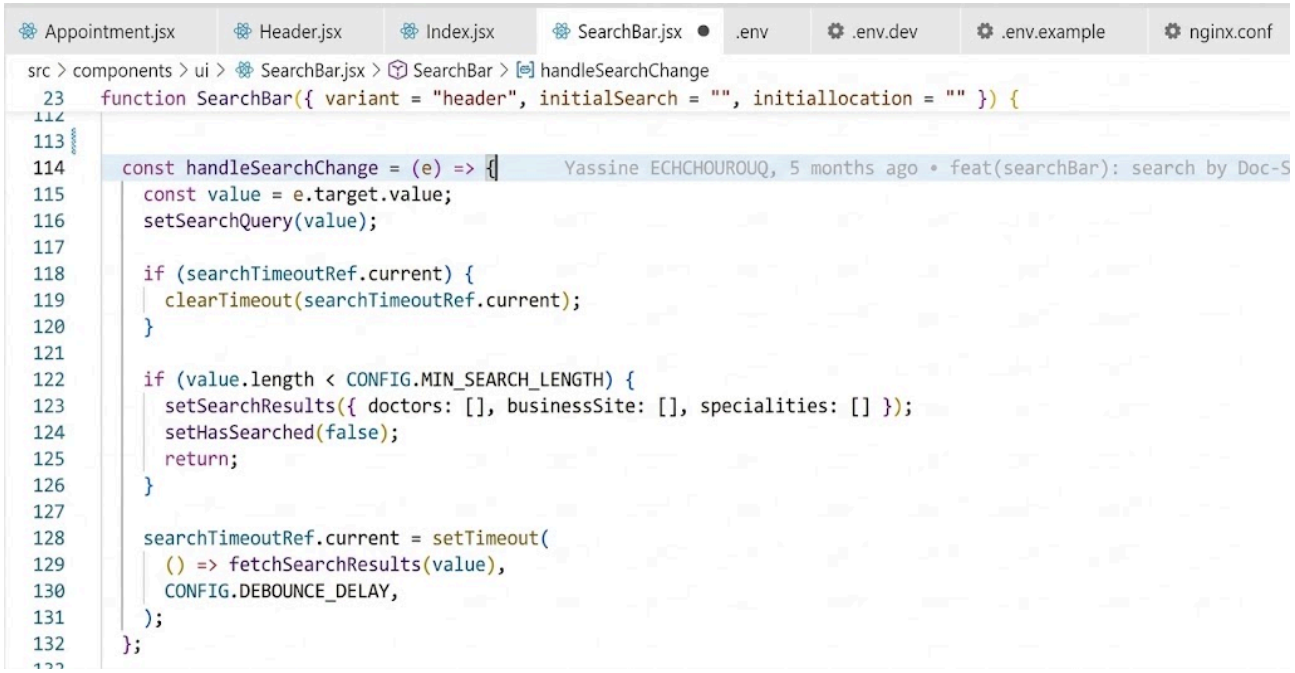
Sur la barre de recherche de la page d'accueil, le visiteur peut saisir un nom, une spécialité ou une ville. La recherche ne se déclenche qu'à partir de trois caractères afin d'éviter les requêtes inutiles. Les résultats de recherche incluent les médecins, les cabinets et les spécialités. Un nombre minimum de résultats est affiché au fur et à mesure de la saisie.

- Si l'utilisateur clique sur un résultat, il est redirigé directement vers la page de l'élément sélectionné.
- S'il valide sa recherche *(submit)*, il est redirigé vers une page dédiée affichant les résultats détaillés.
- Les résultats sont chargés et affichés page par page.

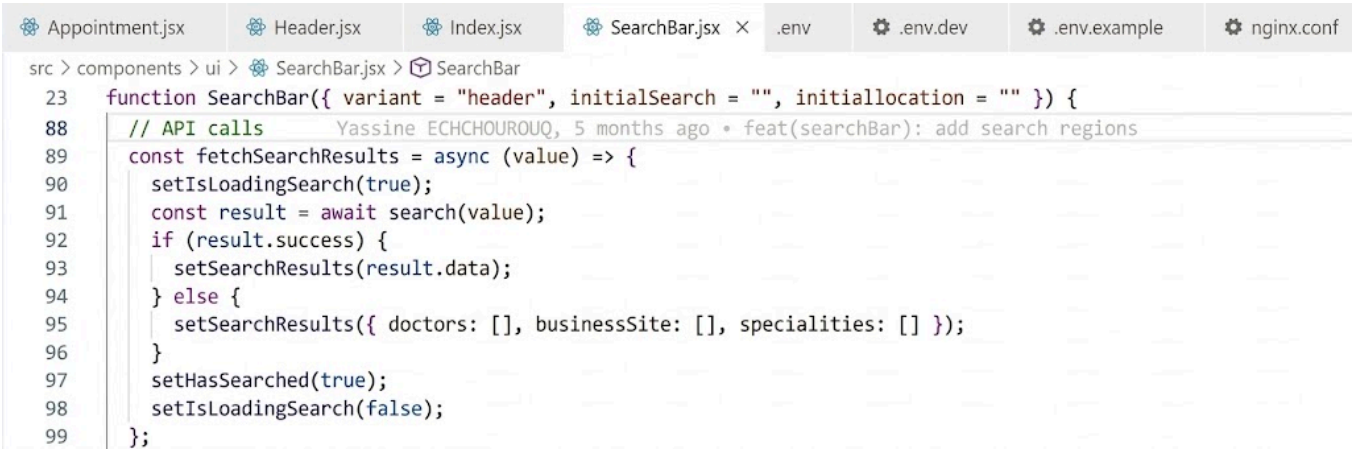
la page d'accueil contient le *component* SearchBar: **Index.jsx:166**



le *component* contient la fonction handleSearchChange(), qui se déclenche si la valeur du input est changé: **SearchBar.jsx:114**, si il n'y a pas une recherche en cours **SearchBar.jsx:118**, Alors fetchSearchResults() **SearchBar.jsx:129**

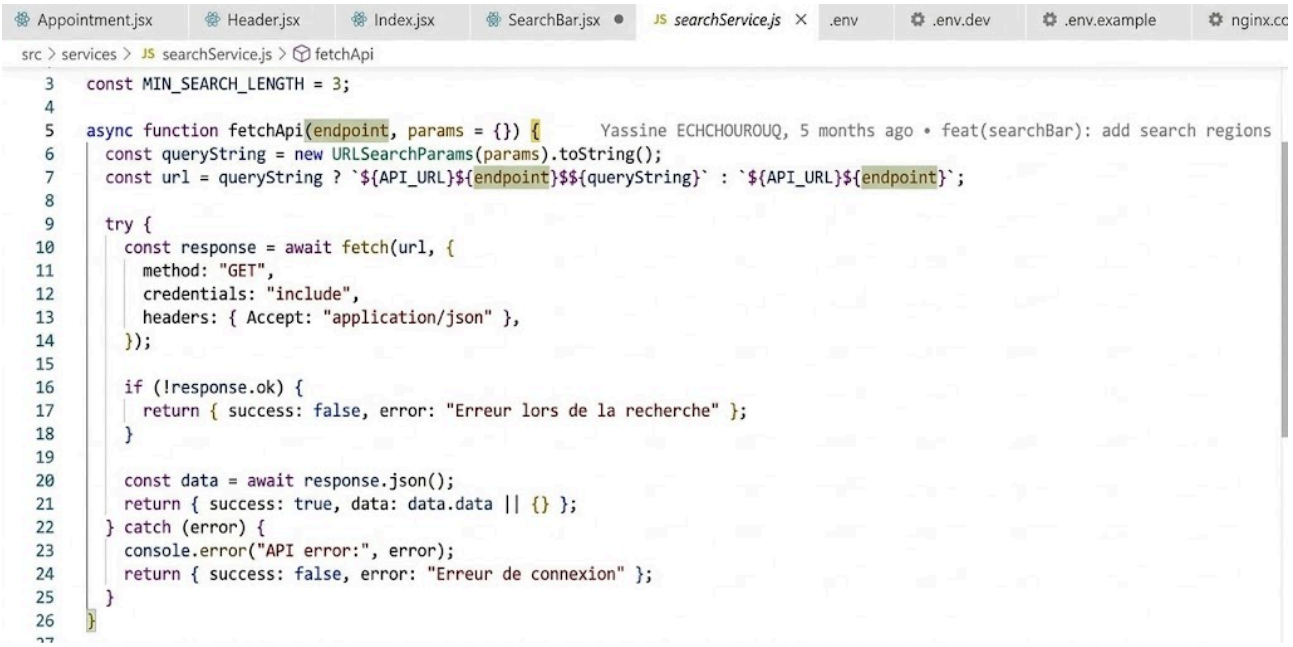


la fonction fetchSearchResult() est appelé **SearchBar.jsx:89** enveloppe la fonction search() la fonction search dans un context asynchrone **SearchBar.jsx:91**, la fonction search() est appelé depuis le service dédié aux recherches.



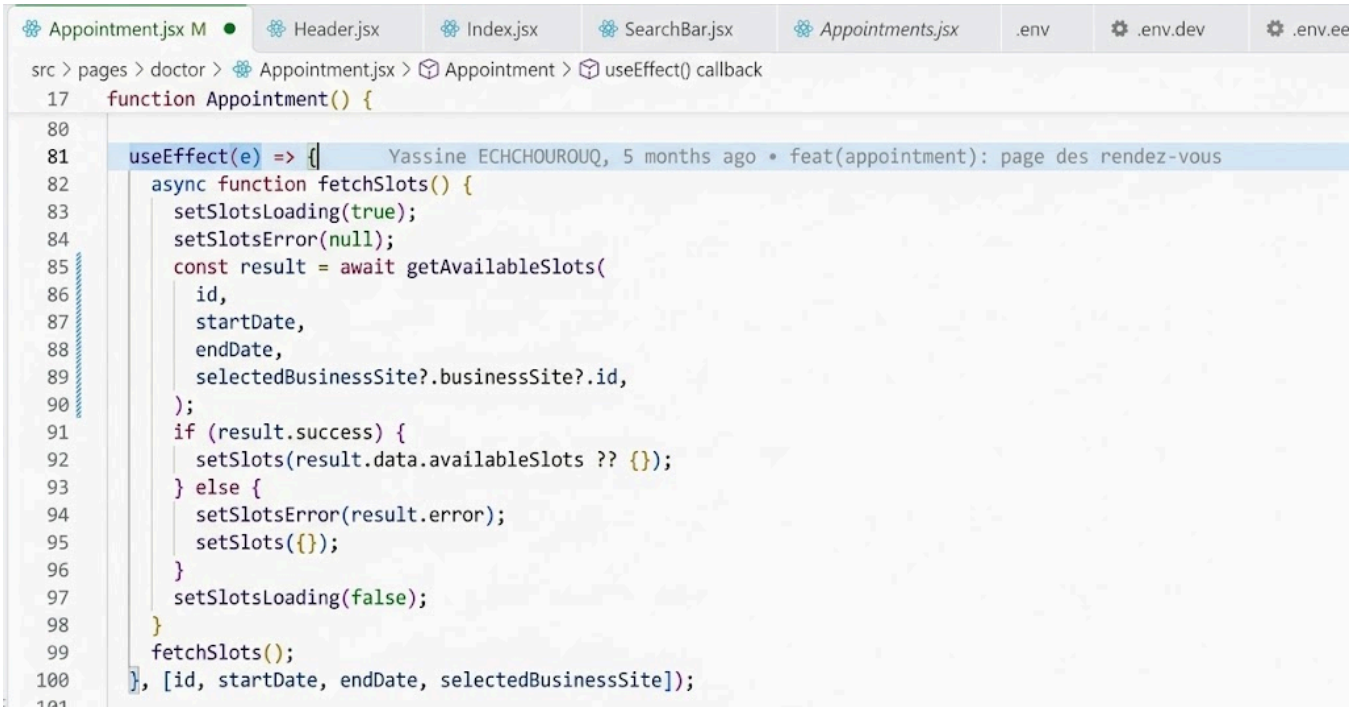
search() est donc un appel vers la fonction **search() searchService.jsx:28**, qui fait appel à la fonction **fetchApi():32**



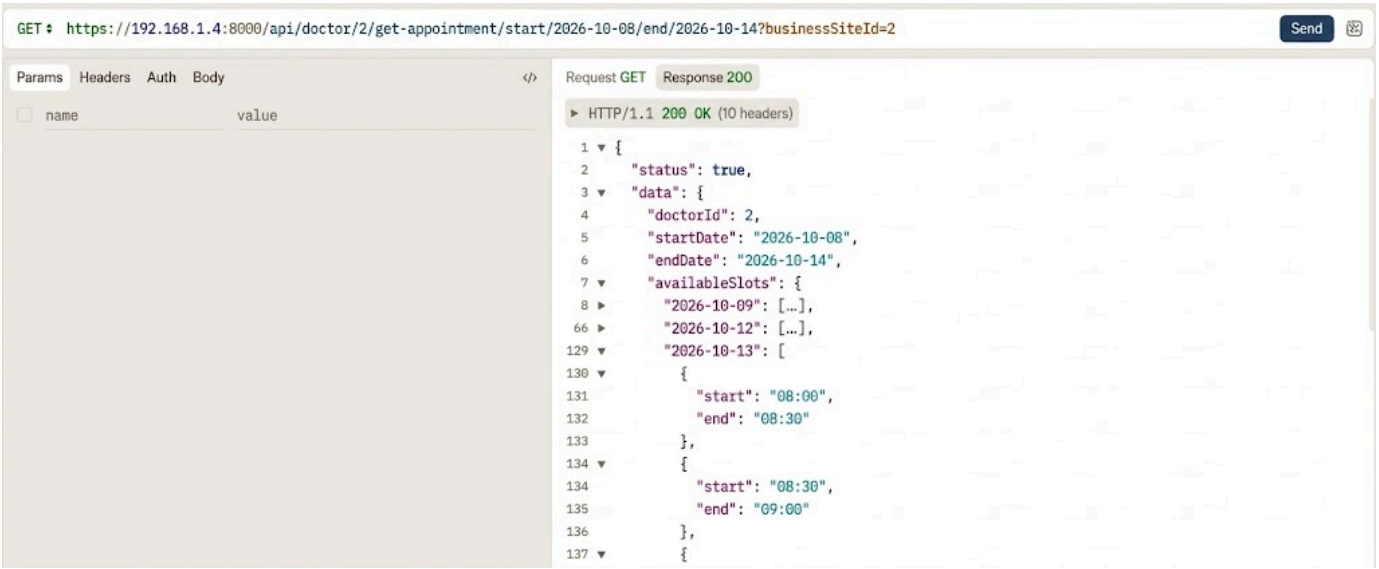


Cette combinaison des couches permet la réutilisation du code à plusieurs endroits dans le projet.

La fiche du médecin présente son profil, ses cabinets et une grille de créneaux. Chaque changement de date déclenche un nouvel appel à l'API, et la grille n'affiche que les créneaux calculés par le serveur : le front ne calcule jamais une disponibilité. Enfin, la réservation exige que le patient soit connecté. S'il ne l'est pas, il est invité à se connecter avant de confirmer.



la fonction getAvailableSlots() reçoit les créneaux disponibles entre une date de départ et une date de fin, d'un doctorId, dans un cabinet, tout est calculé, formaté et envoyé par l'API:



Une fois connecté, le patient accède à son espace. Il y consulte ses rendez-vous, et gère son profil et son mot de passe. Le formulaire de changement de mot de passe affiche en direct les critères qui ne sont pas encore remplis (longueur, majuscule, chiffre, caractère spécial). Ce contrôle améliore l'expérience, mais il ne remplace pas la vérification de l'API, qui reste la seule à faire foi.

4.5.3. ToubibBO: BackOffice

ToubibBO est l'espace professionnel des médecins, également utilisé par les administrateurs. Il est servi en développement sur le port 5174, et sa connexion passe par la route dédiée `/api/auth/doctor/login`, qui n'accepte que les rôles `ROLE_DOCTOR` et `ROLE_ADMIN`. Contrairement à l'application patient, ToubibBO est surtout utilisé sur ordinateur, au cabinet : son interface exploite la largeur de l'écran, avec un menu de navigation latéral et des vues denses en informations. En plus du socle commun, il s'appuie sur deux bibliothèques spécialisées.

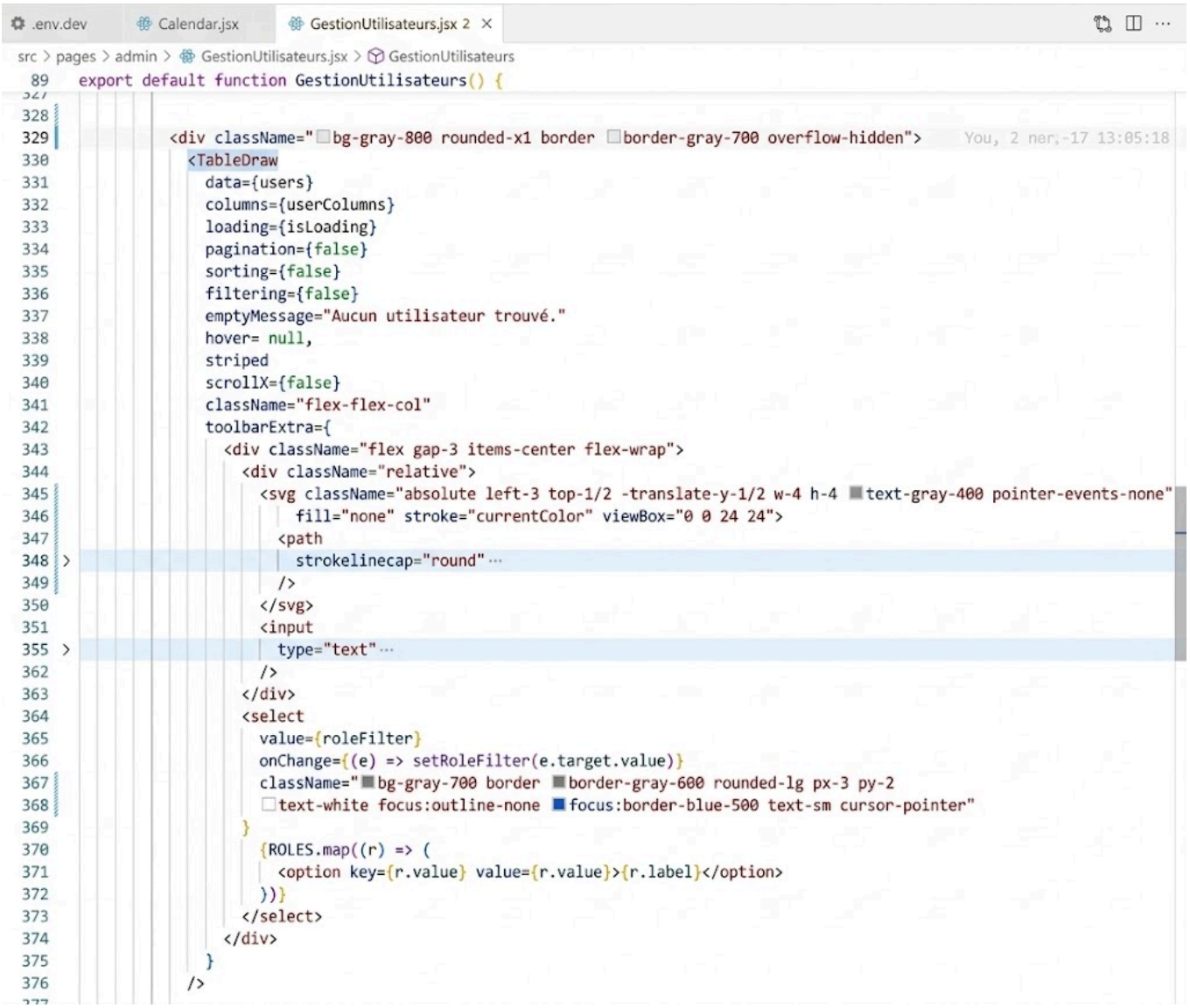
FullCalendar



L'agenda du médecin repose sur FullCalendar, une bibliothèque de calendrier interactif. Elle propose des vues jour, semaine, mois et liste, et permet de déplacer ou de redimensionner un rendez-vous directement à la souris. À chaque changement de période, l'application demande à l'API les rendez-vous correspondants (`GET /api/doctor/calendar`). Chaque modification par glisser-déposer est envoyée à l'API, qui vérifie que le médecin est bien propriétaire du rendez-vous avant de l'enregistrer. Lorsque l'option de partage est activée, les rendez-vous des confrères du cabinet apparaissent aussi dans l'agenda, chacun identifié par le médecin qui le porte. Le médecin peut enfin confirmer ou annuler un rendez-vous, et le nouveau statut s'affiche immédiatement, avec un code couleur propre à chaque statut.

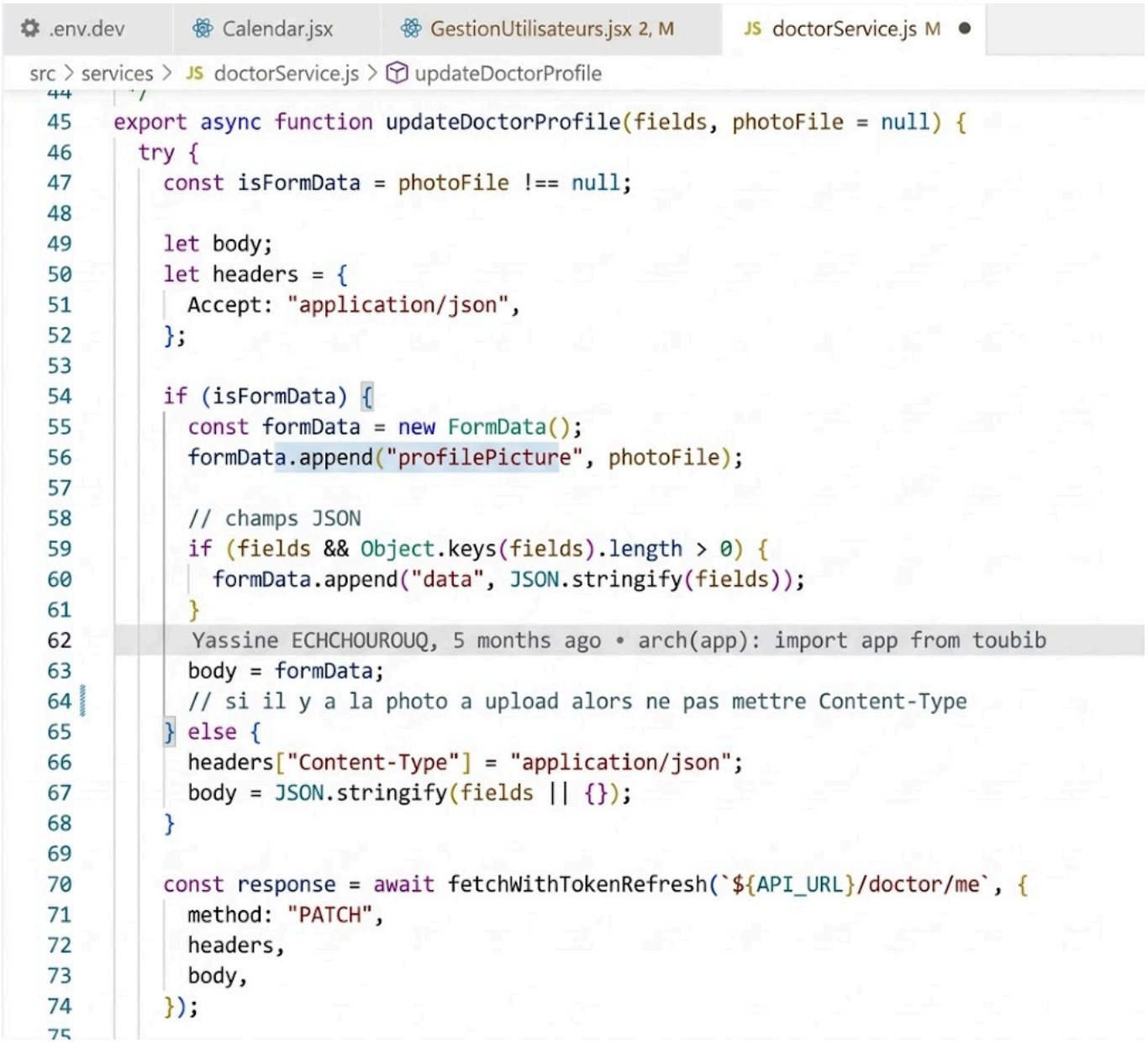
La gestion des patients avec TanStack Table

La liste des patients s'appuie sur TanStack Table, une bibliothèque de tableaux qui gère l'affichage, le tri et la pagination sans imposer de style, ce qui permet de la mettre en forme avec Tailwind. La pagination et la recherche sont réalisées côté serveur : le tableau ne charge qu'une page de patients à la fois, ce qui reste rapide même avec une patientèle importante. Depuis cette liste, le médecin accède à la fiche d'un patient, la modifie, et consulte ou enrichit son historique médical.



Le profil professionnel et les cabinets

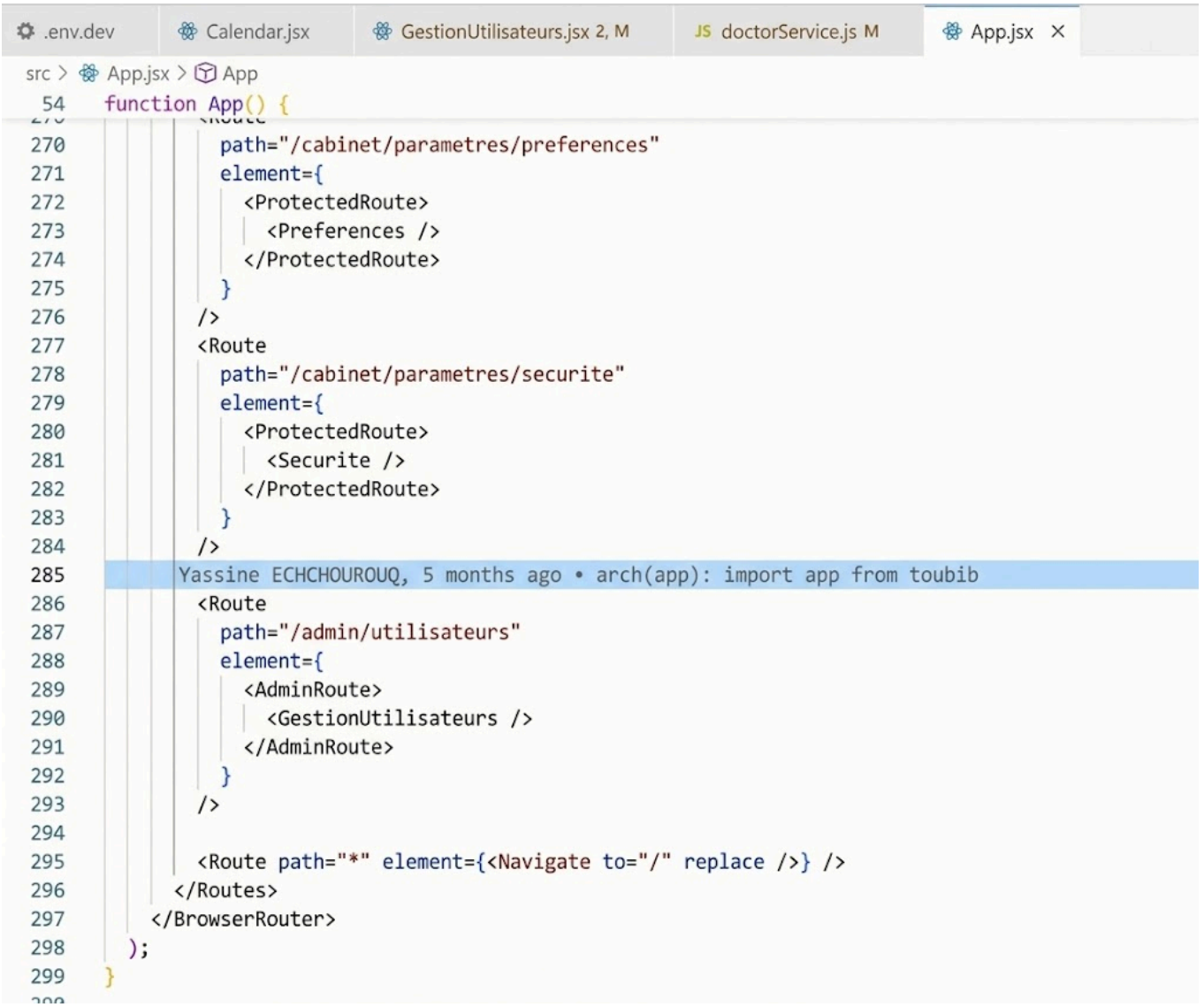
Le médecin gère son profil professionnel (biographie, spécialité, photo) et ses cabinets, chacun avec ses horaires, sa durée et son tarif de consultation. La mise à jour de la photo de profil est le seul endroit du projet où un fichier est envoyé. Le service `doctorService.js` adapte alors le format de la requête : il passe en `multipart/form-data` uniquement lorsqu'une nouvelle photo est choisie, et reste en JSON dans le cas contraire.



L'administration

Les administrateurs utilisent également `ToubibBO`. Les écrans d'administration, accessibles uniquement avec le rôle `ROLE_ADMIN`, présentent la liste des comptes et permettent de suspendre un utilisateur, de promouvoir un compte en médecin et d'imposer un changement de mot de passe.

Masquer ces écrans dans l'interface ne suffit pas à les protéger : c'est le pare-feu de l'API, configuré pour exiger `ROLE_ADMIN` sur tout le préfixe `/api/admin`, qui garantit qu'un médecin ne peut pas y accéder.

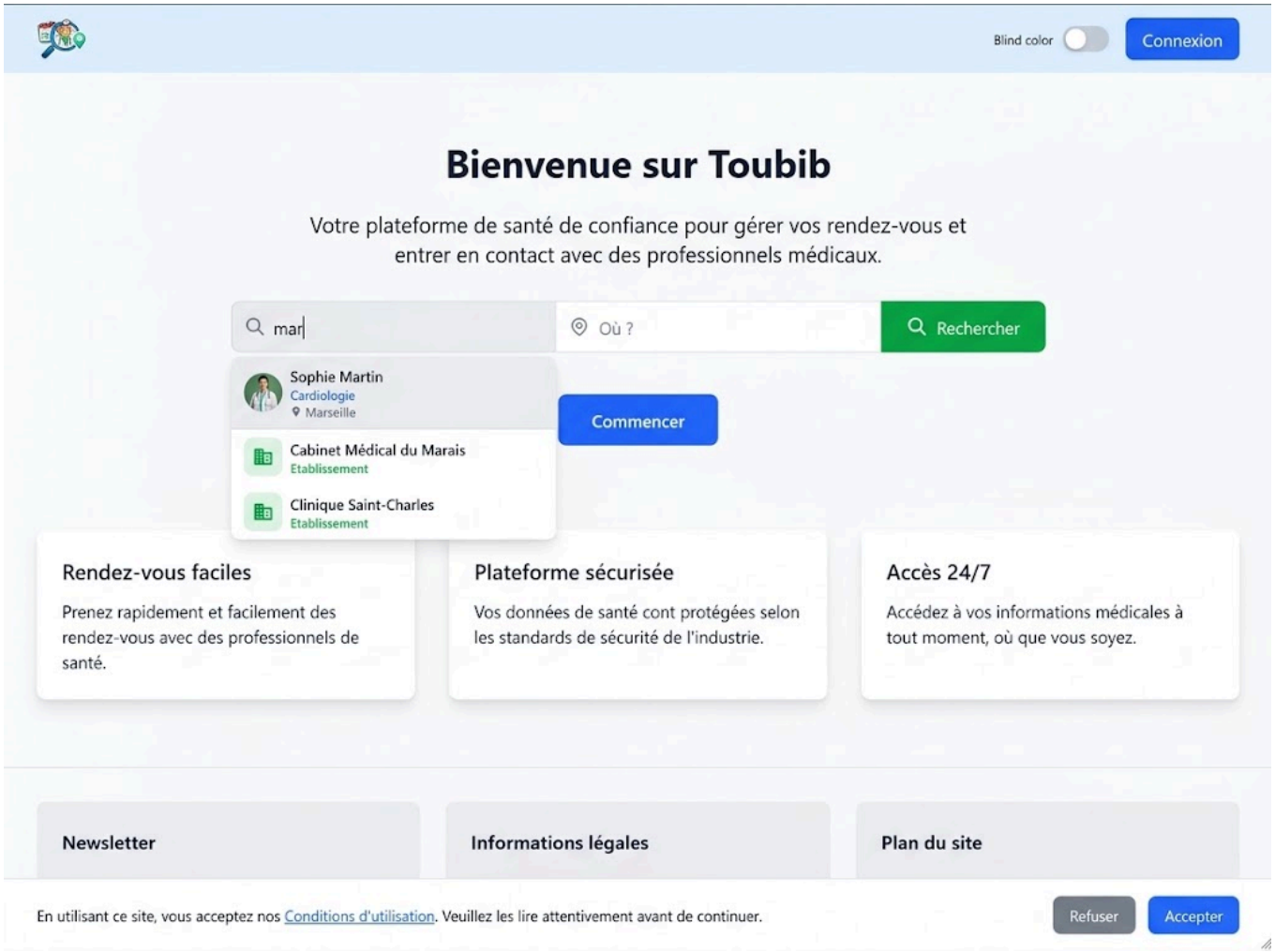


5. PRÉSENTATION DU JEU D'ESSAI

5.1. Réserver un Rendez-vous

Ce jeu d'essai vérifie le parcours principal de l'application **`Toubib`**, de la page d'accueil jusqu'à la confirmation d'un rendez-vous. Pour chaque étape et scénario, il présente l'écran concerné et ses fonctionnalités clés. Les tests ont été réalisés sur l'environnement de développement, avec des données fictives.

Page d'accueil:

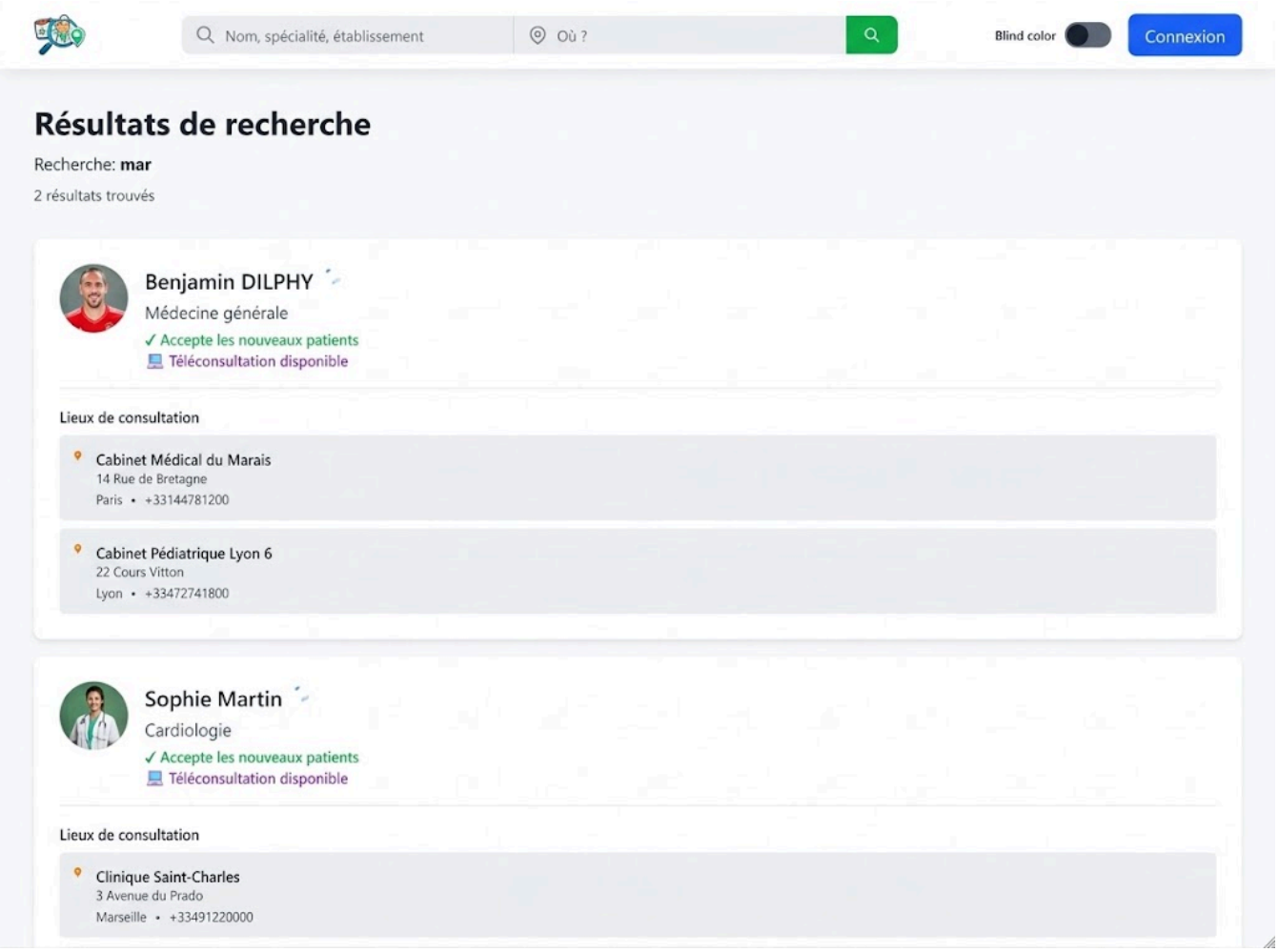


Fonctionnalité clé:

- Bandeau des conditions d'utilisation.
- Mode daltonien.
- Barre de recherche.
  - redirection directe vers la fiche du médecin choisi.
  - si le formulaire est validé *(submit),* alors une redirection vers la page de résultats avancée.
- Bouton de connexion.

Les préférences de l'utilisateur sur le mode visuel et les conditions de l'utilisation, sont stockées sur le navigateur en localStorage.

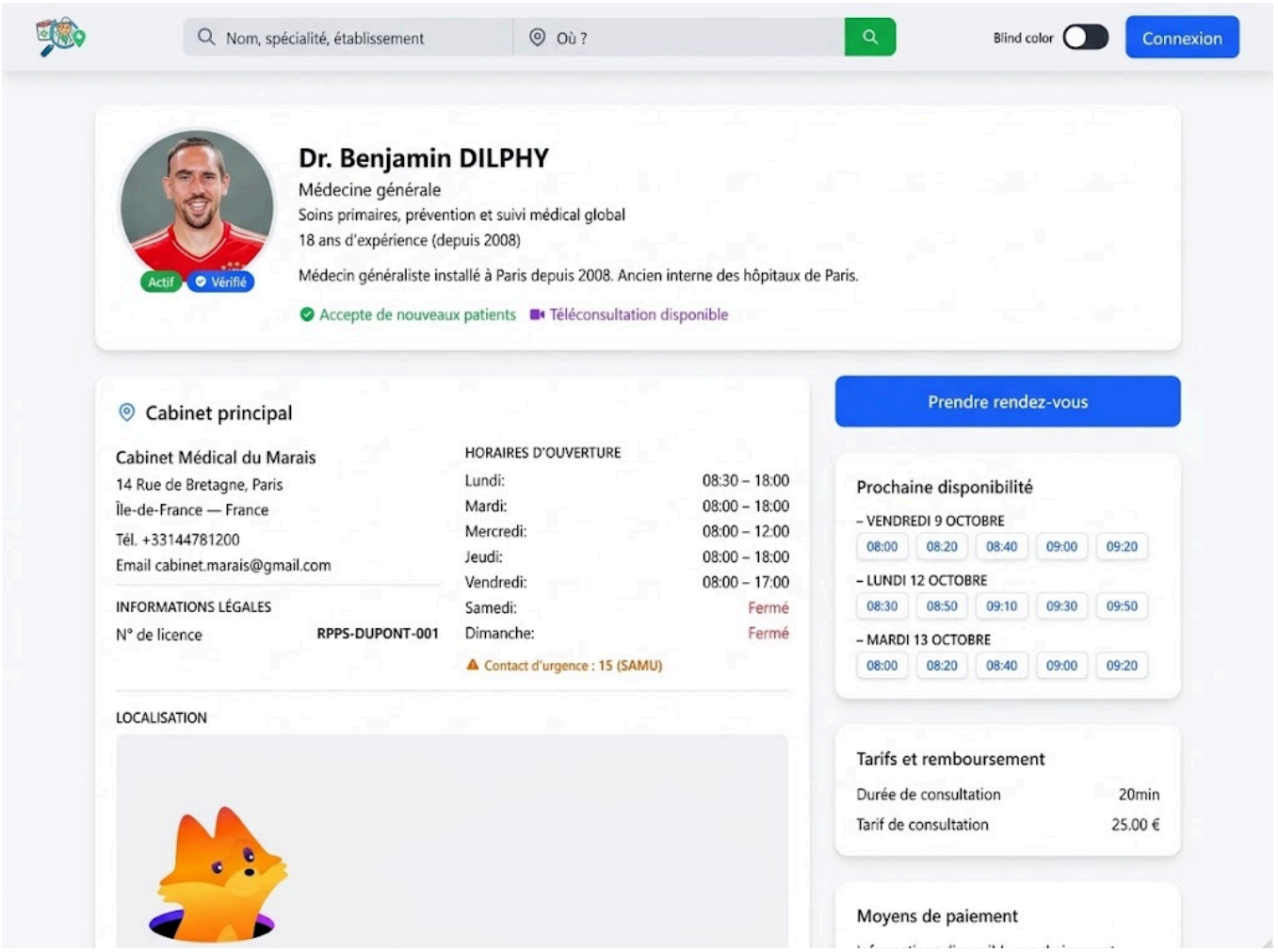
Accès aux résultats

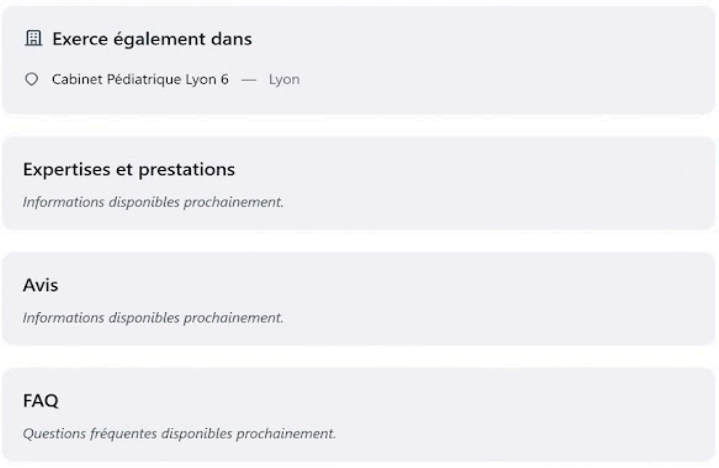


Fonctionnalité clé:

- La barre de recherche est toujours présente, cette fois dans Header et conserve cette position sur les autres pages, à condition qu'elle ne soit pas la page d'accueil.
- La liste des médecins, cabinets et spécialités est paginée.
- Choisir un résultat nous redirige vers la page dédiée.

Page médecin

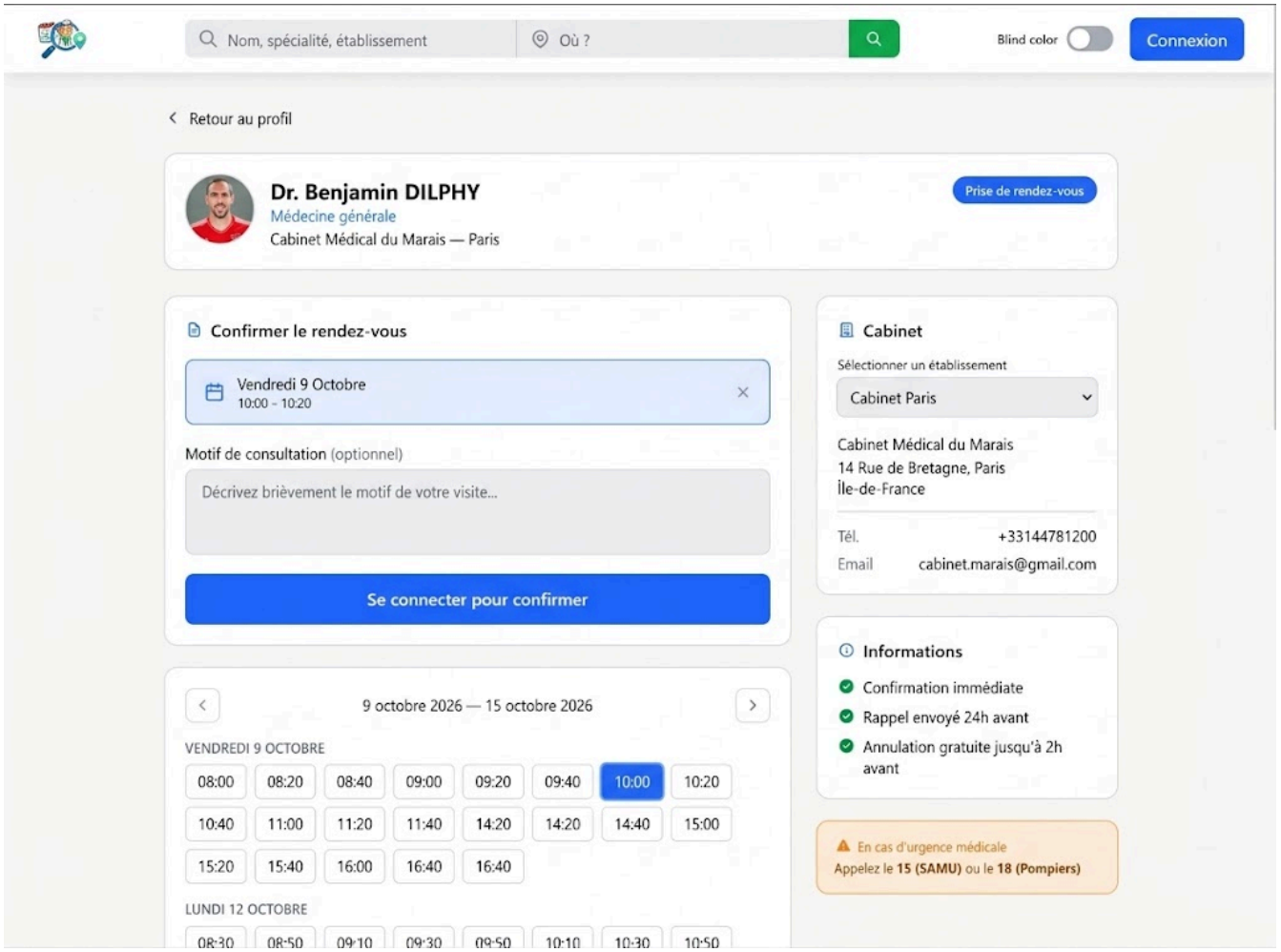




Fonctionnalité clé:

- Fiche complète du médecin.
- Les prochaines disponibilités.
- Bouton “Prendre rendez-vous”.

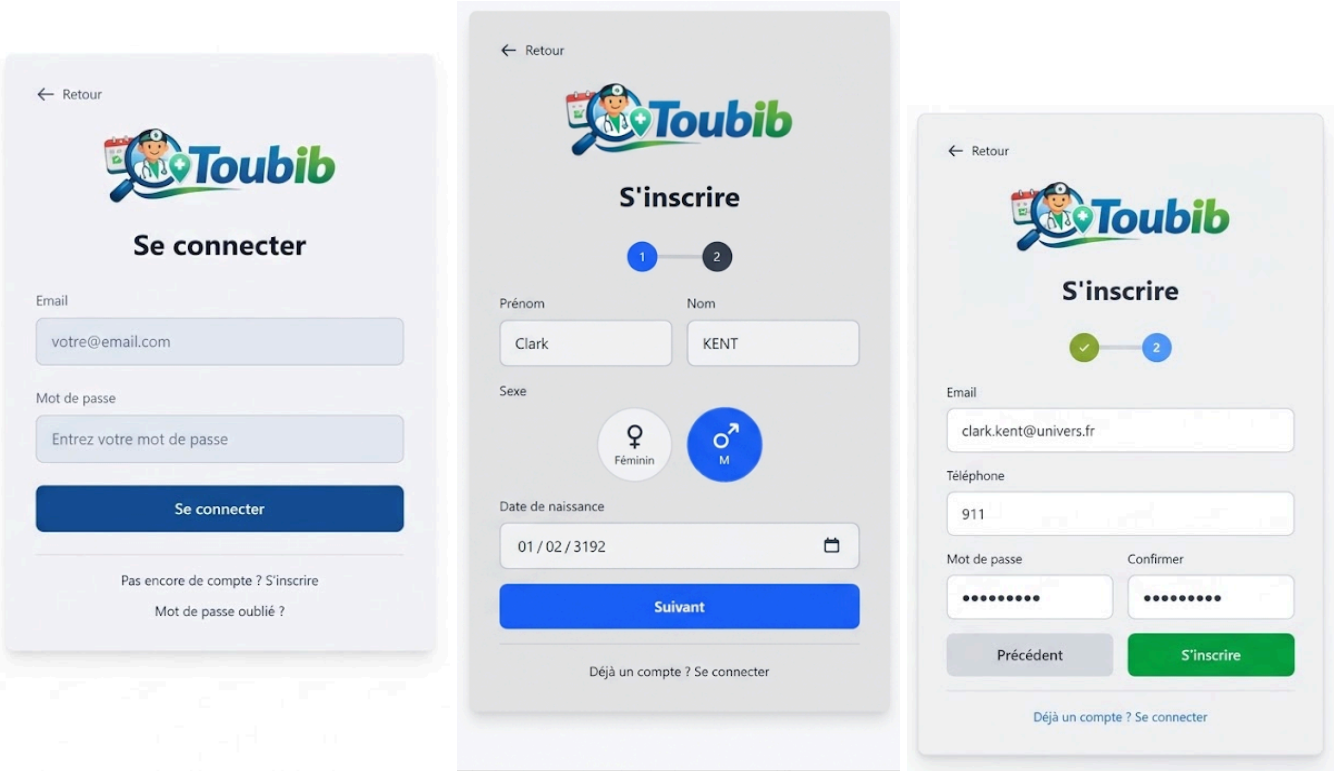
La prise du rendez-vous



Fonctionnalité clé:

- Les créneaux s'affichent selon la durée de consultation du médecin.
- Aucun créneau n'est proposé pendant la pause, si elle est activée.
- Le créneau occupé n'est pas proposé.
- Aucun créneau ne commence moins de trois heures après l'heure actuelle.
- Réservation sans connexion
- Le rendez-vous en cours est enregistré dans le localStorage, puis l'utilisateur est redirigé vers la page de connexion ou d'inscription. Si l'utilisateur est connecté, alors pas de stockage nécessaire

Connexion / Inscription



Fonctionnalité clé:

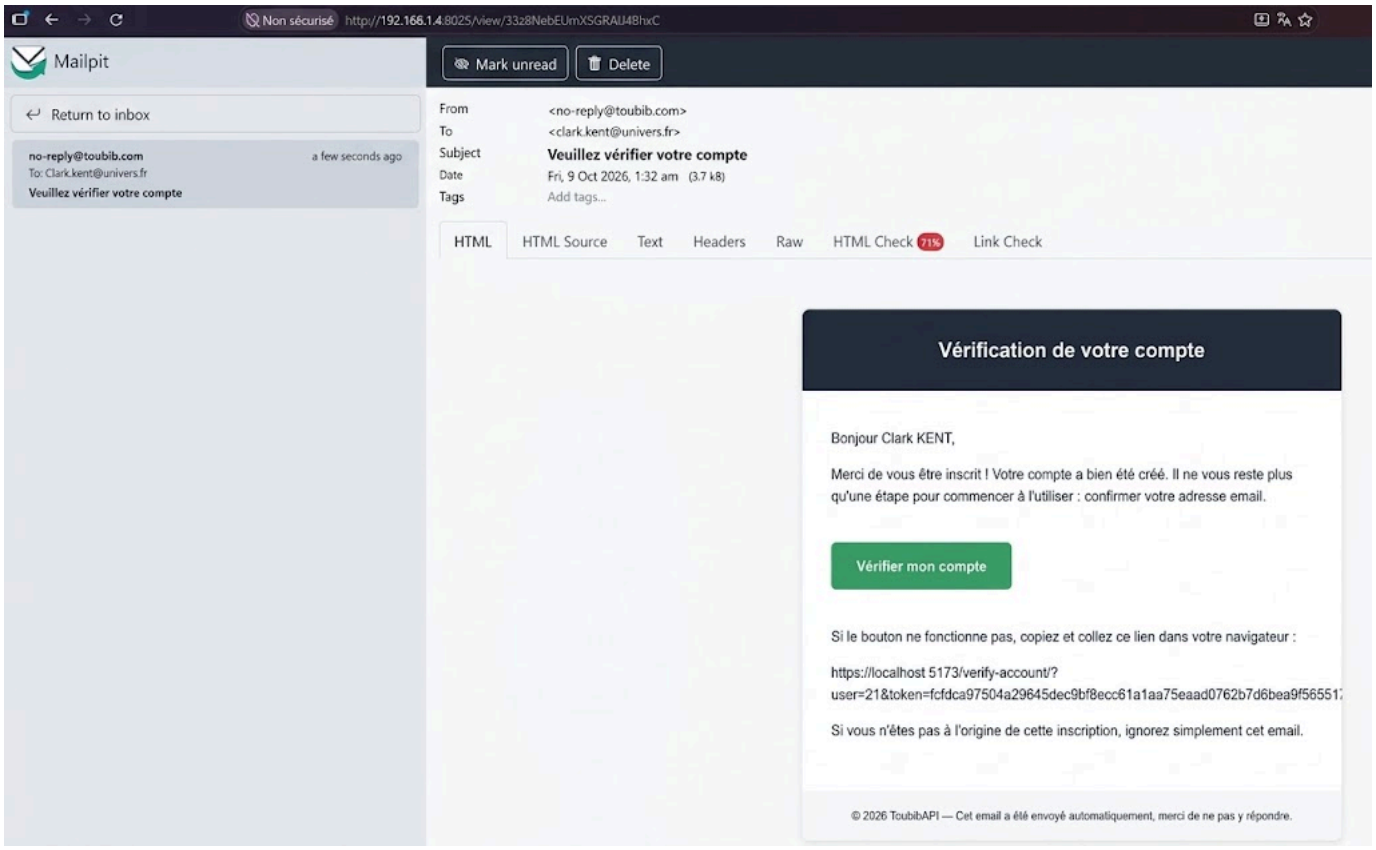
- Connexion réussie, redirection vers la page du rendez-vous en attente.
- Inscription
  - étape 1: informations personnelles (nom, prénom, sexe, date de naissance)
  - étape 2: informations de connexion (Email, téléphone et mot de passe)
- Après validation du formulaire d'inscription, un email de vérification est reçu à l'adresse saisie, avec un lien de vérification.

En environnement de développement, les courriels envoyés par l'application ne partent pas vers de vraies boîtes de réception : ils sont interceptés par Mailpit, un serveur de messagerie de test qui tourne en local dans un conteneur Docker, à partir de l'image `axllent/mailpit`. Mailpit expose deux ports.

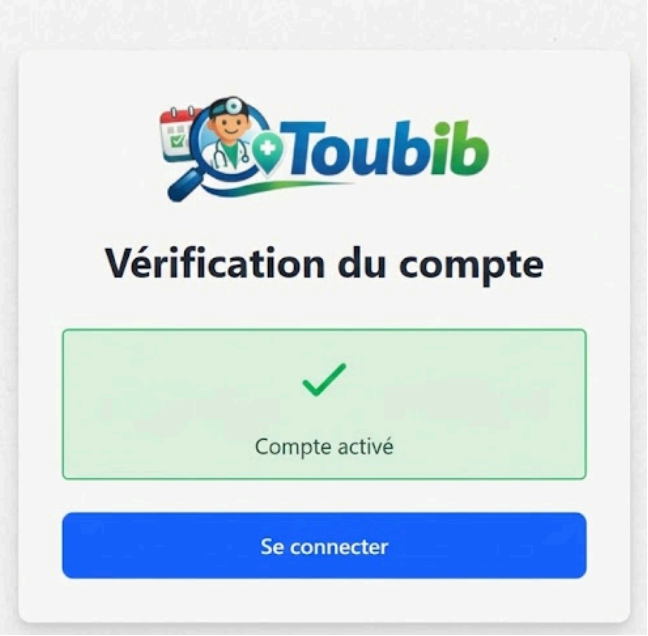
Le port `1025` reçoit les courriels envoyés par l'API via le protocole SMTP, et le port `8025` donne accès à une interface web dans laquelle je peux consulter chaque message reçu : son destinataire, son contenu et ses liens. Côté Symfony, il suffit de faire pointer la variable `MAILER_DSN` vers ce serveur pour que le composant Mailer lui envoie tous les courriels.

Cette solution me permet de tester l'ensemble des parcours qui reposent sur un courriel, comme la vérification de compte ou la réinitialisation du mot de passe, sans risquer d'envoyer un message à

une personne réelle et sans dépendre d'un fournisseur de messagerie externe. En production, il suffit de modifier `MAILER_DSN` pour utiliser un véritable service d'envoi, sans changer le code de l'application.



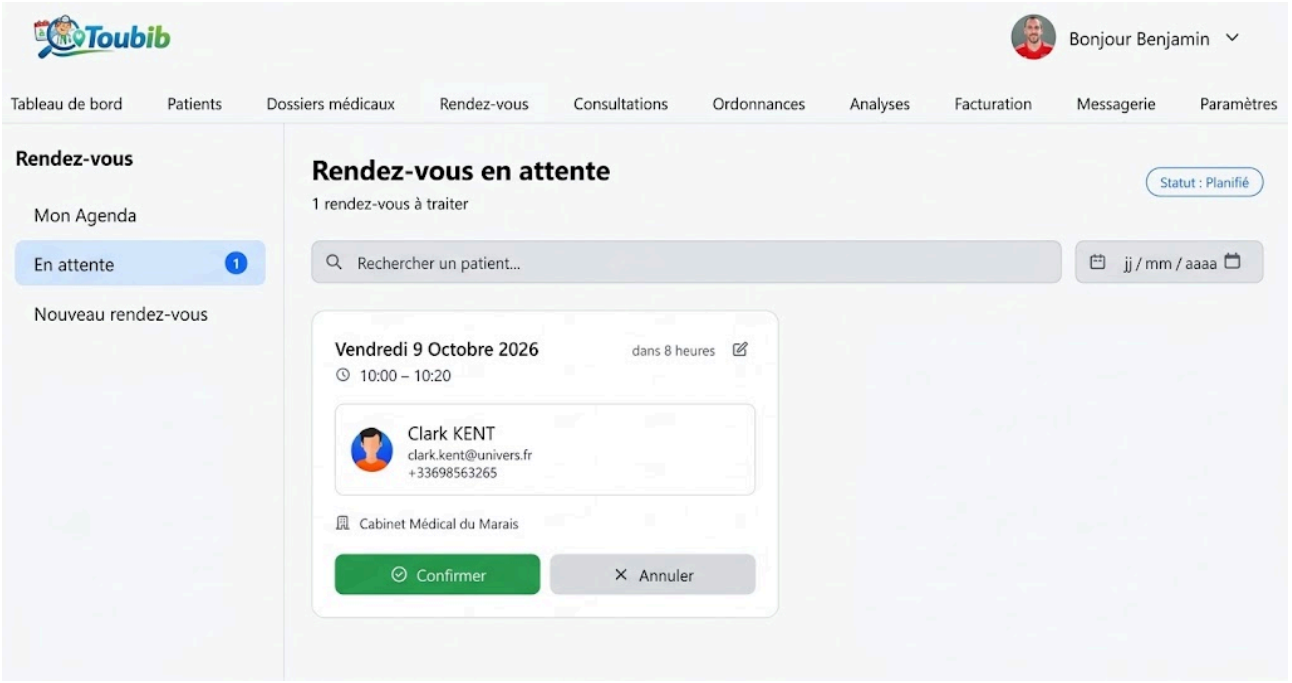
Après avoir reçu le mail, et cliquer sur le lien, nous arrivons sur la page de vérification du token envoyé.



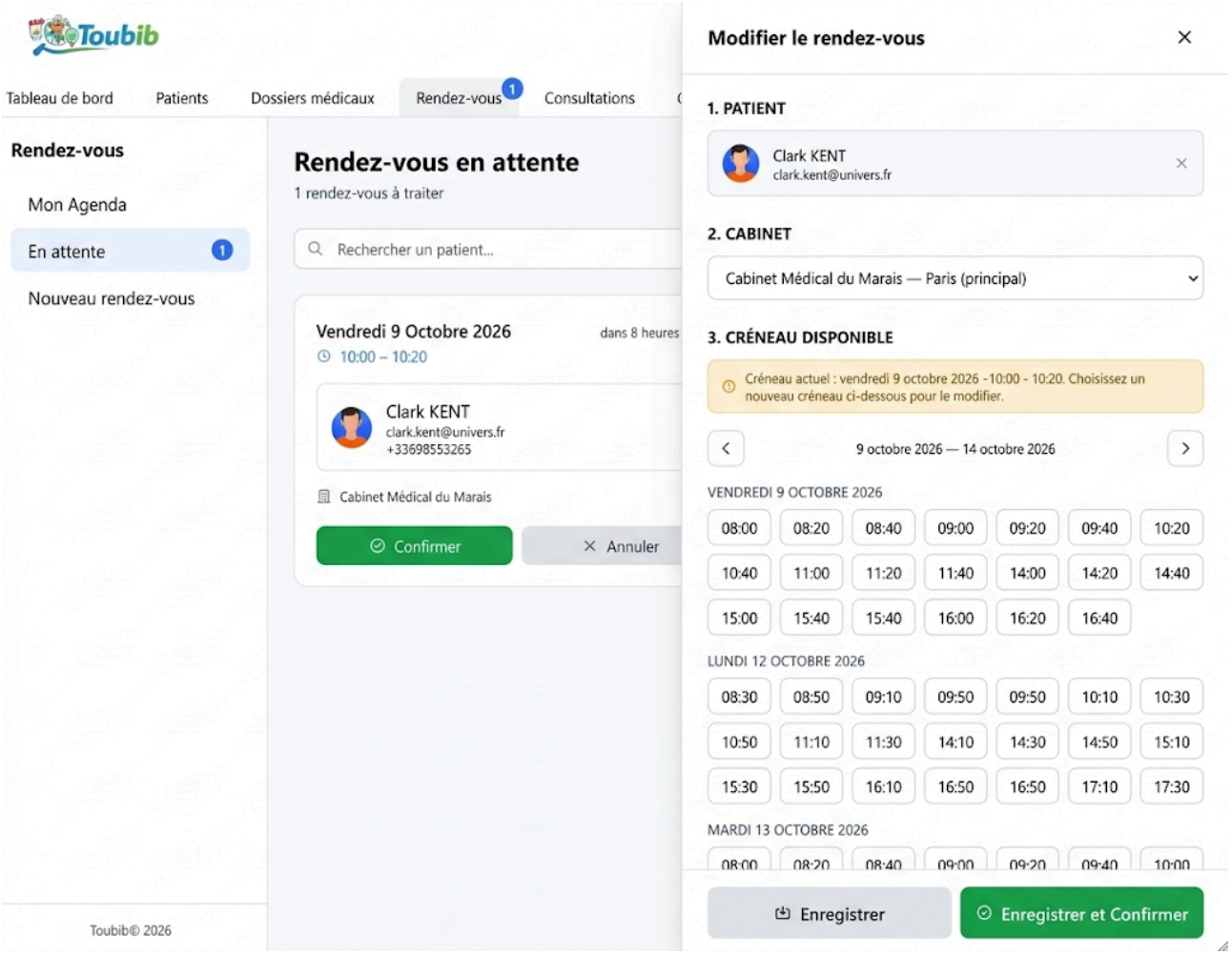
Après une connexion réussie, et avoir confirmé le rendez-vous, le médecin doit le confirmer depuis le back office ToubibBO, mais le rendez-vous s’affichera sur la page “Mes rendez-vous” du patient.

ToubibBO: Gestion des rendez-vous

Liste des rendez-vous “En attente”:

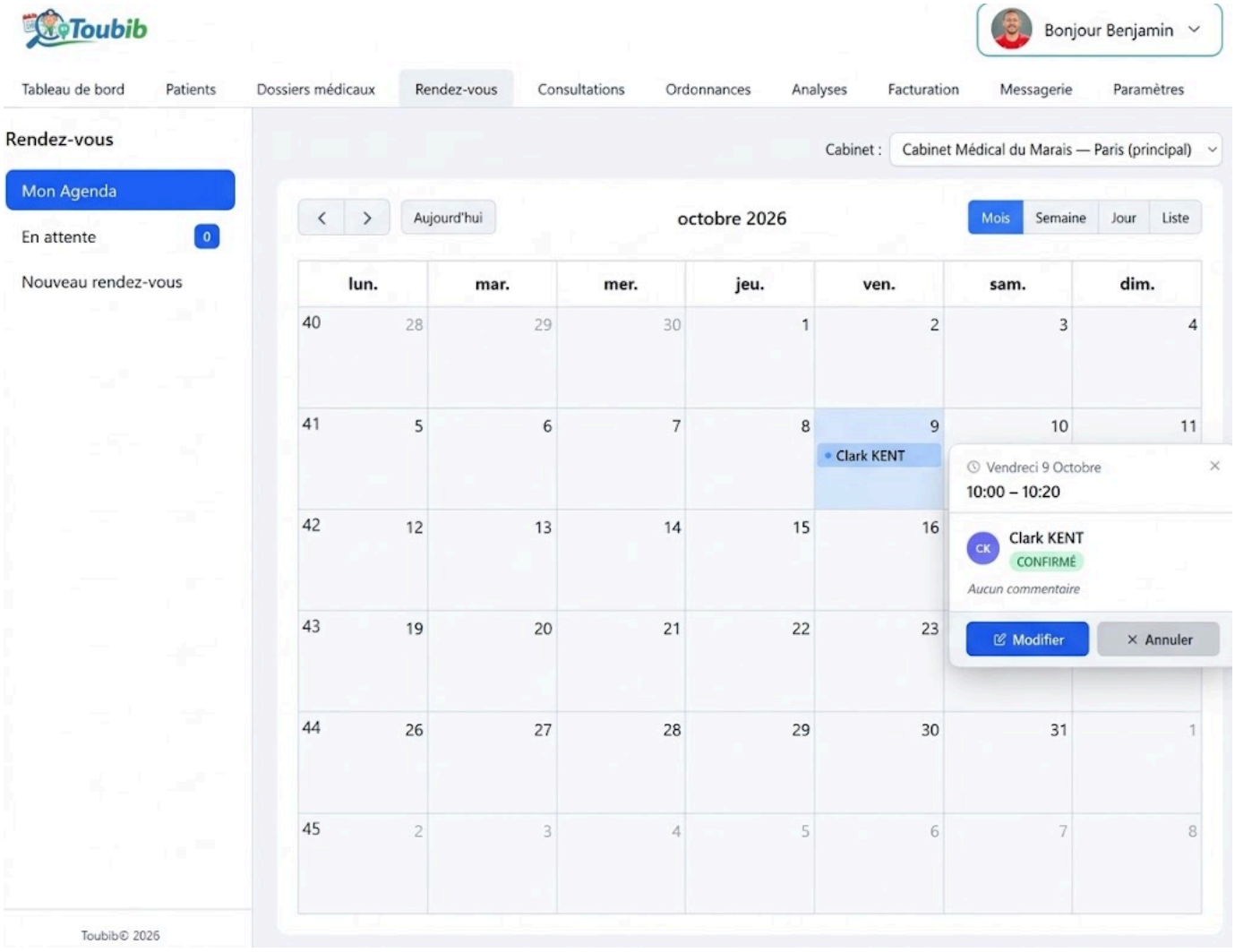


Modifier un rendez-vous avant confirmation:



Fonctionnalité clé:

- Le nouveau rendez-vous apparaît au statut « planifié ».
- Liste des autres rendez-vous à confirmer.
- Le créneau annulé est de nouveau proposé aux patients.
- Confirmer, Annuler, Modifier avant confirmation d'un rendez-vous
- Cliquer sur confirmer: Le statut passe à « confirmé » et s'affiche immédiatement, sans rechargement de la page.
- Cliquer sur annuler: Le statut passe à « annulé ».
- Le patient reçoit le statut du rendez-vous et il reste affiché même si il est annulé.
- Le rendez-vous confirmé est affiché sur l'Agenda:



5.2. Responsivité

Une grande partie des patients consulte un site de prise de rendez-vous depuis un smartphone : pour trouver rapidement un médecin, souvent en déplacement, ou pour vérifier l'horaire d'un rendez-vous. J'ai donc conçu l'application front `Toubib` en responsive design : son interface s'adapte automatiquement à la taille de l'écran, du petit smartphone à l'écran d'ordinateur, sans version séparée pour le mobile.

J'ai appliqué l'approche mobile first. Chaque écran est d'abord conçu pour un petit écran, avec uniquement l'essentiel du contenu, puis enrichi progressivement pour les écrans plus larges. Cette démarche présente deux avantages. Elle oblige à hiérarchiser l'information : sur un petit écran, il faut choisir ce qui compte vraiment, et cette sobriété profite ensuite à toutes les tailles d'écran. Elle allège aussi le code, car les styles de base s'appliquent à tous les écrans et seules les adaptations pour les écrans plus larges sont ajoutées.

Le premier prérequis est la balise `viewport`, présente dans le fichier `index.html`. Elle indique au navigateur mobile d'utiliser la largeur réelle de l'appareil, au lieu de simuler un écran d'ordinateur et de réduire la page :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

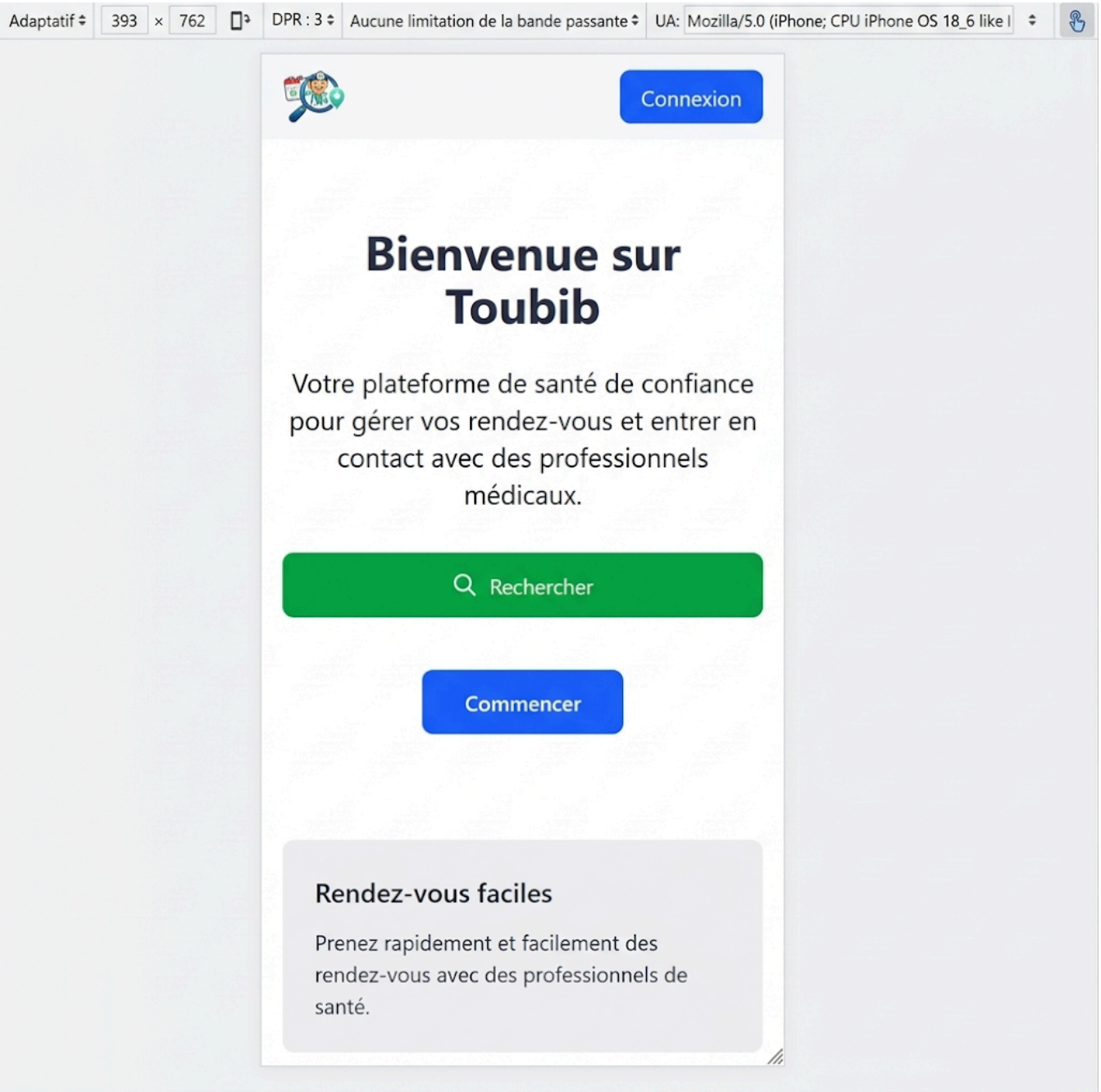
Tailwind CSS est lui-même construit selon l'approche *mobile first*. Une classe sans préfixe s'applique à tous les écrans, à partir du plus petit. Une classe préfixée par un point de rupture (*breakpoint*) ne s'applique qu'à partir d'une certaine largeur. Tailwind 4 définit par défaut les points de rupture suivants :

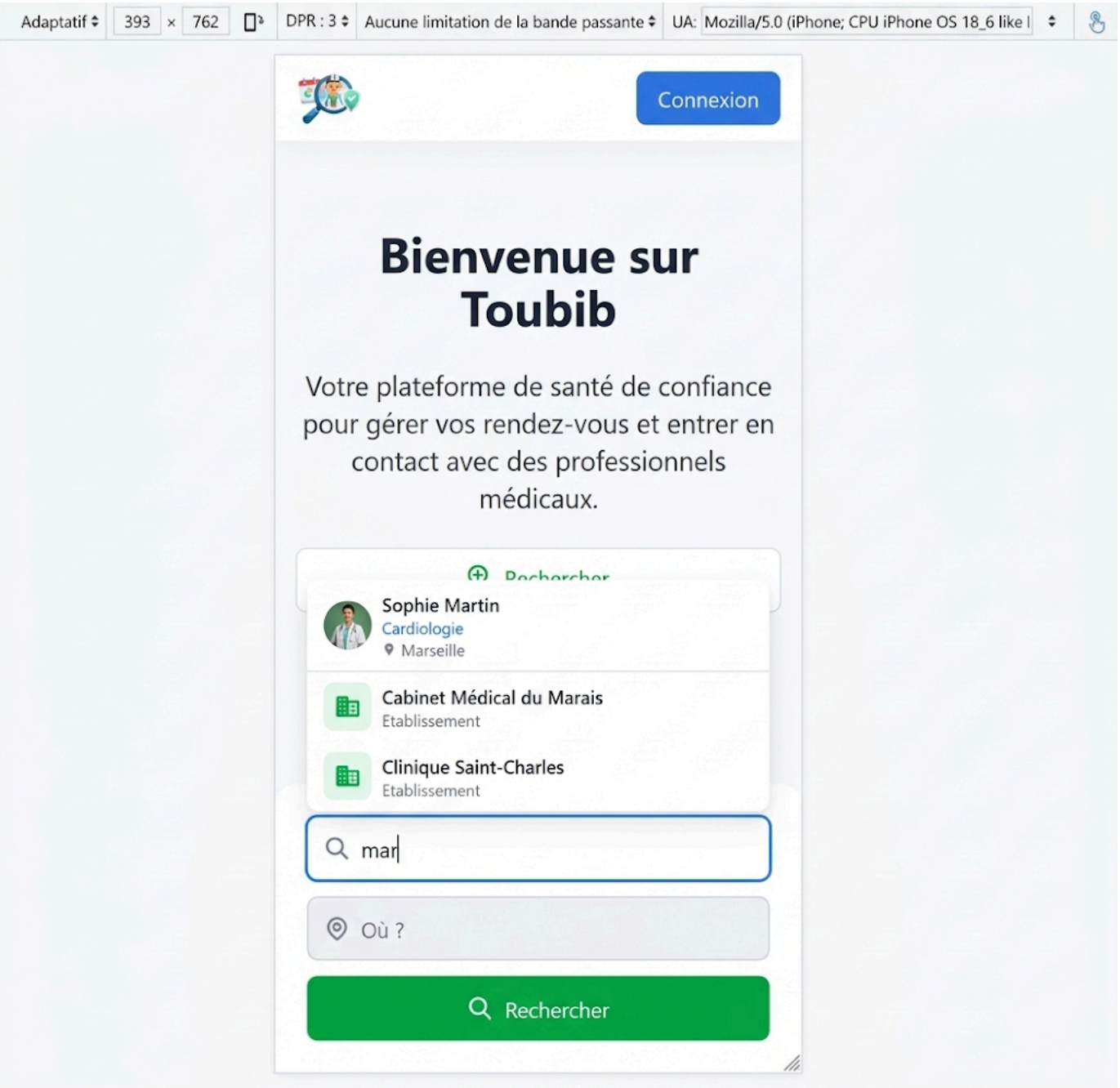
(aucun)	0 px	Smartphones, en mode portrait
`sm:`	640 px	Grands smartphones, en mode paysage
`md:`	768 px	Tablettes
`lg:`	1024 px	Petits ordinateurs portables
`xl:`	1280 px	Écrans d'ordinateur

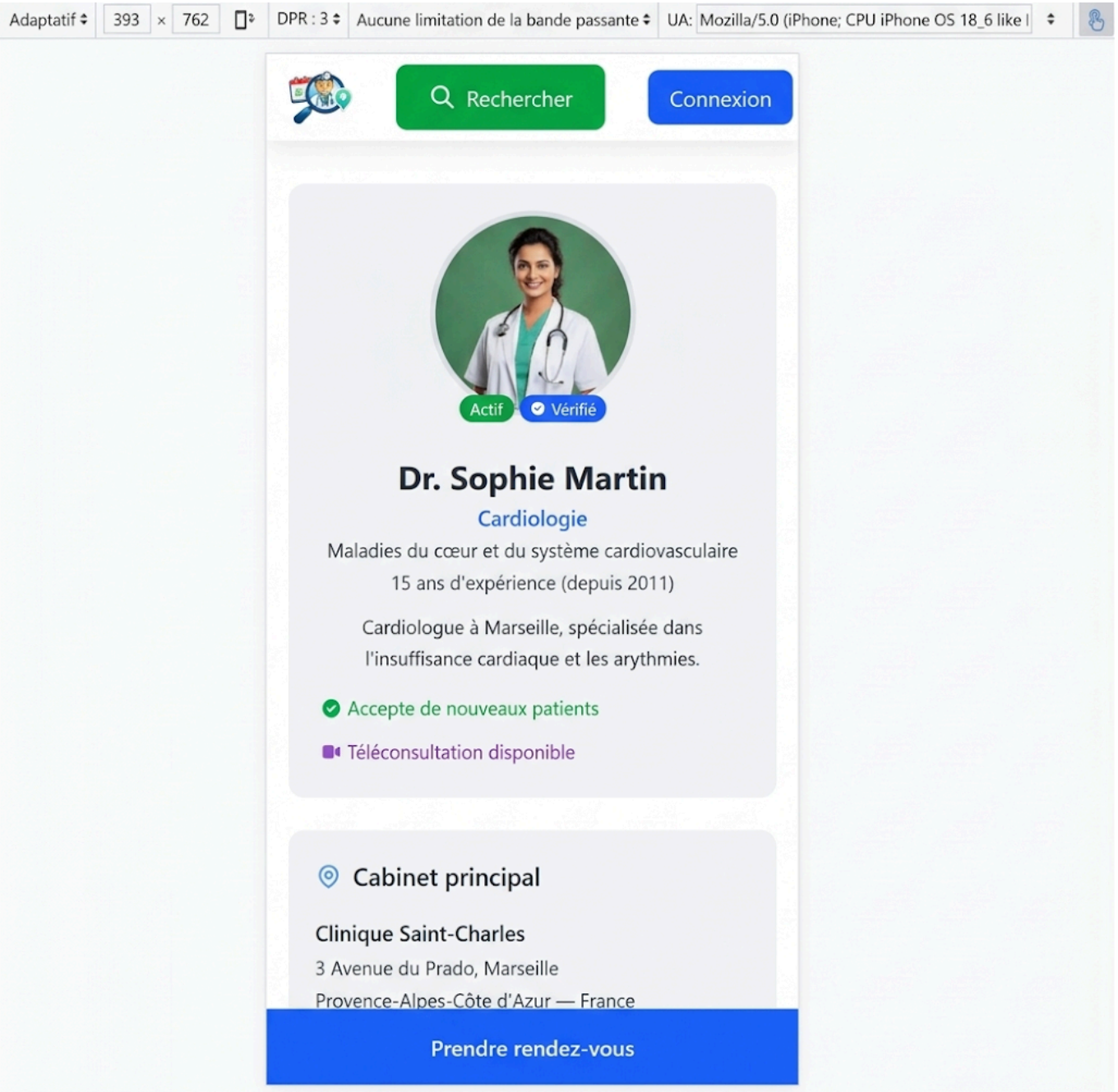
Par exemple, une liste de médecins peut s'afficher sur une colonne sur mobile, puis sur deux colonnes sur tablette et trois sur ordinateur, avec une seule ligne de classes :

```
<div className="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3">
```

La lecture se fait de gauche à droite : une colonne par défaut, deux à partir de 768 px, trois à partir de 1024 px. Les mises en page reposent sur Flexbox et CSS Grid, deux modèles de disposition conçus pour s'adapter à l'espace disponible. Les tailles sont exprimées de préférence en unités relatives plutôt qu'en pixels fixes, pour éviter les débordements horizontaux.


Adaptatif
393
×
762
DPR : 3
Aucune limitation de la bande passante
UA: Mozilla/5.0 (iPhone; CPU iPhone OS 18_6 like I
Connexion
Bienvenue sur Toubib
Votre plateforme de santé de confiance pour gérer vos rendez-vous et entrer en contact avec des professionnels médicaux.
Rechercher
Commencer
Rendez-vous faciles
Prenez rapidement et facilement des rendez-vous avec des professionnels de santé.


Adaptatif
393
762
DPR : 3
Aucune limitation de la bande passante
UA: Mozilla/5.0 (iPhone; CPU iPhone OS 18_6 like
Connexion
Bienvenue sur Toubib
Votre plateforme de santé de confiance pour gérer vos rendez-vous et entrer en contact avec des professionnels médicaux.
Sophie Martin
Cardiologie
Marseille
Cabinet Médical du Marais
Etablissement
Clinique Saint-Charles
Etablissement
mar
Où ?
Rechercher



À l'inverse, je n'ai pas rendu l'application `ToubibBO` *responsive*. Ce choix est volontaire et découle de l'analyse de son usage.

`ToubibBO` est un outil de travail professionnel. Le médecin l'utilise au cabinet, sur un ordinateur, pendant ses journées de consultation, pour gérer son agenda, suivre ses patients et administrer ses cabinets. L'administrateur l'utilise aussi dans un cadre bureautique. Les écrans de `ToubibBO` sont par nature denses en informations, et ils exploitent pleinement la largeur d'un écran d'ordinateur :

- L'agenda en vue semaine, réalisé avec FullCalendar, affiche sept colonnes de jours, chacune découpée en créneaux horaires. Réduit à la largeur d'un smartphone, il deviendrait illisible.
- Les tableaux de patients, réalisés avec TanStack Table, présentent de nombreuses colonnes côte à côte (nom, téléphone, date de naissance, dernier rendez-vous), que le médecin doit pouvoir comparer d'un coup d'œil.

5.3. Accessibilité

L'accessibilité numérique consiste à concevoir une application utilisable par tous, y compris par les personnes en situation de handicap visuel, auditif, moteur ou cognitif. Pour une plateforme de santé comme Toubib, cette exigence est particulièrement importante : les patients qui ont besoin de prendre rendez-vous chez un médecin sont, plus souvent que la moyenne, des personnes âgées, malvoyantes ou en difficulté avec les outils numériques. Pour guider mes choix, je me suis appuyé sur le RGAA (Référentiel général d'amélioration de l'accessibilité), qui sert de base aux obligations légales en France. Il repose sur quatre principes : le contenu doit être perceptible, utilisable, compréhensible et compatible avec les technologies d'assistance, comme les lecteurs d'écran.

Une attention particulière a été portée au daltonisme, les formes les plus courantes sont la deutéranopie et la protanopie, elles rendent difficile la distinction entre le rouge et le vert, tandis que la tritanopie, plus rare, touche la distinction entre le bleu et le jaune. Ce trouble pose problème dès qu'une information repose uniquement sur la couleur : un rendez-vous confirmé affiché en vert et un rendez-vous annulé affiché en rouge peuvent paraître presque identiques à une personne daltonienne. J'ai donc ajouté à Toubib un mode daltonien, activable par un bouton accessible depuis toutes les pages. Lorsqu'il est activé, la palette de couleurs de l'interface est remplacée par une palette qui évite les associations rouge/vert, au profit de contrastes fondés sur des teintes et des luminosités bien différentes. Le choix de l'utilisateur est enregistré dans le `localStorage` du navigateur : il est conservé d'une page à l'autre et lors des visites suivantes. Ce mode reste cependant une aide supplémentaire : la couleur n'est jamais le seul moyen de transmettre une information. Le statut d'un rendez-vous est toujours accompagné de son libellé, et une erreur de saisie est signalée par un message texte en plus du changement de couleur du champ.

6. VULNÉRABILITÉS DE SÉCURITÉ ET VEILLE

6.1. Veille technologique

Les technologies du web évoluent très vite : de nouvelles versions apportent des fonctionnalités, d'autres cessent d'être maintenues, et un projet qui n'est pas suivi devient rapidement obsolète. J'ai donc mis en place une veille régulière sur les trois technologies principales de Toubib, à partir de sources officielles : les blogs et les pages de version publiés par les équipes de PHP, de Symfony et de React.

6.1.1. PHP

L’application utilise aujourd’hui PHP 8.4. Ma veille porte d'abord sur son cycle de vie. D'après le calendrier officiel de php.net, PHP 8.4 bénéficie d'un support actif (corrections de bogues et de sécurité) jusqu'au 31 décembre 2026, puis d'un support limité aux correctifs de sécurité jusqu'au 31 décembre 2028. Le projet repose donc sur une version maintenue, mais une montée de version devra être planifiée avant la fin de 2028.

PHP 8.5 est sorti le 20 novembre 2025, il apporte plusieurs évolutions qui peuvent être intéressantes pour **Toubib**.

6.1.2. Symfony

L'API repose sur Symfony 8, et Symfony 8.0 est sorti le 27 novembre 2025 et exige PHP 8.4 au minimum, ce qui explique la version de PHP retenue pour le projet.

Cette veille a fait ressortir un point important pour le projet. Les versions mineures de Symfony qui ne sont pas des versions LTS (*Long Term Support*) ne sont maintenues que pendant une durée courte. D'après le blog officiel de Symfony, la branche 8.0 a atteint sa fin de maintenance fin juillet 2026, avec une dernière version 8.0.16 publiée le 29 juillet 2026 : elle ne reçoit plus aucun correctif, y compris de sécurité, ce qui nécessite une migration vers la version 8.1, puis le suivi des versions suivantes, Symfony 8.2 étant annoncée pour novembre 2026. Cette observation m'a fait comprendre qu'utiliser la dernière version d'un framework implique aussi de suivre son rythme de mises à jour.

6.1.3. React

Cette veille a aussi orienté mes objectifs d'apprentissage. React n'était que le premier pas, aujourd'hui je compte me former à Next.js, un framework construit sur React. Là où ce dernier laisse au développeur le choix de l'organisation, Next.js impose un cadre conventionnel : la structure des dossiers détermine directement les routes de l'application, ce qui rend un projet immédiatement lisible pour tout développeur qui connaît le framework. Il intègre nativement TypeScript, proposé par défaut à la création d'un projet.

Le typage des données permet de détecter de nombreuses erreurs dès l'écriture du code plutôt qu'à l'exécution, par exemple une propriété mal nommée dans la réponse de l'API, et documente la forme des données échangées entre les composants. Next.js apporte également le rendu côté serveur et la génération de pages statiques, qui améliorent le temps d'affichage et le référencement naturel, mais ce ne sera peut être pas pour ce projet.

Enfin, Next.js offre une optimisation automatique des images et un découpage du code par page. Cet apprentissage m'aidera à adopter un framework plus complet.

6.2. Veille de sécurité

En complément de la veille technologique, je surveille les failles de sécurité publiées pour les dépendances du projet. Les bases de référence sont la GitHub Advisory Database, qui recense les avis de sécurité des paquets Composer et npm, et la base CVE (*Common Vulnerabilities and Exposures*), qui attribue un identifiant unique à chaque vulnérabilité connue. Pour vérifier automatiquement les dépendances installées, Composer et npm proposent les commandes `composer audit` et `npm audit`, qui comparent les versions utilisées avec ces bases et signalent les paquets vulnérables.

J'ai prévu d'intégrer ces contrôles de sécurité plus tard dans le pipeline de déploiement continus (CI/CD), avec un outil d'analyse comme SonarQube : à chaque modification, le code est analysé automatiquement pour détecter les failles de sécurité et les mauvaises pratiques avant d'être fusionné.

6.2.1. Symfony/LexikJWT

L'authentification de **Toubib** repose sur le bundle LexikJWTAuthenticationBundle. En échangeant avec mes collègues de travail, j'ai découvert qu'une faille d'énumération d'utilisateurs avait touché d'anciennes versions du bundle : il renvoyait des messages d'erreur différents selon que le compte existait ou non. Un attaquant pouvait ainsi tester une liste d'adresses électroniques et savoir lesquelles correspondaient à un compte réel, avant de cibler ces comptes. La faille, de gravité faible, a été corrigée dans les versions 2.10.7 et 2.11.3, qui renvoient désormais un message générique.

Cette faille m'a directement servi dans la conception de Toubib. Mon point d'entrée de connexion renvoie le même message, « Identifiants invalides », avec le même code 401, que l'adresse soit inconnue ou que le mot de passe soit faux. Plus largement, cette veille a confirmé plusieurs bonnes pratiques que j'ai appliquées à l'usage des JWT : une durée de vie courte du jeton d'accès (une heure), un jeton de rafraîchissement révocable enregistré en base, une signature par clé RSA dont la *passphrase* reste hors du dépôt Git, et des cookies `HttpOnly` et `Secure`, inaccessibles au JavaScript.

6.2.2. Symfony/Doctrine

Doctrine protège contre les injections SQL grâce aux requêtes préparées, mais une bibliothèque n'est jamais infaillible : des failles sont régulièrement découvertes, y compris dans des outils réputés sûrs. Il faut donc maintenir ses dépendances à jour et vérifier régulièrement les avis de sécurité publiés, ce que l'analyse automatique dans le pipeline permet de faire. Il faut aussi contrôler toutes les données reçues, y compris celles qui paraissent anodines : dans Toubib, les paramètres de pagination de la recherche et des listes de patients doivent toujours être convertis en entiers et bornés avant d'être utilisés dans une requête.

6.2.3. FTP

Les photos de profil des médecins sont stockées sur un serveur de fichiers accessible par FTP. Ma veille sur ce protocole a mis en évidence une faiblesse importante. FTP est un protocole ancien qui ne chiffre rien : l'identifiant, le mot de passe et le contenu des fichiers circulent en clair sur le réseau. Un attaquant placé sur le chemin des échanges peut les intercepter avec un simple analyseur de trafic, et ainsi récupérer les identifiants du serveur de fichiers.

Il existe deux alternatives sécurisées :

- FTPS, qui ajoute une couche de chiffrement TLS au protocole FTP.
- SFTP, qui est un protocole distinct reposant sur SSH. Il chiffre l'ensemble des échanges dans une seule connexion, sur le port 22, et permet une authentification par clé publique.

La migration vers SFTP fait partie des améliorations prioritaires, afin que les échanges restent protégés même si le serveur de fichiers est déplacé ou exposé. Les fichiers reçus restent par ailleurs contrôlés par le `FileUploadHelper` avant tout envoi : taille limitée à 2 Mo, formats JPEG, PNG et WebP uniquement, et vérification que le contenu est réellement une image.

6.3. Processus de recherche

La documentation de référence de mes technologies est majoritairement rédigée en anglais : la documentation officielle de Symfony, de React et de Doctrine, les blogs des équipes de développement, les avis de sécurité de GitHub ou la base CVE. Je maîtrise l'anglais technique, ce qui me permet de consulter ces sources directement dans leur version originale, toujours la plus complète et la plus à jour, sans dépendre d'une traduction.

Ma démarche de recherche suit généralement les mêmes étapes. Je commence par formuler le problème avec des mots-clés précis en anglais, en reprenant les termes techniques exacts : le nom du composant, le message d'erreur, le numéro de version. Je privilégie ensuite les sources officielles, comme la documentation, le dépôt GitHub du projet ou son blog, parce qu'elles font référence et correspondent aux versions récentes. Les forums comme Stack Overflow et les issues GitHub viennent en complément, pour des cas particuliers ou des erreurs peu documentées.

Je vérifie alors systématiquement la date des réponses et la version concernée, car une solution valable pour Symfony 5 ne l'est pas forcément pour Symfony 8. Je croise plusieurs sources avant de retenir une solution, puis je la teste dans mon environnement de développement avant de l'intégrer au projet.

Cette démarche s'est révélée utile tout au long du développement de Toubib, par exemple, comme je l'ai mentionné plus haut, mes collègues m'ont signalé une faille de sécurité au niveau de l’authentification, puis j'ai consulté l'avis GHSA-2frx-j9hj-6c65, publié pour LexikJWTAuthenticationBundle, que j'ai compris qu'un message d'erreur différent selon que le compte existe ou non suffit à révéler quels utilisateurs sont inscrits. Cette lecture m'a conduit directement au choix d'un message unique, « Identifiants invalides », pour la connexion de Toubib.

CONCLUSION

Toubib a été pour moi bien plus qu'un projet de certification. En partant d'un besoin concret, faciliter la prise de rendez-vous médicaux pour les patients et la gestion de leur activité pour les médecins, j'ai conçu et développé une plateforme composée de trois applications : une API REST développée avec Symfony, qui centralise les données et l'ensemble des règles métier, et deux interfaces React, l'une destinée aux patients, l'autre aux médecins et aux administrateurs. Le cœur de la plateforme fonctionne aujourd'hui de bout en bout : inscription et authentification sécurisée, recherche de médecins, calcul des créneaux disponibles et réservation, gestion des cabinets, agenda et suivi des rendez-vous du médecin.

Ce projet m'a permis de mettre en pratique l'ensemble des compétences du titre de Développeur Web et Web Mobile. Côté front-end, j'ai appris à maquetter une application à partir du besoin, à concevoir une interface *responsive* pensée *mobile first* et accessible, et à développer des composants React dynamiques qui communiquent avec une API. Côté back-end, j'ai conçu une base de données relationnelle, développé une couche d'accès aux données protégée contre les injections SQL et mis en place une authentification par jetons JWT. J'ai aussi appris à raisonner en termes de sécurité, en considérant que toute donnée reçue d'un client doit être contrôlée et que seule l'API fait foi. Au-delà du code, j'ai organisé mon travail selon la méthode Scrum, versionné le projet avec Git et conteneurisé l'ensemble de l'environnement avec Docker.

Sur le plan personnel, cette année marque un véritable retour vers le métier qui me correspond. Après plusieurs années passées dans d'autres fonctions, j'ai choisi de me consacrer pleinement au développement web. Je souhaite continuer à progresser sur les technologies que j'ai utilisées et me former à de nouveaux outils, comme Next.js, pour développer des applications toujours plus fiables et mieux structurées.

Je tiens à remercier en premier lieu ma femme et ma fille, qui m'ont soutenu tout au long de cette année sabbatique. Leur patience et leurs encouragements m'ont donné l'espoir de continuer à atteindre mes objectifs, et poursuivre mes rêves.

Je remercie infiniment mon formateur, Laurent, pour son accompagnement, ses conseils et sa disponibilité tout au long de la formation.

Je remercie mes collègues de travail, Kevin, David, Alexia et Nizar, pour leur soutien et leur aide au quotidien, qui m'ont permis de progresser, d'apprendre chaque jour une nouvelle chose dans un environnement professionnel et stimulant.

Enfin, je remercie Latifa, Akiyo et Cheikh, mes collègues de formation, pour le temps qu'ils ont consacré aux tests de recette de **Toubib**. Leur regard extérieur m'a permis de détecter des problèmes que je ne voyais pas et d'améliorer la qualité de l'application.

SOURCES ET BIBLIOGRAPHIE

- Genilink Internal Documentation.
- Josh Clark, *Design tactile* (titre original : *Tapworthy: Designing Great iPhone Apps*).
- Steven HOOBER, *Research: How Do Users Really Hold Mobile Devices?* http://bkaprt.com/dft/01-03
- Victor Thuillier, *PROGRAMMEZ EN ORIENTÉ OBJET EN PHP*
- Atlassian, *Manage how work flows in your team-managed project* - https://support.atlassian.com/jira-software-cloud/docs/manage-how-work-flows-in-your-team-managed-project/
- Atlassian, *Working with workflows* - https://confluence.atlassian.com/display/JIRA/Configuring+Workflow
- PHP-FIG, *PSR-1: Basic Coding Standard* - https://www.php-fig.org/psr/psr-1/
- PHP-FIG, *PSR-4: Autoloader* - https://www.php-fig.org/psr/psr-4/
- PHP-FIG, *PSR-12: Extended Coding Style* - https://www.php-fig.org/psr/psr-12/
- PHP-FIG, *PER Coding Style* - https://www.php-fig.org/per/coding-style/
- Symfony, *Coding Standards* - https://symfony.com/doc/current/contributing/code/standards.html
- Symfony, *Best Practices* - https://symfony.com/doc/current/best_practices.html
- RGAA, *Référentiel général d'amélioration de l'accessibilité* - https://accessibilite.numerique.gouv.fr/
- PHP, *Supported Versions* - https://www.php.net/supported-versions.php
- Phoronix, *PHP 8.5 Released* - https://www.phoronix.com/news/PHP-8.5-Released
- Symfony, *Symfony 8.0* - https://symfony.com/blog/symfony-8-0-reaches-its-end-of-maintenance
- React, *React 19.2* - https://react.dev/blog
- CVE Archive, https://www.cve.org/
- GitHub Advisory Database - https://github.com/advisories/

ANNEXE

Les repositories Github sont accessibles depuis le lien: **https://4ntii.github.io/projet-toubib/**
Toubib est disponible sur le lien suivant: **https://toubib.byethost24.com/**

ToubibBO (BackOffice) n'est pas encore hébergé, pour des raisons de coût exorbitant de l'hébergement.